

Statistical Sentence Plausibility Checker

Glenn Barden

April 26, 2010

Originality Avowal

I verify that I am the sole author of this report, except where explicitly stated to the contrary.

Signed: GLENN S. BARDEN

Dated: April 20th 2010

Abstract

Despite modern word processing packages including combinations of both spell and grammar checkers, there is still no wide-spread solution for ensuring that typographical errors are prevented. This results in documents being produced that may contain large amounts of errors completely missed by both the typist and the automated packages within the word processing document. Using statistical analysis of sentences it is possible to reduce the number of errors that pass through such checks.

The purpose of this project was to create an application which could analyse the user's input sentence, and produce an output indicating to the user whether their sentence was correct or not. This concept could then be expanded in the future to become part of all word processing packages to reduce the likelihood of typographical errors occurring.

After analysis of this application with a large sample of test data, it is clear that such a system can reduce the number of incorrect sentences passing as correct, when compared to traditional word processing spell or grammar checkers that are currently available. Although the current system is unlikely to be commercially viable in its current form, the concept proved is accurate enough to lend itself to future expansions and implementation in a larger form.

Acknowledgements

I would like to thank my supervisor, Dr. Jonathan Ginzburg, to whom I am indebted for all his advice and kind words throughout this project.

Contents

1	Introduction	1
1.1	Project Motivation	1
1.2	Main Problems to be Solved	2
2	Background	3
2.1	Key Theoretical Areas	3
2.2	Mathematical Theory	3
2.3	Standard Form Results	7
3	Design	9
3.1	Requirements	9
3.2	Smoothing	10
3.3	Dictionary	12
3.4	Corpus	12
3.5	Final Threshold Value	18
3.6	Specification	18
3.7	User Interface	18
3.8	Test Plan	23
4	Implementation	24
4.1	Getting Input	24
4.2	Spell Check	25
4.3	Likelihood Check	26
4.4	Timers	28
4.5	View Typos	28
4.6	Graphical User Interface	29
4.7	ActionListeners	32
4.8	Interesting Feature	40
5	Optimisation	41
5.1	Interesting Feature Solved	41
5.2	Threshold Values	43
5.3	Improvements	46
5.4	Lite Version	47
6	Integration	50
6.1	Making System Load as Executable File	50
6.2	Distribution	51
6.3	Release	51
7	Testing	52

8	Evaluation	58
8.1	System Accuracy	58
8.2	Spell Check Time Taken	62
8.3	Likelihood Check Time Taken	63
8.4	Requirements Revisited	67
8.5	System Limitations	69
8.6	System Strengths	69
9	Conclusions	71
9.1	Future Work	72
A	Specification Diagrams	74
A.1	Use Case Diagram	74
A.2	State Chart	75
A.3	Class Diagram	76
B	User Interface Designs	77
B.1	Main Screen	77
B.2	Menus	78
C	Test Plan	79
D	Ethics and Standards Used	85
D.1	Ethics	85
D.2	Standards	85
E	Testing Screenshots	89
F	Program Listings	114
F.1	Full System	114
F.2	Lite System	138
G	Dictionary Used	160
H	Corpora Used	161
I	User Guide	162
J	Bibliography	163

List of Figures

1	Lite System User Interface	48
2	Correct Sentences Time To Spell Check	62
3	Incorrect Sentences Time To Spell Check	62
4	Time Taken for UES Likelihood Check	63
5	Time Taken for FES Likelihood Check	64
6	Time Taken for MLE Likelihood Check	64
7	Time Taken for Plus-One Likelihood Check	65
8	Time Taken for Witten-Bell Likelihood Check	65
9	Time Taken for Glenn Smoothing Likelihood Check	66
10	Comparison of the Time Taken by Smoothing Type	66
11	Initial System Use Case Design	74
12	Initial System State Chart Design	75
13	Initial System Class Diagram Design	76
14	User Interface Design	77
15	Menu Design	78
16	Test 1 Screenshot	89
17	Test 2 Screenshot	90
18	Test 3 Screenshot	90
19	Test 4 Screenshots	91
20	Test 5 Screenshots	92
20	Test 5 Screenshots (Continued)	93
21	Test 6 Screenshot	94
22	Test 7 Screenshot	94
23	Test 8 Screenshot	95
24	Test 9 Screenshots	95
24	Test 9 Screenshots (Continued)	96
25	Test 10 Screenshots	97
25	Test 10 Screenshots (Continued)	98
26	Test 12 Screenshots	99
26	Test 12 Screenshots (Continued)	99
27	Test 13 Screenshots	100
28	Test 14 Screenshots	101
29	Test 15 Screenshots	102
29	Test 15 Screenshots (Continued)	103
30	Test 16 Screenshots	104
30	Test 16 Screenshots (Continued)	104
31	Test 17 Screenshots	105
32	Test 18 Screenshots	105
33	Test 19 Screenshot	106
34	Test 20 Screenshot	106
35	Test 21 Screenshot	107
36	Test 22 Screenshot	107
37	Test 23 Screenshot	107
38	Test 25 Screenshots	108

39	Test 33 Screenshot	109
40	Test 34 Screenshot	110
41	Test 35 Screenshot	110
42	Test 41 Screenshots	111
43	Test 42 Screenshot	112
44	Test 43 Screenshot	112
45	Test 44 Screenshot	113

List of Tables

1	Hardware Requirements	11
2	Software Requirements	11
3	Results of Different Sentence Analysis Techniques	16
4	KLM-GOMS Operations	20
5	Negative and Positive Results	41
6	Sentence Analysis Values Using Tool	44
7	Results of Witten-Bell Smoothing	46
8	Testing Results	57
9	Evaluation Results	59
10	Test Plan of the System	84

1 Introduction

The history of the modern day spell checker can be traced back to those which existed in stand alone packages for CP/M computers in the late 1970s[1]. These packages had a very limited lifetime and as the 1980s progressed, the developers of the most popular word processing packages of the day had begun to incorporate spelling checkers into their packages, making the stand alone packages of before redundant. Two of the market leaders of the word processing program with built-in spell checkers at the time were 'WordStar'[2] and 'WordPerfect'[3]. Since then, spell checkers have remained largely the same - as inbuilt packages in word processing programs designed to flag up spelling errors to the user. However, in the last ten years, the focus has moved on to developing algorithms which are capable of recognising a misspelled word, even if the word itself is in the vocabulary, based on the context of the surrounding words. The most notable solution is Andrew Golding and Dan Roth's Winnow-based spelling correction algorithm, published in 1999, which is able to recognise about 96% of context-sensitive spelling errors, in addition to ordinary spelling errors[4]. However, despite the success of Golding and Roth's algorithm, there are still no widespread facilities available within normal packages to prevent erroneous but correctly spelt sentences to be permitted inside the user's text.

1.1 Project Motivation

Despite the advances in spell checking software, which now even include real-time graphical aids, these systems are not immune to allowing completely erroneous sentences pass as correct. The most widespread example of the erroneous sentences which spell checkers will pass as correct is a poem composed by Dr. Jerrold H. Zar in 1991, assisted by Mark Eckman, with an original length of 225 words, and containing 123 incorrectly used words[5]. It is clear by looking at this poem that it is obviously using words incorrectly, but spell checkers will allow it to be passed as correct, due to there being no spelling mistakes.

It is the aim of this project to prevent such an extraordinary amount of errors passing as correct.

The results of this project are of great interest to the field of Computer Science. This an area of development where alot of attention is being focused by the world's largest software creators, and these solutions developed are discussed in Section 1.2. The solution to this problem is also very useful for next word prediction, which is used in speech devices for the disabled, as well as in speech recognition software.

The problem of typographical errors in documents receives a lot of national press coverage when these mistakes result in upset, company embarrassment or financial implications. For example, recently Penguin Group publishers were forced to pulp and reprint a cook book after one recipe listed 'salt and freshly ground black people' instead of the correct 'black pepper'[6]. This mistake cost the company £12,000 to rectify. Other recent examples include a typo in the press release of a spell-checking firm[7], much to their embarrassment, and a

mistake by Hertfordshire workmen resulting in a confusing road marking[8]. All these mistakes cost money to resolve, and have led to the formation of the Typo Eradication Advancement League[9], who attempt to prevent typographic errors going uncorrected.

1.2 Main Problems to be Solved

The main problems which this project hopes to solve are as follows:

1. To stop spelling errors in a sentence passing as correct
2. To prevent typographical errors in a sentence passing as correct
3. To prevent a sentence where incorrect words have been substituted for correct words passing as correct

From this point, these problems will be referred to as **P1**, **P2** and **P3** respectively.

2 Background

2.1 Key Theoretical Areas

There are a number of different theories and strategies that can be employed to make the check of the sentence as thorough and as accurate as possible. The most successful project to date is Andrew Golding and Dan Roth's Winnow-based spelling correction algorithm[4] mentioned in Section 1, but others include the context sensitive spell checker in Google Wave[10], as well as the contextual spelling checker available in the Beta 2 version of Microsoft Word 2007[11].

These systems work by storing a large file of the most likely errors that users have been found to make, such as those used for editing Wikipedia pages[12]. When each word (or two-word string) is typed in by the user, it is checked against the entries within the file. If the string is found in the file then a list of alternatives are suggested (which may also be stored in the file, shown in brackets in the Wikipedia article[12]). This may prevent the most likely errors from occurring, but is a very inflexible method and also depends entirely on the accuracy and size of the file listing the most likely errors. No calculations occur, it is merely a case of database retrieval. This method is unable to stop anything apart from the most likely errors, which is a major flaw, and is a reason why this method is not particularly accurate.

Errors within sentences can be detected by algorithms which examine the words surrounding the errors. For example, while the phrase 'in about fifteen minuets' is perfectly grammatical English, it is a very unlikely combination of words. The program to be created will need to look for low probability combinations like this. As a general overview of the program's functionality, each sentence entered will have to be given a probability based on the likelihood of it's occurrence. This number will then be compared to a pre-defined threshold value for an acceptable sentence within the program. If the number is less than this value, it will be noted as a possible mistake. There are many ways to do this, which will be explained in Section 2.2.

This project will prove the concept that statistical analysis of a sentence can be used to determine the correctness of the sentence. As such, this system will only be designed to analyse a single sentence at a time, although it is possible that this could be expanded in the future if the concept is proved to be successful.

2.2 Mathematical Theory

All material in this section is adapted from other sources. Sections 2.2.1 - 2.2.6 are from Jarafsky and Martin[13], Sections 2.2.7 - 2.2.10 are from Chen and Goodman[14], while Section 2.2.11 and Section 2.2.12 are taken from MacCartney[15].

2.2.1 Uniform Event Space (UES)

This makes each outcome of similar length strings equally likely. For example selecting one word out of the full Oxford English Dictionary, which has 301100

entries, would have a probability of $P = \frac{1}{301100}$. Selecting a five word string would therefore have $301100^5 = 2.4748779 \times 10^{27}$ different possibilities, and hence a probability of $P = 4.0406 \times 10^{-28}$.

However, it is quite obvious that this is a naive way of calculating the likelihood of a sentence as every word's usage in written English varies in a way not taken into account here. It is also a false assumption that because a sentence has more words by definition it is made less likely than a sentence with fewer words, which is the case using Uniform Event Space.

2.2.2 Frequentist Event Space (FES)

This is slightly more sophisticated than the UES method introduced in Section 2.2.1. Instead of random selection, you take the probabilities of a word occurrence from a corpus to derive the frequency of occurrence.

For example if you have three words of probabilities $\frac{1}{10}$, $\frac{1}{5}$ and $\frac{1}{2}$, the frequency of occurrence is calculated as:

$$P = \frac{1}{10} \times \frac{1}{5} \times \frac{1}{2} = \frac{1 \times 1 \times 1}{10 \times 5 \times 2} = \frac{1}{100} \quad (1)$$

The major flaw in this scheme however, is that it allows the words within the string to be in any order and have the same probability, which intuitively is not correct.

2.2.3 Maximum Likelihood Estimation (MLE)

For a fixed set of data and underlying probabilities, MLE picks the values that make the data 'more likely' than any other values would make them. Maximum Likelihood Estimation gives a way to determine a solution in the case of the normal distribution and many other problems. If a uniform prior distribution is assumed over the parameters, the Maximum Likelihood Estimate coincides with the most probable values.

$$P(i^{th} word = word1 | i - 1^{th} word = word2) = \frac{C(\omega_2 \omega_1)}{C(\omega_2)} [13] \quad (2)$$

However, it is important to note that if $p(s) = 0$ then $p(s|A)$ will be zero, and the string s will never be considered. Therefore we need a way to prevent this from happening. This is known as smoothing and is introduced below. Smoothing is a technique for adjusting the Maximum Likelihood Estimate of probabilities to produce more accurate answers. They tend to make distributions more uniform by increasing low values and decreasing high values.

2.2.4 Plus-One Smoothing

First, you must count the number of different words $|V|$ stored in the corpus. You add 1 to the numerator of the MLE and then add $|V|$ to the denominator.

$$P(i^{th} word = word1 | i - 1^{th} word = word2) = \frac{C(\omega_2 \omega_1) + 1}{C(\omega_2) + |V|} [13] \quad (3)$$

Doing this prevents the possibility of a zero answer, as even if $C(\omega_2\omega_1) = 0$ then adding one to this will always result in an answer. However, this technique can still be improved on.

2.2.5 Witten-Bell Smoothing

To do this, you first need to find the following values:

- $T(\omega_{i-1})$ is the number of different words that occur to the right of ω_{i-1}
- $N(\omega_{i-1})$ is the number of all word occurrences to the right of ω_{i-1}
- $Z(\omega_{i-1})$ is the number of bigrams in the current data set, starting with ω_{i-1} that do not occur in the training data. You can also find this by using the formula $Z = |V| - T$

Witten-Bell uses different formulae for zero and non-zero counts.

For zero counts:

$$c(\omega_i|\omega_{i-1}) = \frac{T(\omega_{i-1})}{Z(\omega_{i-1})} \frac{N(\omega_{i-1})}{(N(\omega_{i-1}) + T(\omega_{i-1}))} [13] \quad (4)$$

$$P(\omega_i|\omega_{i-1}) = \frac{T(\omega_{i-1})}{(Z(\omega_{i-1}) \cdot (N(\omega_{i-1}) + T(\omega_{i-1})))} [13] \quad (5)$$

For non-zero counts:

$$c(\omega_{i-1}\omega_i)^{wittenbell} = c(\omega_{i-1}\omega_i) \frac{N(\omega_{i-1})}{(N(\omega_{i-1}) + T(\omega_{i-1}))} [13] \quad (6)$$

$$P^{wittenbell}(\omega_i\omega_{i-1}) = \frac{c(\omega_{i-1}\omega_i)}{(N(\omega_{i-1}) + T(\omega_{i-1}))} [13] \quad (7)$$

2.2.6 Good-Turing Estimate

The basic idea of this type of smoothing is to re-estimate the amount of probability mass to assign to N-Grams with zero or low counts, by looking at the number of N-Grams with higher counts. For this, we would need to calculate the following values:

- N_c - the number of N-Grams that occur c times. This is referred to as the frequency of frequency c
- N_0 is the number of bigrams b of count 0, N_1 is the number of bigrams b of count 1 and so on

The Good-Turing Estimate gives a smoothed count c^* based on the set of N_c for all c , as follows:

$$c^* = (c + 1) \frac{(N_c + 1)}{N_c} [13] \quad (8)$$

The main problem with the Good-Turing Estimate is that the value of c^* required to calculate the exact value can become far too complex, leading most implementations to cap the value with the threshold value k .

2.2.7 Church-Gale Smoothing

This combines the Good-Turing estimate with bucketing. Each N-gram is assigned to one of several buckets based on its frequency predicted from lower-order models. The value of $PML(w_{i-1})PML(w_i)$ determines into which bucket a bigram falls. The Good-Turing estimation is then performed within each bucket.[14]

2.2.8 Jelinek-Mercer Smoothing

Jelinek-Mercer Smoothing uses interpolation between unigram and bigram models. This means that if two strings are assigned the same zero probability, then the formula aims to give the one with the higher unigram count the higher probability. To do this the following recursive formula is used:

$$P_{interp}(w_i|w_{i-n+1}^{i-1}) = \lambda w_{i-n+1}^{i-1} PML(w_i|w_{i-n+1}^{i-1}) + (1 - \lambda w_{i-n+1}^{i-1}) P_{interp}(w_i|w_{i-n+2}^{i-1}) [14] \quad (9)$$

2.2.9 Katz Smoothing

As with in Good-Turing, we are computing adjusted counts with Katz Smoothing. Bigrams with non zero count r are discounted according to discount ratio d_r , which is approximately $\frac{r^*}{r}$. The count mass subtracted from non zero counts is redistributed among the zero-count bigrams according to next lower-order distribution. The adjusted count is calculated as follows:

$$c_{katz}(w_{i-1}^i) = d_r r \text{ if } r > 0 \text{ or } \alpha(w_i - 1) PML(w_i) \text{ if } r = 0$$

$\alpha(w_i - 1)$ is chosen so that $\sum w_i c_{katz}(w_{i-1}^i) = \sum w_i c(w_{i-1}^i)$ as below:

$$\alpha(w_{i-1}) = \frac{1 - \sum w_i : c(w_{i-1}^i) > 0 P_{katz}(w_i|w_{i-1})}{1 - \sum w_i : c(w_{i-1}^i) > 0 PML(w_i)} [14] \quad (10)$$

And finally we compute $P_{katz}(w_i|w_{i-1})$ from the corrected count by normalising:

$$P_{katz}(w_i|w_{i-1}) = \frac{c_{katz}(w_{i-1}^i)}{\sum w_i c_{katz}(w_{i-1}^i)} [14] \quad (11)$$

We want the discounts to be proportional to the Good-Turing discounts, and we also want the total count mass saved to equal the count mass which Good-Turing assigns to zero counts.

2.2.10 Absolute Discounting

Very much like the Jelinek-Mercer method, this involves the interpolation of higher and lower order models, but instead of multiplying the higher order

PML by a λ we subtract a fixed discount of $\delta \in [0, 1]$ from each non-zero count as follows:

$$P_{abs}(w_i|w_{i-n+1}^{i-1}) = \frac{\max\{c(w_{i-n+1}^i) - \delta, 0\}}{\sum_{wi} c(w_{i-n+1}^i)} + (1 - \lambda w_{i-n+1}^{i-1}) P_{abs}(w_i|w_{i-n+2}^{i-1}) [14] \quad (12)$$

To make this sum to one, the following is performed:

$$1 - \lambda w_{i-n+1}^{i-1} = \frac{\delta}{\sum w_i c(w_{i-n+1}^i) N_1 + (w_{i-n+1}^{i-1})^\bullet} [14] \quad (13)$$

δ is found using held-out estimation.

2.2.11 Kneser-Ney Smoothing

This is an extension of absolute discounting with a unique way of constructing the lower-order (backoff) model. The idea of this smoothing is that the lower-order model is significant only when count is small or zero in the higher-order model, and so should be optimised for that purpose. For example, 'San Francisco' is common - but the word 'Francisco' only occurs after the word 'San'. 'Francisco' will therefore get a high unigram probability, and so absolute discounting will give a high probability to 'Francisco' appearing. It would be better to give 'Francisco' a low unigram probability, because the only time it occurs is after 'San'. This is represented as follows:

$$N_1 + (\bullet w_i) = |\{w_{i-1} : c(w_{i-1} w_i) > 0\}| N_1 + (\bullet\bullet) = \sum_{wi} N_1 + (\bullet w_i) [15] \quad (14)$$

Let lower order distribution be:

$$PKN(w_i) = \frac{N_1 + (\bullet w_i)}{N_1 + (\bullet\bullet)} [15] \quad (15)$$

And when this is all put together:

$$PKN(w_i|w_{i-n+1}^{i-1}) = \frac{\max\{c(w_{i-n+1}^i) - \delta, 0\}}{\sum w_i c(w_{i-n+1}^i)} + \frac{\delta}{\sum w_i c(w_{i-n+1}^i)} N_1 + (w_{i-n+1}^{i-1})^\bullet PKN(w_i|w_{i-n+2}^{i-1}) [15] \quad (16)$$

2.2.12 Modified Kneser-Ney Smoothing

This was introduced by Chen and Goodman. It has much better performance than the traditional Kneser-Ney Smoothing as it introduces two-counts and does not use data based on training counts[15].

2.3 Standard Form Results

Answers to the likelihood check are likely to be expressed in standard form (also known as scientific form or exponential notation). This is a way of representing

numbers that are, in this case, too small to be conveniently written in traditional decimal form. In Java, standard form results are expressed in scientific notation as follows:

$$aEb \tag{17}$$

Where $1 \leq a < 10$ and $b < 0$. This can be 'translated' as $a \times 10^b$. Some examples of standard form results and their decimal equivalents are listed below:

- $0.01 = 1 \times 10^2 = 1.00E - 2$
- $0.001 = 1 \times 10^3 = 1.00E - 3$
- $0.0001 = 1 \times 10^4 = 1.00E - 4$
- $0.00001 = 1 \times 10^5 = 1.00E - 5$
- $0.000001 = 1 \times 10^6 = 1.00E - 6$

As the system will give answers in scientific notation, from here this notation will also be used in this document.

3 Design

3.1 Requirements

3.1.1 List of Requirements

The requirements for the project have been defined by the three problems which the system is designed to solve, which are listed in Section 1.2.

For ease of reference the program requirements have been split into four sections as follows:

1. Basic Program Functions
 - a. To allow the user to type a sentence for checking
 - b. To allow the user to submit the sentence for checking
 - c. To allow the user to enter another sentence if the one submitted has been rejected
 - d. To have a clear all button which returns the program to its initial state
 - e. To prevent the user using the likelihood check button until the submitted sentence is spelt correctly
 - f. To prevent the user entering another sentence if the one submitted has been accepted, unless the clear all option is selected
 - g. To have a button or command to close/terminate the program
 - h. To allow the user to copy/paste text into the input box
 - i. To allow the user to open a text file for input
 - j. To allow the user to save their input as a text file
 - k. To have the ability to print the users input
 - l. To have the ability to print the system generated output
 - m. To have a help section for the user
 - n. To provide the user with undo and redo options to remove/repeat the action most recently performed
 - o. To provide the user with a time taken output for spelling and likelihood checks
2. Spell Check Functionality (solve **P1**)
 - a. To produce a visual output stating whether or not all the words in the sentence are spelt correctly
 - b. To have an option to view all the incorrectly spelt words in the sentence
 - c. To show the number of incorrectly spelt words in the sentence
3. Likelihood Check Functionality (solve **P2** and **P3**)

- a. To produce a visual output stating whether or not the sentence is seen as 'likely' or 'unlikely'
 - b. To present numerical data of the sentence likelihood
 - c. To produce a visual output providing the most likely typographic or grammatical errors within the submitted sentence
 - d. To provide a drop down list where the user can change the type of smoothing used in the likelihood check
4. User Requirements
- a. To be able to back up the system easily
 - b. The system must be easy to use
 - c. The system must run quickly
 - d. The system must not have excessive delays between input and output
 - e. The system must be small enough in size to be easily transferable between machines
 - f. The system must be programmed in a way to easily allow extensions to the code
 - g. The system must run from double clicking a single item on the user's desktop

3.1.2 Hardware

The system will be constructed upon my home computer. Therefore to enable this to be possible, it must pass the following hardware requirements, listed in Table 1. The requirements are necessary to allow the machine to run it's operating system, as well as the latest versions of JCreator and Java.

3.1.3 Software

The computer must also pass the following software requirements, listed in Table 2, necessary to allow the system to be programmed as intended.

3.2 Smoothing

It is clear after evaluating the different types of smoothing in Section 2.2 that no one type of smoothing is going to be able to fulfil all of the user requirements perfectly. This is because, as a general observation, the more accurate the result obtained, the slower it will be to obtain it. Therefore, if the system takes too long to calculate the result, the system will fail requirements 4c and 4d which state that the system must run quickly and not have excessive delays between user input and the system outputting a result.

Component	Requirement	Passed/Failed
Processor	166MHz	Passed, the computer has 2.01GHz
RAM	32MB	Passed, the computer has 512MB
Available Disk Space	125MB	Passed, the computer has 65GB free
Monitor	Must have a monitor	Passed, a 17' TFT monitor
Mouse	Must have a mouse	Passed
Keyboard	Must have a keyboard	Passed
USB port	Must have at least 1 free for back ups if desired	Passed, 8 USB ports are installed of which 4 are free
Internet connection	Must have to enable software updates	Passed, broadband connection at 4MB/sec download

Table 1: Hardware Requirements

Software	Requirement	Passed/Failed
Operating System	Windows 2000 or later, Linux Red Hat 2.1 or later	Passed, the system will be programmed using Linux Mint 8
JCreator LE	Must have version 4 or later installed	Passed, JCreator LE Version 4.50.010 is installed
Java Environment	Must have latest Java version installed	Passed, Java Version 6, Update 16 is installed

Table 2: Software Requirements

As different users will have different requirements of the system, it is important to present the user with an option of different types of smoothing, so that they can obtain a quick result or a more accurate result depending on what they require. Therefore, it is proposed that the system will contain the following types of smoothing from Section 2.2, with a justification for each type:

- Uniform Event Distribution (UED) - will be incredibly quick to calculate, so will present the user with an incredibly quick answer
- Frequentist Event Distribution (FED) - will also be a very quick calculation, but will be slightly more accurate than with UED
- Maximum Likelihood Estimation (MLE) - will be more accurate than the

previous two, still quite quick to calculate and is required for Witten-Bell smoothing

- Plus-One smoothing - is more accurate than MLE which we have already determined the system requires, and takes minimal more time to compute an answer
- Witten-Bell smoothing - will be the slowest to calculate, but will be the most accurate

One of the advantage of these five types of smoothing is that they all appear to 'build upon' the previous types of smoothing, which increases system efficiency and also makes it possible to pass all of the system requirements listed in Section 3.1.1

3.3 Dictionary

The dictionary in this project will not necessarily be the final dictionary used by the end users. One of the strengths of this system is the fact that the dictionary can be changed or edited by the end user to suit their more specific needs. The size or completeness of the dictionary is not important at this stage - it is more important that the dictionary is thorough enough to give an accurate representation of the correctness of the system. To save time a pre-defined English dictionary was downloaded from the Internet which would provide approximately 100,000 entries as this would show both system correctness, as well as system stability for a large amount of words. A link to the whole dictionary can be found in Appendix G.

Any alternative dictionary to be used in replacement of this file must follow the following formatting standards:

- To be stored as a plaintext *.txt file
- For each entry to be separated by a new line or return character
- The file must be called 'Dictionary' and placed in the home directory of the system

3.4 Corpus

There are three types of corpus that will be required for the system:

- A single word corpus (unigram) of the form:
 - To be stored as a plaintext *.txt file
 - For each entry to be stored as following `word1 % occurrences_of_word1`
 - For each entry to be separated by a new line or return character
 - The file must be called 'SingleCorpus' and placed in the home directory of the system

- A two word corpus (bigram) of the form:
 - To be stored as a plaintext *.txt file
 - For each entry to be stored as following `word1 <<space>> word2 % occurrences_of_word1_word2`
 - For each entry to be separated by a new line or return character
 - The file must be called 'DoubleCorpus' and placed in the home directory of the system
- 'T' corpus for Witten-Bell calculations of the form:
 - To be stored as a plaintext *.txt file
 - For each entry to be stored as following `word1 % Tcount_of_word1`
 - For each entry to be separated by a new line or return character
 - The file must be called 'T' and placed in the home directory of the system
 - Each 'T' value is to correspond with the 'Double Corpus' value for that word, subtracted from the total number of words in the corpus used

Again, the size and completeness of the corpora is not important at this stage - it is more important that they are thorough enough to give an accurate representation of the correctness of the system. It is also vital that the corpora are from the same source, as corpora from different sources will produce less accurate results. Therefore reduced versions of the freely available Brown corpus were used - with the single word list comprising of 2000 entries from The Virtual Language Centre[16] and the double word list comprising of 24000 entries made available by Cognitive Science department at Osnabruck University[17]. Even though these lists may not be complete and may result in some 'zero probability' branches in the calculations due to some word phrases not being present, this will once again serve to confirm the correctness of the system.

3.4.1 Differences Between Spoken and Written Language Corpora

The study of Natural Language Processing such as that explored within this project could not be complete without mentioning it's usage in Automated Speech Recognition (ASR) systems. Much like the system to be created here, ASR systems use bigrams, but in this instance not to calculate the probability of the string in question. These systems evaluate what word is most likely to come next when spoken in a sentence. This is particularly useful for language devices for the disabled, as well as for creating automated lecture notes or automated meeting notes. By predicting what are likely next words, it reduces the amount of errors that can occur when the automated notes are created. For example - if the system hears the phrase 'a panda' and the next word is difficult to determine between 'cub' or 'club', the system can ascertain that the

word 'cub' is most likely to follow, and return the string 'the panda cub'. To do this however, corpora different from those used in this project are required. In written text the string 'the the the the' is very unlikely to occur, unless as a direct quote from a spoken sentence! However, in spoken language, it is plausible that either someone with a stammer, or someone just struggling to think of the next word may make this utterance. To compensate for this, as well as other filled pauses (um/uh) and the usage of different pronunciations to indicate the most likely next word (the/thiy; a/ay), different corpora are used. These corpora differ greatly from the Brown corpus, to such an extent that replacing the corpora provided in this project with those of the spoken word may actually yield surprisingly incorrect results.

One of the main reasons for this is that our everyday spoken language is surprisingly sparse when compared to written language. Normal spoken word corpora are far smaller than those of the typed word for this very reason. Also, the use of filled pauses as explained above, as well as repeating single words, which would be eliminated in most written documents by spell checkers, can create large differences between the two sets of corpora.

It is not inconceivable that a project such as the one being undertaken here, could be extended in the future to meet the needs of an ASR system in an attempt to increase their notorious unreliability.

3.4.2 Threshold Value

Throughout this project, it is clear that for a sentence to be labelled 'likely' or 'unlikely' its probability has to be compared with a pre-defined 'threshold' value. If the sentence's probability is less than this threshold, it must be labelled 'unlikely', and the converse if its probability is equal to or higher than the threshold. Therefore a key part of the accuracy of this system comes down to the accuracy of the threshold value.

Therefore, it is important that we begin with an idea of what this threshold value is likely to be, before perfecting it in the optimisation process in Section 5. To begin with, we will use one value for all types of smoothing, honed to Witten-Bell's results. We will then give a separate value for each type of smoothing during the optimisation process, to increase the accuracy of the system.

We will do this by analysing the values of twelve sentences which can be seen as varyingly 'likely', and twelve sentences which can be seen as varyingly 'unlikely'.

The twenty-four sentences are as follows:

Likely

1. I made the plans
2. The man is small
3. Is this the train?
4. He shouted
5. In this instance

6. Add a dozen
7. In that instance
8. What is going on?
9. The event of the week
10. I ate the whole thing
11. She smiled at me
12. A fair amount

Unlikely

13. Be there in fifteen five minutes
14. I have on idea
15. He ate his best friend
16. The stream flew quickly over
17. He went to his clubbed
18. Add flour too the mixture
19. The the the the
20. Folder circuit you do have any mates
21. River sky building no way mr
22. Gingerbread flows quick over the rocks
23. Charge broken in the pen
24. What jacket do you eat

3.4.3 Results

Sentence	MLE	Plus One	Witten-Bell
Likely			
1	5.15E-09	2.59E-15	4.69E-09
2	5.45E-08	5.25E-14	4.98E-08
3	2.72E-09	1.09E-14	2.45E-09
4	1.26E-03	1.30E-05	0.001226
5	3.93E-05	4.86E-09	3.71E-05
6	1.48E-04	3.51E-10	1.37E-04
7	0.00E+00	1.25E-10	2.40E-16
8	1.54E-05	3.51E-10	1.37E-04
9	2.70E-09	1.29E-13	1.43E-05
10	0.00E+00	1.36E-21	1.62E-15
11	4.29E-06	5.64E-15	4.04E-06
12	0.00E+00	2.10E-11	1.52E-15
Unlikely			
13	0.00E+00	5.87E-27	5.00E-32
14	0.00E+00	2.15E-15	2.89E-24
15	0.00E+00	1.00E-24	4.32E-31
16	0.00E+00	1.96E-23	2.55E-31
17	0.00E+00	1.02E-20	0.00E+00 ¹
18	0.00E+00	7.48E-24	6.00E-27
19	0.00E+00	8.16E-19	6.24E-23
20	0.00E+00	4.05E-31	4.33E-46
21	0.00E+00	2.00E-29	2.31E-38
22	0.00E+00	3.35E-28	9.81E-38
23	0.00E+00	5.49E-21	2.47E-30
24	0.00E+00	1.27E-22	5.88E-31

Table 3: Results of Different Sentence Analysis Techniques

3.4.4 Analysis of Values

Looking at Table 3, it is clear that we can draw the following conclusions:

- If we were to use the lowest Witten-Bell value in the 'likely' section as our threshold value (1.52E-15):

¹The zero Witten-Bell value in the table above is due to the number being so low that it could not fit on the screen of a normal scientific calculator. It is possible that the system may be able to display this number. For simplicities sake, the value has been taken as zero for the purposes of this table, even though this is not entirely accurate.

- 100% of likely sentences would pass as likely under Witten-Bell smoothing
 - 0% of likely sentences would pass as unlikely under Witten-Bell smoothing
 - 0% of unlikely sentences would pass as likely under Witten-Bell smoothing
 - 100% of unlikely sentences would pass as unlikely under Witten-Bell smoothing
- If we were to use the highest Witten-Bell value in the 'unlikely' section as our threshold value (6.24E-23):
 - 100% of likely sentences would pass as likely under Witten-Bell smoothing
 - 0% of likely sentences would pass as unlikely under Witten-Bell smoothing
 - 0% of unlikely sentences would pass as likely under Witten-Bell smoothing
 - 100% of unlikely sentences would pass as unlikely under Witten-Bell smoothing
- If we were to use the lowest Plus-One value in the 'likely' section as our threshold value (1.36E-21):
 - 100% of likely sentences would pass as likely under Plus-One smoothing
 - 0% of likely sentences would pass as unlikely under Plus-One smoothing
 - 25% of unlikely sentences would pass as likely under Plus-One smoothing
 - 75% of unlikely sentences would pass as unlikely under Plus-One smoothing
- If we were to use the highest Plus-One value in the 'unlikely' section as our threshold value (2.15E-15):
 - 58% of likely sentences would pass as likely under Plus-One smoothing
 - 42% of likely sentences would pass as unlikely under Plus-One smoothing
 - 0% of unlikely sentences would pass as likely under Plus-One smoothing
 - 100% of unlikely sentences would pass as unlikely under Plus-One smoothing

- With MLE - if we were to use zero as our value:
 - 75% of likely sentences would pass as likely under MLE
 - 100% of unlikely sentences would pass as unlikely under MLE

3.5 Final Threshold Value

As both the highest 'unlikely' Witten-Bell value, and lowest 'unlikely' Witten-Bell value both produce 100% accurate results on this reduced data, a point half way between the two will be chosen as the initial value for the threshold. Therefore the number **0.000000000000000119861639910954** will be chosen as the initial value, as this is the half-way point between these two values. This value will be refined in the optimisation process in Section 5, and other values will be chosen for each type of smoothing.

3.6 Specification

3.6.1 UML Diagrams

For ease of reference, all UML specification diagrams are contained within Appendix A.

3.7 User Interface

For ease of reference, the final designs for the User Interfaces are contained within Appendix B.

3.7.1 Rationale For Design

When designing the user interface and the general functionality the following things were considered:

- To make the system as user friendly as possible
 - Cut/Copy/Paste
 - Select all
 - Tooltips for blind and inexperienced users
 - Shortcut buttons for experienced users
 - Easily understood commands
 - To reduce GOMS-KLM time
- To make sure the system provides all the services an experienced or power user would expect
- To make the system as easy to extend as possible
 - Coded to easily allow extensions

- Coded so that experienced or power users can edit the dictionary and corpora to adjust to their accuracy requirements
- Java will be used to build the system as it was the language in which I have most knowledge and lends itself to the Object-Oriented design that this system will portray. As the system was to be built following the BCS Code of Good Practise[26], it was important to ensure that Section 2 (act professionally as a specialist) was followed, and as a Java specialist and knowing the boundaries of my ability with it, it was important to make sure I used it within my own abilities. Ethics and standards used in the project are discussed further in Appendix D.

3.7.2 Keyboard Shortcuts

The following keyboard shortcuts will be used within the system to aid with user navigation:

- ALT-F4 - Exit
- CTRL-O - Open
- CTRL-S - Save
- CTRL-P - Print Input
- CTRL-Z - Undo
- CTRL-Y - Redo
- CTRL-X - Cut
- CTRL-C - Copy
- CTRL-V - Paste
- CTRL-A - Select All
- CTRL-K - View All Mistakes
- CTRL-T - View Possible Typos
- CTRL-H - View User Guide

3.7.3 KLM-GOMS Analysis

The objective of this analysis is to access the efficiency of a user interface by estimating the time that is required to accomplish certain tasks with a given user interface. This relies on dividing goals into the basic operations described in Table 4, each of which is given a corresponding time estimate[22]. Therefore it is possible to estimate the time taken by a user to accomplish tasks with the User Interface designed. The ten most likely user actions were then listed and

Operation	Meaning	Typical Time Estimate (seconds)
K	Typing a single keystroke	0.35
P	Pointing at an object on the screen with the mouse	1.10
H	Switching hands between mouse and keyboard	0.40
D	Dragging the mouse	<i>Varies</i>
B	Clicking a mouse button	0.10
M	Mentally preparing for a physical action	<i>Varies</i>
R	System Response	<i>Varies</i>

Table 4: KLM-GOMS Operations

tested for each of the potential interfaces. This is how the final User Interface shown in Appendix B were decided upon. Rejected designs are described in Section 3.7.5. For this analysis, keyboard shortcuts were not used - as this would mean no matter how poorly designed the interface, the result would be the same for each, which defies the point of the analysis.

- Enter a four letter sentence
 - $M + P + B + M + 4K = 2M + P + B + 4K$
 - $1.1 + 0.1 + 1.4 = 2.6\text{seconds}$
- Spell check a four letter sentence
 - $M + P + B + M + 4K + M + P + B + R = 3M + 2P + 2B + 4K + R$
 - $2.2 + 0.2 + 1.4 = 3.8\text{seconds}$
- Likelihood check a four letter sentence
 - $M + P + B + M + 4K + M + P + B + R + M + P + B + R = 4M + 3P + 3B + 4K + 2R$
 - $3.3 + 0.3 + 1.4 = 5.0\text{seconds}$
- Open a file in home directory
 - $M + P + B + M + P + B + M + P + B + B + R = 3M + 3P + 4B + R$
 - $3.3 + 0.4 = 3.7\text{seconds}$
- Save a file in home directory
 - $M + P + B + M + P + B + M + P + B + B + H + 4K + R = 3M + 3P + 4B + H + 4K + R$

- $3.3 + 0.4 + 0.4 + 1.4 = 5.5\text{seconds}$
- Print an input or output with default printer and default options
 - $M + P + B + M + P + B + M + P + B + R + R = 3M + 3P + 3B + 2R$
 - $3.3 + 0.3 = 3.6\text{seconds}$
- Use any shortcut key
 - $M + K + K + R = M + 2K + R$
 - 0.7seconds
- Change the smoothing type
 - $M + P + B + R + M + P + R + M + P + B + R = 3M + 3P + 2B + 3R$
 - $3.3 + 0.2 = 3.5\text{seconds}$
- View time taken for any check
 - $M + P + B + R + M + P + R + M + P + B + R = 3M + 3P + 2B + 3R$
 - $3.3 + 0.2 = 3.5\text{seconds}$
- Clear a sentence
 - $M + P + B + R + M + P + B + R = 2M + 2P + 2B + 2R$
 - $2.2 + 0.2 = 2.4\text{seconds}$

3.7.4 Heuristic Evaluation

To further assess the User Interface, Nielsen and Molich's nine-point assessment of User Interfaces has been used[23]. The results of each check for the finalised User Interface design are below:

1. Contain simple and natural dialog - Screens do not contain any irrelevant or rarely used information. Such tasks will be hidden away in menus.
2. Speak the user's language - The concepts and words used are familiar to the user.
3. Minimise user memory load - It has been attempted that the words used on menus and buttons shall mean that the user does not have to remember anything new for the system. Default shortcuts have been used to aid this.
4. Be consistent - Similar features (such as drop down menus) work in similar ways to each other.
5. Provide feedback - By using the brightly coloured tick and cross, feedback is glaringly obvious to the user.

6. Provide clearly marked exits - The default exit option will be the 'X' in the top right of the screen, and the exit option will also be provided through the 'File' menu and through the 'ALT-F4' shortcut.
7. Provide shortcuts - Detailed in Section 3.7.2.
8. Provide good error messages - Will be provided in default error or information boxes.
9. Prevent errors - All errors should be handled by the system without presenting this to the user.

3.7.5 Alternative Designs

Other potential designs included the following:

- Addition of shortcut toolbar
 - Did not reduce GOMS-KLM time enough to justify
 - Increase memory overheads
 - May result in increased mistakes as user selects button by mistake
- Have a clear all button on the main screen
 - Would reduce GOMS-KLM time enough to justify
 - May result in the user resetting everything by mistake which is incredibly frustrating and reason not to implement this button
- To have the output presented in a dialogue box as opposed to having its own pane
 - Less intuitive
 - Also, after the user is alerted to any mistakes would be unable to change them unless memorised all the information inside the dialogue box with regards to what was wrong as the box would have to be hidden before the changes made
- To have the likelihood and spell checks interactive as the user enters the sentence, and stop them as soon as a spelling mistake or unlikely sentence is entered
 - This would vastly increase system overheads
 - This would also prevent the user entering the whole sentence that they wish to check if there is a mistake made, which prevents the user requirement 1a being passed
 - It may be the case that the first two words the user enters are unlikely but the rest of the sentence is very likely. However, the system would already have prevented input and so there is no way the sentence could then be passed as correct

- Would also prevent the pasting in of input which would fail system requirement 1h

3.8 Test Plan

Table 10 within Appendix C shows the test plan which was to be followed to ensure that the system was working as planned. These tests were carried out and their results shown in Section 7.

It is also intended to check the accuracy of the system results by comparing the results obtained against each of the sentences manually calculated in Table 3. This will ensure that the system is producing correct output results.

4 Implementation

All code in the following section is written exclusively by myself, unless directly stated to the contrary.

The system was constructed using the BCS Code of Conduct[25] and the Code of Good Practise[26]. The sections of these documents which relate to the construction of this system are explained in detail in Appendix D.

4.1 Getting Input

4.1.1 Original Solution

The original solution was to get the user's input from the command line using a `BufferedReader` and pass this to the system by setting the input from here as a string named `enteredSentence` and then performing any operations on the string as required.

```
System.out.print("Enter your string: ");
BufferedReader br = new BufferedReader(new InputStreamReader
    (System.in));
br.toString();
String enteredSentence = null;
try {
    enteredSentence = br.readLine();
}
catch (IOException ioe) {
    System.out.println("IO error");
    System.exit(1);
}
```

4.1.2 Link to JTextArea

There was no reason to change the above code until the GUI was implemented, as described in Section 4.6. When the GUI was implemented however, it was clear that the input could no longer be taken from the command line, and had to come from the `JTextArea` called `inputBox`, and was submitted on the press of the `spellCheck` or `likelihoodCheck` buttons. The following is the code used to perform these actions:

```
// Take text from inputBox on spellCheck button click
public void actionPerformed(ActionEvent event){
    if (event.getSource () == spellCheck){
        String enteredSentence = inputBox.getText();
    }
}
// Take text from inputBox on likelihoodCheck button click
if (event.getSource () == likelihoodCheck){
    String acceptedEnteredSentence = inputBox.getText();
}
```

4.2 Spell Check

4.2.1 Original Solution

The original solution involved saving each word of the dictionary in an `ArrayList` of `Strings` as follows:

```
ArrayList<String> dictionary = new ArrayList<String>();  
dictionary.add("a");  
dictionary.add("abba");
```

The spell check was then performed by comparing each word of the string `enteredSentence` iteratively against each entry in `dictionary` and then accepting the string if it were contained, and rejecting it if it wasn't, as follows:

```
if (dictionary.contains(enteredSentence[i])){  
    System.out.println(enteredSentence[i] + " is in the list!");  
}  
else {  
    System.out.println(enteredSentence[i] + " is not in the list!");  
}
```

4.2.2 Importing Text Dictionary

In the end, it was clear that using an `ArrayList` to store a large dictionary would be both inefficient and impractical for addition and removal of items. Therefore there were two options - to use a database or to import the dictionary from a secondary external source. In the end it was decided that storing the dictionary as a text file would more than solve the inefficiency problem, and would also be much more practical than a database as it would take up a lot less space and be a lot quicker to create. It was also make it very simple for end users to easily edit the dictionary to their own needs. Therefore, for every word in `enteredSentence`, the system would have to compare this to every entry in the `dictionary.txt` file, shown in full in Appendix G, and if every word could be matched, pass the sentence. This was done as follows, and was adapted from the code available at:

```
http://www.java-tips.org/java-se-tips/java.io/  
how-to-read-file-in-java.html  
//Before button press if loop  
File dictionary = new File("Dictionary.txt");  
StringBuffer contents = new StringBuffer();  
BufferedReader dictionaryReader = null;  
//Once spellCheck button has been pressed  
try{  
    dictionaryReader = new BufferedReader(new FileReader(dictionary));  
    String text = null;  
    while ((text = dictionaryReader.readLine()) != null){  
        contents.append(text).append(System.getProperty("line.  
separator"));  
    }  
}
```

```
    }
    catch (FileNotFoundException e){
    }
    catch (IOException e){
    }
    finally{
        try{
            if (dictionaryReader != null){
                dictionaryReader.close();
            }
        }
        catch (IOException e){
            e.printStackTrace();
        }
    }
    //Compare words iteratively with dictionary
    String[] words = wholeSentence.toLowerCase().split (" ");
    for (int i=0; i < words.length; i++)
    //Empty sentence
    if (words[i].isEmpty()){
        wrongWords.add("EMPTY STRING");
    }
    //If word is in dictionary, add to correctWords ArrayList
    else if (dictionary.contains(words[i])){
        correctWords.add(words[i]);
    }
    //If word is not found in dictionary, add to wrongWords ArrayList
    else{
        wrongWords.add(words[i]);
    }
}
```

4.3 Likelihood Check

4.3.1 Importing Text Corpora

With the corpora, the same method used in Section 4.2.2 was used, with the difference that the system wasn't just trying to find if a word were there or not. Once the word was found within the corpus, the string had to be returned, and split by the delimiter character and the number returned. The string was split by delimiter as follows:

```
String delimiter = "%";
while ((line2 = brSingleDictionary.readLine()) != null){
    numeratorFES=(line2.split(delimiter));
}
```

4.3.2 Formulae For Each Smoothing Type

The following shows very basically the formulae used for each type of smoothing. Of course, this is not the full detail of each type of smoothing. To see this, Appendix F contains the full code for each type of smoothing.

- UES

```
int length = numberOfWordsInSentence;
String convert;
convert = Integer.toString(length);
power = Double.parseDouble(convert);
UES = (1.0/Math.pow(V,power));
```
- FES

```
numeratorFES=(line2.split(delimiter));
if(numeratorFES[0].equals(wordsFES[j])){
numeratorIntFES = Double.parseDouble(numeratorFES[1]);
FES = FES*(numeratorIntFES/sumOfSingleCorpus);
counterFES++;
break;
}
else if (!numeratorFES[0].equals(wordsFES[j])) {
leastLikelyString = w0w1;
}
```
- MLE

```
if(denominator[0].equals(w0)){
denominatorInt = Double.parseDouble(denominator[1]);
MLE = MLE*(numeratorInt/denominatorInt);
counter++;
break;
}
else{
denominatorInt = 0.0;
}
```
- Plus-One

```
if(numerator[0].equals(w0w1)){
numeratorInt = Double.parseDouble(numerator[1]);
break;
}
else{
numeratorInt = 0.0;
}
if(denominator[0].equals(w0)){
counter2 = 0;
denominatorInt = Double.parseDouble(denominator[1]);
```

```
plusOne = plusOne*((numeratorInt+1)/(denominatorInt+V));  
break;  
}  
else if(!denominator[0].equals(w1)){  
counter2++;  
}
```

- Witten-Bell
//Non-Zero Count
WittenBellCalculator =
WittenBellCalculator*(countValue/(NValueInt+TValueInt));
//Zero Count
WittenBellCalculator =
WittenBellCalculator*(TValueInt / ((ZValueInt*NValueInt)
+TValueInt));

4.4 Timers

To enable the system to provide the user with the time taken for spell checks and likelihood checks, two variables per check were introduced into the system as follows:

```
//Spelling timers  
double startSpellTime;  
double elapseSpellTime;  
//Likelihood timers  
double startLikelihoodTime;  
double elapseLikelihoodTime;
```

At the start of both the spell and the likelihood checks the following assignments took place, to note the time that the check began:

```
startSpellTime = System.currentTimeMillis(); //Start timer  
startLikelihoodTime = System.currentTimeMillis(); //Start likelihood  
timer
```

At the end of each check the total time taken was calculated by subtracting the start time from the current time, as follows:

```
elapseSpellTime = System.currentTimeMillis()-startSpellTime; //End  
timer and subtract from start time  
elapseLikelihoodTime = System.currentTimeMillis()-startLikelihoodTime  
; //End timer and subtract from start time
```

These results were then added to the 'Time Taken' menu options as an Action-Listener, described in Section 4.7.10.

4.5 View Typos

The idea of this option is just to show the user the least likely two word string combination within their entered text. As each type of smoothing calculates the probability of each two word substring within the string, it is clear that each

result is already calculated by the system, and that all it has to do is display the lowest calculated. The following segment of code is taken from the Witten-Bell calculator and gives an example of how this was achieved:

```
if ((TValueInt / ((ZValueInt*NValueInt)+TValueInt))<leastLikely){
leastLikely = (TValueInt / ((ZValueInt*NValueInt)+TValueInt));
leastLikelyString = WBOWB1;
}
```

The lowest result was then added to the 'View Possible Typos' menu option as an ActionListener, which is described in Section 4.7.9.

4.6 Graphical User Interface

After the functionality of the system was complete, the User Interface was created. The following steps were performed in the following order.

4.6.1 Creation of JFrame

The JFrame was created with the name `frame`. It was set to appear, and to have the 'spell check' icon in the top left hand corner of the window.

```
JFrame frame = new GUI();
frame.getRootPane().setWindowDecorationStyle(JRootPane.FRAME);
frame.setUndecorated(true);
frame.setVisible(true);
frame.setIconImage(Toolkit.getDefaultToolkit().getImage
("frameicon.png"));
```

4.6.2 Layout

After the JFrame was created, both the JTextArea and the JButtons were created so that they could be placed on the frame as follows:

```
//Create inputFlow Panel
JPanel inputFlow = new JPanel();
inputFlow.setLayout (new BorderLayout());
inputFlow.add(inputScrollPane, BorderLayout.CENTER);
//Create outputFlow Panel
JPanel outputFlow = new JPanel();
outputFlow.setLayout (new BorderLayout());
outputFlow.add(outputScrollPane, BorderLayout.CENTER);
outputFlow.add(colourOutput, BorderLayout.EAST);
//Create buttonPane Panel
JPanel buttonPane = new JPanel();
buttonPane.setLayout (new FlowLayout());
buttonPane.add(spellCheck);
buttonPane.add(likelihoodCheck);
//Combine nameLabel and inputFlow Panel to create topPane Panel
```

```
JPanel topPane = new JPanel();
topPane.setLayout(new BorderLayout());
topPane.add(nameLabel, BorderLayout.NORTH);
topPane.add(inputFlow, BorderLayout.CENTER);
//Combine outputFlow and buttonPane to create bottomPane Panel
JPanel bottomPane = new JPanel();
bottomPane.setLayout(new BorderLayout());
bottomPane.add(outputFlow, BorderLayout.CENTER);
bottomPane.add(buttonPane, BorderLayout.SOUTH);
//Combine topPane and bottomPane to create whole
JPanel whole = new JPanel();
whole.setLayout(new GridLayout(2,1));
whole.add (topPane);
whole.add (bottomPane);
//Set whole as the main content pane of frame
this.setContentPane(whole);
```

4.6.3 JMenuBars, JMenus and JMenuItem

The JMenus were created first, as follows:

```
JMenu fileMenu = new JMenu ("File"); //File menu
fileMenu.setMnemonic ('F');
JMenu editMenu = new JMenu ("Edit"); //Edit menu
editMenu.setMnemonic ('E');
JMenu toolsMenu = new JMenu ("Tools"); //Tools menu
toolsMenu.setMnemonic ('T');
JMenu helpMenu = new JMenu ("Help"); //Help menu
helpMenu.setMnemonic ('H');
```

The JMenuBar was then created as follows:

```
JMenuBar bar = new JMenuBar ();
setJMenuBar (bar);
//Add file, edit, tools and help to JMenuBar
bar.add (fileMenu);
bar.add (editMenu);
bar.add (toolsMenu);
bar.add (helpMenu);
```

The example below shows the JMenuItem Open, with it's default icon, it's mnemonic, it's shortcut key and it's ToolTipText all displayed. It is then added to the fileMenu. This process was repeated for every JMenuItem.

```
JMenuItem openDocument = new JMenuItem ("Open...", new ImageIcon
("open.png"));
openDocument.setMnemonic ('O');
openDocument.setAccelerator(KeyStroke.getKeyStroke("ctrl O"));
openDocument.setToolTipText("Open a previously saved document.");
```

```
fileMenu.add(openDocument);
```

4.6.4 Radio Buttons

The Change Smoothing item needed a menu itself, so that the user could choose the correct type of smoothing. As only one type of smoothing could be selected at once, `JRadioButtonMenuItems` were used. The same method as detailed below was used for the Time Taken menu, where the time taken for both spelling and likelihood checks could be displayed.

```
//CHANGE SMOOTHING
JMenu changeSmoothing = new JMenu ("Change Smoothing");
changeSmoothing.setMnemonic ('O');
toolsMenu.add(changeSmoothing);
//SUBMENU IN CHANGE SMOOTHING
ButtonGroup group = new ButtonGroup();
//Uniform Event Space
JRadioButtonMenuItem uniformEventSpace = new JRadioButtonMenuItem
("Uniform Event Space");
group.add(uniformEventSpace);
changeSmoothing.add(uniformEventSpace);
//Frequentist Event Space
JRadioButtonMenuItem frequentistEventSpace = new JRadioButtonMenuItem
("Frequentist Event Space");
group.add(frequentistEventSpace);
changeSmoothing.add(frequentistEventSpace);
//MLE
JRadioButtonMenuItem maximumLikelihoodEstimation =
new JRadioButtonMenuItem
("Maximum Likelihood Estimation");
group.add(maximumLikelihoodEstimation);
changeSmoothing.add(maximumLikelihoodEstimation);
//PLUS-ONE
JRadioButtonMenuItem plusOne = new JRadioButtonMenuItem
("Plus One");
group.add(plusOne);
changeSmoothing.add(plusOne);
//WITTEN-BELL
JRadioButtonMenuItem wittenBell = new JRadioButtonMenuItem("Witten
Bell");
wittenBell.setSelected(true); //This line selects this as default
group.add(wittenBell);
changeSmoothing.add(wittenBell);
```

4.6.5 Line Spacing

When a line separator was required to help the user view the menus, the following line of text was added in the location between the menu items where it was to be needed:

```
fileMenu.addSeparator();
```

4.6.6 ImageIcons

To aid the user see whether or not the sentence was accepted or not, or if the sentence was likely or not, it was vital to have a clear output from the system in the form of a green tick/red cross to show them this. The following code is adapted from that available at <http://java.sun.com/docs/books/tutorial/uiswing/components/icon.html>. To enable this to happen, the following was done:

```
ImageIcon green = createImageIcon("green.png"); //Tick
ImageIcon red = createImageIcon("red.png"); //Cross
ImageIcon blank = createImageIcon("blank.png"); //Blank
The following was then used to import the images:
protected static ImageIcon createImageIcon(String path){
    java.net.URL imgURL = GUI.class.getResource(path);
    if (imgURL != null){
        return new ImageIcon(imgURL);
    }
    else{
        System.err.println("Image not found");
        return null;
    }
}
```

Blank was then set as the default icon before any checks were performed, and after the system was reset:

```
colourOutput.setIcon(blank);
```

4.7 ActionListeners

4.7.1 Open

The Open ActionListener opens a text file and places its contents in the `inputBox`. It also ensures that only `.txt` files can be opened - and throws a `JOptionPane.ERROR_MESSAGE` if the user tries to open an unsupported file. To allow the user to open a file, a `JFileChooser` is used. This opens up a separate dialog box, in which the user can select the file to open. This part of the code has been adapted from the free code available at:

```
http://forums.sun.com/thread.jspa?threadID=496612 and the JFileFilter
has been adapted from code available from:
http://www.java2s.com/Code/JavaAPI/javaw.swing/
JFileChooseraddChoosableFileFilterFileFilterfilter.htm
openDocument.addActionListener(new ActionListener(){
```

```
public void actionPerformed(ActionEvent e){
    JFileChooser fc = new JFileChooser();
    fc.addChoosableFileFilter(new TextFilter());
    int returnVal = fc.showOpenDialog(inputBox);
    try{
        String strLine;
        File selectedFile = fc.getSelectedFile();
        FileInputStream in = new FileInputStream(selectedFile);
        BufferedReader br = new BufferedReader
            (new InputStreamReader(in));
        while ((strLine = br.readLine()) != null){
            inputBox.setText(strLine);
        }
    }
    catch(Exception ae){
        //No body to catch loop
    }
    while (returnVal == JFileChooser.APPROVE_OPTION &&
        !fc.getSelectedFile().getName().endsWith(".txt")) {
        JOptionPane.showMessageDialog(inputBox, "The file " +
            fc.getSelectedFile() + " is not a supported file format.",
            "Error", JOptionPane.ERROR_MESSAGE);
        inputBox.setText("");
        break;
    }
}
});
```

4.7.2 Save

The Save ActionListener saves the current text in the inputBox to a file. To allow the user to save a file, a `JFileChooser` is used. This opens up a separate dialog box, in which the user can select the location and name of the file to save. This ActionListener also questions the user if they try and overwrite a previously saved file, and mirrors notepad in that it does not force the user to save the file as a .txt file type - it allows them to enter whatever type suits them best. This has been done as it is believed that users will be used to this way of saving files, and so is intuitive to use.

```
saveDocument.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent e){
        JFileChooser fc = new JFileChooser();
        int returnVal = fc.showSaveDialog(inputBox);
        if (returnVal == JFileChooser.APPROVE_OPTION){
            String filename = fc.getSelectedFile().getAbsolutePath();
            File file = new File(filename);
            if (file.exists()){ //Check if user wants to overwrite
```

```
        int r = JOptionPane.showConfirmDialog(inputBox, "Are you
sure that you wish to overwrite file:  "+filename+"?", "Overwrite?",
JOptionPane.YES_NO_OPTION);
        if (r == JOptionPane.YES_OPTION){
            try{
                BufferedWriter bw = new BufferedWriter(new FileWriter
(filename));
                bw.write(inputBox.getText());
                bw.flush();
                bw.close();
            }
            catch (IOException ioe){
                //No body to catch loop
            }
        }
        else{
            fc.showSaveDialog(inputBox);
        }
    }
    else{
        try{
            BufferedWriter bw = new BufferedWriter
(new FileWriter(filename));
            bw.write(inputBox.getText());
            bw.flush();
            bw.close();
        }
        catch (IOException ioe){
            //No body to catch loop
        }
    }
}
});
```

4.7.3 Print Input and Output

Both of these `ActionListeners` prevent the user printing an empty string, by checking if the text entered to the `inputBox` is empty in the `if` branch. If the `inputBox` is not empty, the print dialog occurs and allows the user to select their printing options.

```
printInput.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent e){
        if (inputBox.getText().equals("")){ //Empty string
            JOptionPane.showMessageDialog(inputBox, "Cannot print empty
```

```
input string.", "Error", JOptionPane.ERROR_MESSAGE);
    }
    else{ //Non-empty string
        try{
            inputBox.print();
        }
        catch(Exception ae){
            //No body to catch loop
        }
    }
}
});

printOutput.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent e){
        if(outputBox.getText().equals("")){
            JOptionPane.showMessageDialog(inputBox, "Cannot print empty
output string.", "Error", JOptionPane.ERROR_MESSAGE);
        }
        else{
            try{
                outputBox.print();
            }
            catch(Exception ae){
                //No body to catch loop
            }
        }
    }
});
```

4.7.4 Exit

The exit ActionListener brings up a confirmation message, where the user has to select the Yes option to exit the system, and can select No to take them back to the default frame. This is to prevent users accidentally closing the system and losing their progress.

```
exitItem.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent e){
        int exit = JOptionPane.showConfirmDialog(inputBox, "Are you sure
that you wish to exit?", "Exit?", JOptionPane.YES_NO_OPTION,
JOptionPane.WARNING_MESSAGE);
        if (exit == JOptionPane.YES_OPTION){
            System.exit(0);
        }
        else {
```

```
        //No body to else loop
    }
}
});
```

4.7.5 Undo/Redo

Both of these ActionListeners make use of an UndoManager added to the inputBox:

```
import javax.swing.undo.UndoManager;
UndoManager um;
um = new UndoManager();
inputBox.getDocument().addUndoableEditListener(um);
//UNDO
undoItem.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent e){
        if (um.canUndo()){
            um.undo();
        }
        else {
            //No body to else loop
        }
    }
});
//REDO
redoItem.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent e){
        if (um.canRedo()){
            um.redo();
        }
        else{
            //No body to else loop
        }
    }
});
```

4.7.6 Cut/Copy/Paste/Select All

The cut,copy,paste and select all options all perform their jobs on the inputBox. The ActionListeners are as follows:

```
//CUT
cutItem.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent ev){
        inputBox.cut();
    }
});
```

```
//COPY
copyItem.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent ev){
        inputBox.copy();
    }
});
//PASTE
pasteItem.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent ev){
        inputBox.paste();
    }
});
//SELECT ALL
selectAllItem.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent ev){
        inputBox.selectAll();
    }
});
```

4.7.7 Clear All

The Clear All ActionListener simply has to return everything to exactly the way that it was before any changes were made. Therefore it needs to do the following:

- Clear inputBox
- Make sure inputBox can be edited
- Enable spellCheck button
- Disable likelihoodCheck button
- Clear outputBox
- Remove the colourOutput icon
- Clear wrongWords array
- Clear correctWords array
- Set elapseSpellTime as 0
- Set elapseLikelihoodTime as 0
- Reset leastLikely number
- Reset leastLikely string to the empty string

4.7.8 View Mistakes

The View Mistakes ActionListener first of all checks whether or not the `wrongWords` array is empty or not. If it is empty, there can clearly be no incorrectly spelt words and so an `INFORMATION_MESSAGE` is displayed explaining this. If it is not empty, then the wrong words are displayed in a different `INFORMATION_MESSAGE`.

```
viewMistakes.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent ev){
        if (wrongWords.isEmpty()){
            JOptionPane.showMessageDialog(inputBox,
                "There were no incorrectly spelt words on the last
                performed check, or no check has taken place.",
                "List of Mistakes", JOptionPane.INFORMATION_MESSAGE);
        }
        else {
            JOptionPane.showMessageDialog(inputBox, "The following
                "+wrongWords.size()+" words are spelt incorrectly:
                "+wrongWords, "List of Mistakes",
                JOptionPane.INFORMATION_MESSAGE);
        }
    }
});
```

4.7.9 View Possible Typos

In much the same way as above, if the `leastLikelyString` array is empty there cannot have been a likelihood check run and so an `INFORMATION_MESSAGE` telling the user this is displayed. If the string equals 'UES' or 'MLE' specific messages are returned, else the least likely two word string is returned to the user.

```
viewTypos.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent ev){
        if (leastLikelyString.equals("")){ //No check performed
            JOptionPane.showMessageDialog(inputBox, "No check has yet
                been performed.", "Most Likely Typos", JOptionPane.INFORMATION_MESSAGE);
        }
        else if (leastLikelyString.equals("UES")){
            JOptionPane.showMessageDialog(inputBox, "In UES all strings
                have the same probability. Change smoothing to obtain the most
                likely typos.", "Most Likely Typos", JOptionPane.INFORMATION_MESSAGE);
        }
        else if (leastLikelyString.equals("MLE")){
            JOptionPane.showMessageDialog(inputBox, "Unable to obtain least
                likely probability in MLE when probability is equal to 0.", "Most Likely
                Typos",
                JOptionPane.INFORMATION_MESSAGE);
        }
    }
});
```

```

    }
    else {
        JOptionPane.showMessageDialog(inputBox, "The following two
word string contains the lowest probability of your entered sentence:
"+leastLikelyString+ "ü", "Most Likely Typos",
JOptionPane.INFORMATION_MESSAGE);
    }
}
});

```

4.7.10 Time Taken

Both Time Taken ActionListeners performed exactly the same tasks, but `likelihoodTime` replaced the `elapsedSpellTime` below with `elapsedLikelihoodTime`. If the time equals 0, it is clear no check has taken place, and so an error message is displayed, else the time taken is displayed.

```

spellingTime.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent ev){
        if (elapsedSpellTime == 0.0){
            JOptionPane.showMessageDialog(inputBox, "No spell check has
taken place.", "Time Taken for Spelling Result",
JOptionPane.INFORMATION_MESSAGE);
        }
        else {
            JOptionPane.showMessageDialog(inputBox, "The last spell check
took "+(int)elapsedSpellTime+"milliseconds (ms).", "Time Taken for Spelling
Result", JOptionPane.INFORMATION_MESSAGE);
        }
    }
});

```

4.7.11 Change Smoothing

The Change Smoothing ActionListeners all simply change a variable to the correct number for that type of smoothing, so that when the `likelihoodCheck` button is pressed, the corresponding type of smoothing is performed. The numbers for each smoothing type are as follows and are substituted into the code beneath that. The `leastLikelyString` and `leastLikely` double are also both reset to their original values, to stop the view possible typos ActionListener loops in Section 4.7.9 displaying the result of previous sentences.

- UES: 0
- FES: 1
- MLE: 2
- Plus-One: 3

- Witten-Bell: 4

```
uniformEventSpace.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent ev){
        smoothing = 0;
        leastLikelyString = "";
        leastLikely = 1.0;
    }
});
```

4.7.12 Open PDF Help

This code opens the User Guide PDF document to aid the user with the systems functionalities. This segment of code has been adapted slightly from that available on the 'How To' section of:

<http://www.rgagnon.com/javadetails/java-0014.html>[18]. The small issue with this piece of code is that it means that the PDF can only be opened on Windows machines, as it selects the default FileProtocolHandler from a set location, so that the machine chooses itself what program it uses to open this type of file. This will not work on other platforms however, so user documentation of the same information will be provided with the system.

```
introductionItem.addActionListener(new ActionListener(){
    public void actionPerformed (ActionEvent event){
        try{
            Runtime.getRuntime().exec("rundll32 url.dll,
FileProtocolHandler " + "UserGuide.pdf");
        }
        catch (Exception e){
            //No body to catch loop
        }
    }
});
```

4.8 Interesting Feature

One of the things observed during this version of the system, was that when certain strings were entered for checking, the result obtained would actually cause a negative probability. Table 5 below shows the Witten-Bell count achieved for strings containing of the word 'the' repeated n times. During the optimisation process it was important to ascertain whether or not this was due to overflow, or indeed what the cause of this 'oscillation' between positive and negative values was. This problem was solved in Section 5.1.

Number n	Result
3	2.164666611694525E-10
4	-3.1848319969611068E-15
5	4.6857815397850555E-20
6	-6.8940994876781065E-25
7	1.014315484886727E-29
8	-1.4923426978677127E-34
9	2.1956548638590406E-39
10	-3.230424411280368E-44

Table 5: Negative and Positive Results

5 Optimisation

5.1 Interesting Feature Solved

To begin with, it was believed that the oscillation between negative and positive values for the Witten-Bell check described in Section 4.8 and shown in Table 5 was part of a problem with the `Double` primitive data type. This data type was being used throughout the system for the calculations of probability, and was perhaps causing a problem if the numbers were becoming too small. However, after checking resources about this data type[19], it was clear that as the range from ' $\pm 4.94065645841246544E - 324$ to $1.79769313486231570E + 308$ ' that could be covered by `Double` was not being reached, that the issue was with the 'Zero' branch of the Witten-Bell calculation which had been created. Before Optimisation, this branch looked as follows:

```
else{
    String[] wordsWB = wholeSentence.toLowerCase().split(" ");
    for (int k=length-1; k>0 ; k--){
        String WB0 = wordsWB[k-1];
        String WB1 = wordsWB[k];
        String WB0WB1 = new String(WB0+ " " +WB1);
        try{
            FileInputStream fstream = new FileInputStream("T.txt");
            DataInputStream in = new DataInputStream(fstream);
            BufferedReader brDictionary = new BufferedReader(new
                InputStreamReader(in));
            while((lineWB = brDictionary.readLine()) != null){
                linecount++;
                String[] TValue =(lineWB.split(delimiter));
                if(TValue[0].equals(WB0)){
                    TValueInt = Double.parseDouble(TValue[1]);
                    ZValueInt = V - TValueInt;
                    break;
                }
            }
        }
    }
}
```

```

        }
        else{
            TValueInt = 0.0;
            ZValueInt = V - TValueInt;
        }
    }
}
catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}
try{
    FileInputStream fstream2 = new FileInputStream
    ("SingleCorpus.txt");
    DataInputStream mate = new DataInputStream(fstream2);
    BufferedReader brSingleDictionary =
    new BufferedReader(new InputStreamReader(mate));
    while((line2 = brSingleDictionary.readLine()) != null){
        linecount2++;
        String[] NValue =(line2.split(delimiter));
        if(NValue[0].equals(WB0)){
            NValueInt = Double.parseDouble(NValue[1]);
            break;
        }
        else {
            //No body to else loop
        }
    }
}
catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}
WittenBellCalculator = WittenBellCalculator*(TValueInt /
((ZValueInt*NValueInt)+TValueInt));
break;
}
}

```

To find the issue, two `System.out.print()` methods were inserted as follows and the results recorded:

```

System.out.println(WBOWB1); //To view the string for checking
System.out.println(WittenBellCalculator); //To view the running
Witten-Bell Result

```

- 'The the the'
 - Result: positive

- the the
- -1.4712806026365348E-5
- the the
- 2.164666611694525E-10
- 'The the the the'
 - Result: negative
 - the the
 - -1.4712806026365348E-5
 - the the
 - 2.164666611694525E-10
 - the the
 - -3.1848319969611068E-15

It was then clear from this that the following line of code was causing the problem:

```
ZValueInt = V - TValueInt;
```

With the word 'the', this was resulting in $ZValueInt = 2001 - 69970 = -67969$. After checking the 'T' Corpus it became clear this was the issue as corpus being used had not been a correct corpus. Instead of using the correct T values, the old single corpus was copied across and renamed 'T.txt' and so the results obtained were actually using the wrong corpus. Once the correct corpus was swapped in, the results immediately became correct and no changes to the code were required. The `System.out.print()` methods were then removed to prevent this occurring when the system ran. It was also noted that the V value being used should not be 2001 (the number of words in the reduced corpus) but should be 1000000 (the number of words in the entire Brown corpus). This is because the unigram and bigram results have been calculated as results of 1000000 words being noted, and so this value must be used to ensure this negative issue cannot still occur.

The oscillation between negative and positive results with this were caused due to the mathematical laws of multiplying negative results. To begin with a negative result was obtained, which when multiplied by a second negative result became a positive. This positive was then multiplied by a third negative result, to once again achieve a negative result, and so on. Therefore when n was odd, results were negative ($neg \times neg \times neg = neg$), when n was even, results were positive ($neg \times neg = pos$).

5.2 Threshold Values

Now that the system is functional, and it has been white-box tested to ensure accuracy, it is no longer necessary to calculate all threshold values by hand as it was in Table 3 in Section 3.5. Therefore the twenty-four sentences from Table

3 were used to check both the system against the hand-calculated values for MLE, Plus-One and Witten-Bell smoothing, and to also calculate final Threshold Values to use for all smoothing techniques. The values from this check are listed in Table 6.

Sentence	UES	FES	MLE	Plus-One	Witten-Bell
1	1.00E-24	1.31E-10	5.15E-09	2.59E-15	4.69E-09
2	1.00E-24	1.32E-09	5.45E-08	5.25E-14	4.98E-08
3	1.00E-24	8.51E-10	2.72E-09	1.09E-14	2.45E-09
4	1.00E-12	0.00E+00	1.26E-03	1.30E-05	0.001226
5	1.00E-18	1.98E-08	3.93E-05	4.86E-09	3.71E-05
6	1.00E-18	0.00E+00	1.48E-04	3.51E-10	1.37E-04
7	1.00E-18	1.98E-08	0.00E+00	1.25E-10	2.40E-16
8	1.00E-24	1.48E-10	1.54E-05	3.51E-10	1.37E-04
9	1.00E-30	1.48E-11	2.70E-09	1.29E-13	1.43E-05
10	1.00E-30	0.00E+00	0.00E+00	1.36E-21	1.62E-15
11	1.00E-24	3.68E-12	4.29E-06	5.64E-15	4.04E-06
12	1.00E-18	6.79E-10	0.00E+00	2.10E-11	1.52E-15
13	1.00E-36	5.61E-18	0.00E+00	5.87E-27	5.00E-32
14	1.00E-24	7.66E-11	0.00E+00	2.15E-15	2.89E-24
15	1.00E-30	0.00E+00	0.00E+00	1.00E-24	4.32E-31
16	1.00E-30	0.00E+00	0.00E+00	1.96E-23	2.55E-31
17	1.00E-30	0.00E+00	0.00E+00	1.02E-20	0.00E+00
18	1.00E-30	0.00E+00	0.00E+00	7.48E-24	6.00E-27
19	1.00E-24	6.84E-05	0.00E+00	8.16E-19	6.24E-23
20	1.00E-42	0.00E+00	0.00E+00	4.05E-31	4.33E-46
21	1.00E-36	1.24E-20	0.00E+00	2.00E-29	2.31E-38
22	1.00E-36	0.00E+00	0.00E+00	3.35E-28	9.81E-38
23	1.00E-30	0.00E+00	0.00E+00	5.49E-21	2.47E-30
24	1.00E-30	0.00E+00	0.00E+00	1.27E-22	5.88E-31

Table 6: Sentence Analysis Values Using Tool

5.2.1 UES

The issue with Uniform Event Space is that it is the length of the sentence that defines the probability assigned to it. It is completely unable to decide whether or not the words contained within are likely words, or in a likely order. Therefore it is down to the sentence length where we determine if the sentence is to be likely or not. Therefore I decided that anything over a seven word string would be more unlikely than those sentences less than this, and so chose the value equivalent to this as the threshold value for UES. With the dummy data used in Table 6 this would result in everything being passed as correct,

5.3 Improvements

During the informal testing of the Witten-Bell calculator it was observed that the results were being slightly skewed in the following instances:

1. In general, the longer the sentence the less likely all techniques involving smoothing tends to make it when compared to shorter sentences
2. Also in general, the sentences which execute through the 'zero' branch of the Witten-Bell calculator seem to be inflated when compared to the results obtained from the 'non-zero' branch. By definition this should not be the case as the reason they are going through said branch in the first place is that they are given a zero probability by MLE

Therefore, for use in this system, a new type of smoothing was created to try and reduce the impact of these above anomalies, called Glenn-Smoothing, and that is detailed in Section 5.3.1.

5.3.1 Glenn-Smoothing

The first problem that we wish to solve is that of problem 1 in Section 5.3 above. A workable example is that of the two sentences 'the the the' and 'the the the dog' which are the example which will be expanded in Table 7 below. This

Sentence	Probability
The the the	2.16E-10
The the the dog	2.63E-14

Table 7: Results of Witten-Bell Smoothing

shows that using Witten-Bell smoothing, the first sentence is 8250 times more likely than the second sentence, despite the fact that the second sentence has the phrase 'the dog' in it, which is perfectly permissible. In many ways, this second sentence is 'more correct' than the first, as whereas the first sentence has no correct parts, the second does at least have a two word substring that could be correct. I therefore added the following formula to the achieved Witten-Bell value to create my own type of smoothing and make this gap smaller:

$$P_{Glenn-Smoothing} = P(Witten - Bell) \times (length - 1^{length}) \quad (18)$$

This reduces the gap between the types of smoothing over ten times, so now the gap between the two sentences is just 800 times, which reduces the reliance on the length of sentence making the sentence more likely by over 1100%.

In an attempt to solve the second problem listed in Section 5.3, it was clear that the 'zero' branch was going to have to be edited so that the probability resulting from it was further reduced. To work out the rough amount that this should be done by, I re-performed the Witten-Bell tests from Table 6 by adding

concurrent processing, and is a very efficient way of producing as much output as possible in a short period of time. Both the spell check and each individual likelihood check was given a thread as follows:

```
//In GUI.java
if (smoothing == 1){
    String enteredSentence = inputBox.getText();
    FES fesccheck = new FES(enteredSentence);
    new Thread (fesccheck).start();
}
//In FES.java
public void run() {
    //BEGIN LIKELIHOOD CHECK
    //Functionality
    System.out.println(result);
}
```

The User Interface was then modified to be as simple as possible, with only the Clear All, Change Smoothing and Exit options remaining. The output was modified to print straight to the command line to prevent any time being lost transferring it back to the User Interface. The finished simplified User Interface is shown in Figure 1, with the command line shown in the background.

This system is only 65% the size on the hard disk as the full system, and

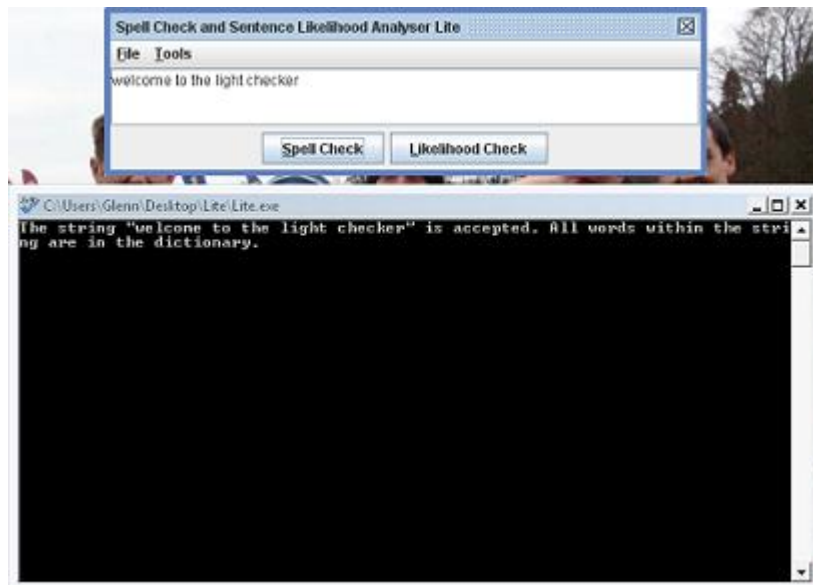


Figure 1: Lite System User Interface

during testing ran between 1% and 10% faster for a given input. However, it does lack a lot of the requirements which the full system holds, in an attempt to keep it as light as possible, and is much less user friendly than the full system.

This however is the trade-off necessary for ultimate efficiency as this system demonstrates.

This system also allows the user to likelihood check words that are not contained within the dictionary, which may be useful if the user wishes to compare two similar sentences but replace a pronoun for a proper noun not contained within the dictionary, for example.

6 Integration

As the products are not going to be mass released, it was decided to produce a file so that it could be downloaded and consequentially tested further by other users. For this to occur the files would have to be downloadable from a website, installed and then loaded as executable files, on the clicking of an icon. This would also ensure that requirement 4g was passed. Sections 6.1, 6.2 and 6.3 detail how this was made possible.

6.1 Making System Load as Executable File

To do this, a freeware program called 'JSmooth' was used. The following options were set for the 'Full' system:

- Skeleton
 - Windowed Wrapper (opens GUI)
- Executable
 - Executable Binary - `Full.exe`
 - Executable Icon - `aboutIcon.gif`
 - Current Directory - `Desktop/Full`
- Application
 - Main Class - `Application`
 - Embedded jar - `ProjectFull.jar`
 - Classpath - `Desktop/Full`

The following options were set for the 'Lite' system:

- Skeleton
 - Console Wrapper (opens GUI and a command line)
- Executable
 - Executable Binary - `Lite.exe`
 - Executable Icon - `LiteIcon.gif`
 - Current Directory - `Desktop/Lite`
- Application
 - Main Class - `Main`
 - Embedded jar - `ProjectLite.jar`
 - Classpath - `Desktop/Lite`

After this steps, the `*.exe` file was compiled and then the icon was run to check that the appropriate operations happened as required. When this was completed, the project build moved on to the next step.

6.2 Distribution

To create a file which could be downloaded and then installed on the end user's machine, a program called 'Nullware Scriptable Install System' was used. The following steps were followed for both systems, but obviously with the word 'Full' substituted for the word 'Lite' where appropriate:

- The whole file 'Desktop/Full' including all the classes, corpora and the executable icons were compressed into a single WinZip file
- In the 'NSIS' window, the 'Installer based on ZIP file' option was selected
- The open button was selected and the file 'Full.zip' was located
- The 'Installer Name' field automatically updated to 'Full'
- In the 'Default Folder' option the dropdown box was used to locate the path '\$DESKTOP'
- Finally, the 'Output EXE file' was named as 'Desktop/FullInstaller.exe' and the generate button was selected
- The 'FullInstaller.exe' file appeared on the desktop

This was then checked on various machines until verified to work as requested.

6.3 Release

Now that the system was bundled up and simple to download, it simply needed to be placed on a website where anybody could access and download the software. Therefore a link to download the system was added to a free site created at the following URL:

<http://glennbarden.webs.com/downloads.htm>[20].

It is now possible to visit this link and download the software and install it onto any computer as required. Contact details were also left on the website so that contact could be made if tester/user finds bugs within the system.

7 Testing

Table 8 below references Table 10 from Section 3.8 and Appendix C. However, for space the 'How to Carry Out' column has been removed, and has been replaced with the 'Actual Result' column which details whether or not the test was successful or not. The tests were all run on the full version of the software, unless explicitly stated.

Screenshots from each applicable test can be viewed in Appendix E, Figures 16 to 45.

Number	Test	Expected Result	Actual Result
1.	Check user can type a sentence for checking	The program allows 'Hello checker' to be entered within the box	The program allowed 'Hello checker' to be entered
2.	Check user can submit sentence for spell checking	The program allows the user to click the button	The program allowed the user to click the button
3.	Check user can submit sentence for likelihood checking	The program allows the user to click the button	The program allowed the user to click the button
4.	Check a second sentence can be entered	The screen should clear, and then allow 'Hello checker' to be entered within the box	The screen cleared and then 'Hello checker' could be entered
5.	Check clear all button works	The screen should clear of all input and output	The screen reverted to its blank default
6.	Check user is unable to likelihood check before the sentence is correctly spelt	The program does not allow the user to click the likelihood check button	It was impossible to select the likelihood check button
7.	Check system prevents user entering another sentence if the current one has been accepted, unless clear all is selected	System should prevent the user typing in new input	It was impossible to select the input box to attempt to enter new input

Continued on Next Page...

Table 8 – Continued

8.	Check the exit function in the File menu	System should exit	A confirmation message was shown, and after confirming, the system exited
9.	Check the copy and paste function works correctly	The word 'Hello' should be inserted at the cursor location	The word 'Hello' was entered at the cursor location
10.	Check that the open function works	The contents of the file should appear in the input box	The contents of the file appeared in the input box
11.	Check that the save function works	The string 'Hello checker' should display in notepad	The string 'Hello checker' appeared in the document 'Hello.txt'
12.	Check the user can print input	'Hello checker' should print on the local printer	'Hello checker' was printed out with correct alignment and properties set
13.	Check the user can print output	The generated system output should print on the local printer	The generated output was printed out with correct alignment and properties set
14.	Check the system's help section works	The user guide should open in a new window	The guide opened in its own window using the system's default viewer
15.	Check undo button works	The pasted in text should be removed	The system reverted to before the paste was performed
16.	Check redo button works	The pasted in text should reappear	The undo operation was reverted
17.	Check timers are zero before input	Both message boxes should state that no input has been received	Both message boxes showed their default 'no input' error messages

Continued on Next Page...

Table 8 – Continued

18.	Check timers are not zero after input and checking has taken place	Both message boxes should state a time for checking	The spell check stated that it took 594ms, and the likelihood checker stated it took 487ms
19.	Check system produces a visual display to show sentence is spelt entirely correctly	System should accept the sentence, explain that it is all spelt correctly and a green tick should be shown	System accepted the sentence, explained it's correctness and the green tick was displayed
20.	Check system produces a visual display to show sentence is not spelt entirely correctly	System should reject the sentence, list the number of errors and display a red cross	System rejected the sentence, listed the number of errors and the first spelling mistake and the red cross was displayed
21.	Check system shows all incorrectly spelt words	All spelling errors should be shown in the pop-up window	The one spelling mistake was displayed
22.	Check system shows the number of incorrectly spelt words	In the output string, it should state there are two spelling mistakes	The system correctly displayed that there were two spelling mistakes in the input string
23.	Check system states whether a string is likely or not	The system should display a green tick or a red cross explaining if the sentence is likely or not	In this instance, the sentence was likely and the system explained this and displayed the green tick
24.	Check system produces a numerical probability of sentence likelihood	The system should display a probability for the string	The Witten-Bell probability 0.5 was displayed for this string in the output panel
25.	Check system presents the least likely substring to the user	The substring 'big doge' should display in the pop-up window	The substring 'big doge' displayed in the pop-up window

Continued on Next Page...

Table 8 – Continued

26.	Check the system is easy to back up	The system should be copied onto the memory stick within a sensible period of time	A backup took less than ten seconds which is deemed more than acceptable
27.	Check that the system is easy to use	The user should be able to spell check and then analyse a sentence's likelihood	The system was shown to a 12 year old brother of reasonable IT competence, and he was able to use the system with no prompt from the tester
28.	Check the text box accepts text quickly	The sentence should appear almost instantaneously to when it is typed	There is no visible delay of input for upto a thousand lines of input
29.	Check the Spell Check button starts the spell check procedure quickly	The program should begin to run almost instantaneously to when it is pressed	There is no noticeable delay between the button being pressed and the starting of any processing of the information
30.	Check the system does not have excessive delays between input and output	The program should run, and produce output almost instantaneously	For sensible length strings of under one hundred words the delay between input and output is very minimal. However over this length, delays do begin to occur. However, as the system is described as a sentence analyser, this is not perceived as a problem

Continued on Next Page...

Table 8 – Continued

31.	Check the system is programmed in a way to easily allow extensions to the code	The programmer should be able to easily add a new type of smoothing to the system	By writing a new class and editing three variables in the main program, a new type of smoothing can be added
32.	Check the exit shortcut key works	System should exit	System displayed a confirmation box, and then exited once the action was confirmed
33.	Check the open shortcut key works	The open window should open	The open window opened
34.	Check the save shortcut key works	The save window should open	The save window opened
35.	Check the print shortcut key works	The print window should open	The print window opened and when print was confirmed, the input to the system printed, as desired
36.	Check the undo shortcut key works	The last change should be undone	The last change was undone
37.	Check the redo shortcut key works	The last undo should be undone	The system undid the last undo operation
38.	Check the cut shortcut key works	The selected text should be removed and added into temporary memory	The selected text was removed, and when paste was selected in a separate application it was clear that the text had been added to temporary memory
39.	Check the copy shortcut key works	The selected text should be added to temporary memory	When paste was selected in a separate application it was clear that the text had been copied to temporary memory

Continued on Next Page...

Table 8 – Continued

40.	Check the paste shortcut key works	The text stored in temporary memory should be inserted into the selected part of the document	The text from the last copy operation was inserted into the document
41.	Check the select all shortcut key works	All the text in the input box should now be highlighted	All text was highlighted
42.	Check the view all mistakes shortcut key works	All mistakes from the last spell check operation should be visible	The View Mistakes window opened showing that there were no spelling mistakes on the last run of the system
43.	Check the view possible typos shortcut key works	The most likely typo/grammatical mistake from the last likelihood check should be visible	The View Possible Typos window opened, and stated that no likelihood check had yet been performed
44.	Check the view user guide introduction shortcut key works	The user guide introduction should display	The user guide opened in its own window using the default viewer application as expected

Table 8: Testing Results

It was also mentioned at the end of Section 3.8 that the system would be checked for accuracy by checking the results it produced against the hand calculated results in Table 3. After this extensive testing no erroneous results or results that differed even slightly from those expected were found. Therefore the evaluation of the system could commence.

8 Evaluation

All results within the evaluation were achieved using the full version of the system, unless explicitly stated to the contrary.

8.1 System Accuracy

8.1.1 Spell Check

The accuracy of the spell check part of the system solely depends on the accuracy of the dictionary being used. Therefore the dictionary used is the key part of the system. All the spell check has to do is prevent incorrect spellings passing as correct, and this is exactly what it has achieved. Any spelling mistake will prevent the sentence passing as correct. Every word that passes correct is contained within the dictionary and hence, the spell check part of the system is correct.

8.1.2 Likelihood Check

Systems such as the likelihood check part of the application are often evaluated by comparing them with a human labelled Gold Standard test set, based on percent correct: the percentage of all results from the system which agree with that of the human measured Gold Standard[13].

In Section 1.1 a poem written by Dr. Jerrold H. Zar in 1991, assisted by Mark Eckman, with an original length of 225 words, and containing 123 incorrectly used words was discussed[5]. This poem is often used to describe the inaccuracy of spell checkers and their passing of incorrect sentences as correct. In a normal spell checker, this poem would pass as 100% correct, whereas actually just the opening two lines which are correct (6%). Therefore it was decided to use this poem to test the accuracy of the system as part of a Gold Standard, and combine this result with the 24 sentences used throughout this project, and evaluated as part of testing in Section 7. A further forty sentences with typographic errors will then be created to bring the Gold Standard up to 100 sentences in total, to give an accurate representation. The results obtained from the poem by Jerrold Zar, are described in Table 9. Sentences which were evaluated as likely are given a tick (✓) and unlikely are given a cross (✗). Sentences 1 and 2 should pass as likely, with the remainder being unlikely.

Line	UES	FES	MLE	Plus-One	Witten-Bell	Glenn
1	✓	✗	✗	✗	✓	✓
2	✓	✗	✗	✗	✗	✗
3	✓	✗	✗	✗	✗	✗
4	✓	✗	✗	✗	✗	✗
5	✓	✗	✗	✗	✗	✗

Continued on Next Page...

Table 9 – Continued

6	✓	✗	✗	✗	✗	✗
7	✓	✗	✗	✗	✗	✗
8	✓	✗	✗	✗	✗	✗
9	✓	✗	✗	✗	✗	✗
10	✓	✗	✗	✗	✗	✗
11	✗	✗	✗	✗	✗	✗
12	✓	✗	✗	✗	✗	✗
13	✗	✗	✗	✗	✗	✗
14	✓	✗	✗	✗	✗	✗
15	✓	✗	✗	✗	✗	✗
16	✓	✗	✗	✗	✗	✗
17	✓	✗	✗	✗	✗	✗
18	✓	✗	✗	✗	✗	✗
19	✗	✗	✗	✗	✗	✗
20	✓	✗	✗	✗	✗	✗
21	✓	✗	✗	✗	✗	✗
22	✓	✗	✗	✗	✗	✗
23	✗	✗	✗	✗	✗	✗
24	✓	✗	✗	✗	✗	✗
25	✓	✗	✗	✗	✗	✗
26	✓	✗	✗	✗	✗	✗
27	✓	✗	✗	✗	✗	✗
28	✓	✗	✗	✗	✗	✗
29	✗	✗	✗	✗	✗	✗
30	✓	✗	✗	✗	✗	✗
31	✗	✗	✗	✗	✗	✗
32	✓	✗	✗	✗	✗	✗
33	✗	✗	✗	✗	✗	✗
34	✓	✗	✗	✗	✗	✗
35	✓	✗	✗	✗	✗	✗
36	✓	✗	✗	✗	✗	✗
Correct	11	34	34	35	35	35

Table 9: Evaluation Results

The 40 further sentences to bring the Gold standard upto 100 sentences were as follows, with the results shown afterwards in brackets. These sentences were split 20-20 with the first twenty representing correct sentences, and the final twenty sentences the same as the previous twenty but with a single typographical error in each.

For example sentence(✓✓XXXX) represents that the sentence was evaluated as likely in UES and FES, but unlikely in the other smoothing types.

1. I want to drive (✓✓✓✓✓✓)
2. See you on the other side (✓XXXX✓✓)
3. No way! (✓✓✓✓✓✓)
4. The car is not necessarily the fastest (✓XXXX✓)
5. The president of the United States (✓X✓X✓✓)
6. Turn the light on (✓✓✓X✓✓)
7. Now is the time to fight (✓XXXX✓✓)
8. The president is able to talk to the public (XXXX✓✓)
9. The light of the Lord (✓✓XX✓✓)
10. Is there a room for four children? (✓XXXX✓✓)
11. Can you see me now? (✓X✓X✓✓)
12. Do you know? (✓✓✓✓✓✓)
13. You will have to wait (✓X✓X✓✓)
14. You have to check the bag (✓XXXX✓✓)
15. I am not sure (✓✓✓✓✓✓)
16. I have told them before (✓X✓X✓✓)
17. The source of the river (✓✓✓X✓✓)
18. You have always been there for me (✓XXXX✓✓)
19. I am sure of the facts (✓X✓X✓✓)
20. Leave it with me (✓✓✓✓✓✓)
21. I want too drive (✓XXXXX)
22. See you of the other side (✓XXXX✓✓)
23. On way! (✓✓X✓✓✓)
24. The cat is not necessarily the fastest (✓XXXX✓)
25. The president on the United States (✓XXXXX)
26. Turn the light no (✓✓XXXX)
27. Not is the time to fight (✓XXXX✓✓)
28. The president is able to talk to the pubic (XXXXXX)

29. The lite of the Lord (✓XXX✓✓)
30. Is their a room for four children? (✓XXXXX)
31. Can you see me know? (✓XXXXX)
32. Do you no? (✓✓X✓✓✓)
33. You will have to weight (✓XXXXX)
34. You have to cheque the bag (✓XXXXX)
35. I am knot sure (✓XXXXX)
36. I have tolled them before (✓XXXXX)
37. The sauce of the river (✓XXX✓✓)
38. Ewe have always been there for me (✓XXX✓✓)
39. I am shore of the facts (✓XXXXX)
40. Leave if with me (✓✓XXXX)

Now we can make the following conclusions about each type of smoothing within the system, by comparing the twenty-four sentences displayed in Table 6, the thirty-six sentences from the poem evaluated in Table 9 and the forty further sentences evaluated above:

- UES: $\frac{12}{24} + \frac{11}{36} + \frac{30}{60} = 53\%$
- FES: $\frac{19}{24} + \frac{34}{36} + \frac{24}{60} = 77\%$
- MLE: $\frac{21}{24} + \frac{34}{36} + \frac{32}{60} = 87\%$
- Plus-One: $\frac{21}{24} + \frac{35}{36} + \frac{23}{60} = 79\%$
- Witten-Bell: $\frac{24}{24} + \frac{35}{36} + \frac{32}{60} = 91\%$
- Glenn-Smoothing: $\frac{24}{24} + \frac{35}{36} + \frac{32}{60} = 91\%$

It is clear from these results that Witten-Bell and Glenn-Smoothing are the most accurate within this system. However of the two, when they both gave incorrect results the latter was closest to giving the correct answer, and so it is safe to assume that the smoothing created explicitly for this system is in fact the most accurate in most cases. It is also interesting to note, that at no point for the 100 sentences trialled did all six smoothing types all give the wrong result (0%) whereas the same correct result was obtained by all six types on 21% of all sentences, a number rising to an impressive 64% when UES is removed from this comparison. Also notable here is the impressive performance of MLE when compared to Plus-One smoothing. MLE was very strong in finding the typographical errors in the final part of the evaluation, and scored 100% in the second set of twenty sentences, so did not allow a single incorrect sentence to pass as correct which ranked it as the highest of all the formulae in this set. These results were also verified with those generated by the 'Lite' system, and no discrepancies between the sets of data were noticed.

8.2 Spell Check Time Taken

Figure 2 shows the length of time taken for the system to spell check the 24 sentences listed in Section 3.4.2 when they are listed by length of sentence:

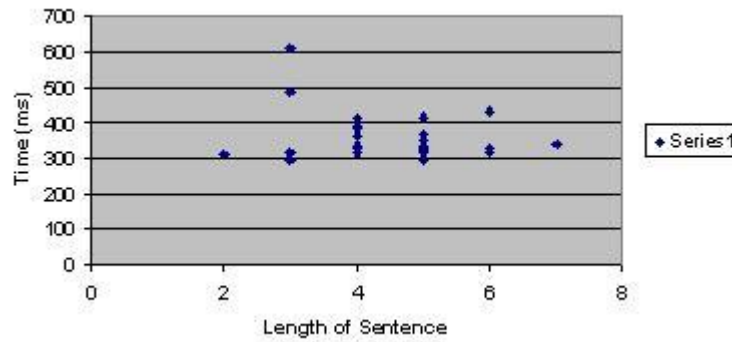


Figure 2: Correct Sentences Time To Spell Check

Figure 3 shows the length of time for the system to spell check 24 sentences from length 1 upto length 24 with all words within the sentence being incorrect:

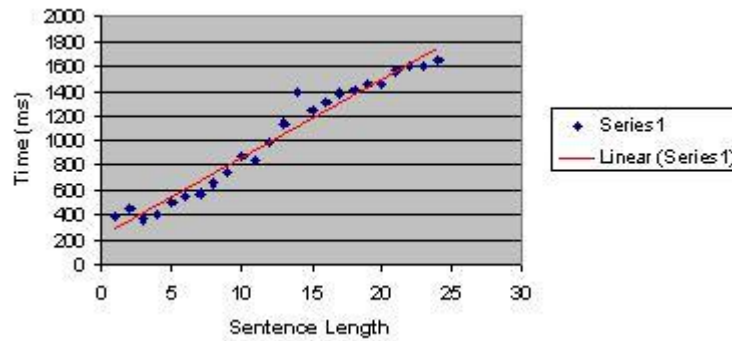


Figure 3: Incorrect Sentences Time To Spell Check

8.2.1 Conclusion

It is clear here that the length of the sentence does not determine the time taken for correct sentences, but for incorrect sentences the number of incorrect words within the sentence does determine the time taken for spell checking. This is

what we would expect, as the length of time taken to find and retrieve a correct word depends on where that word is stored within the dictionary. As this is not an issue with incorrect words (as for each incorrect word a full check of the dictionary is carried out taking n time per word) we can see that the spell check has a linear $O(n)$ running time, shown by our linear trendline on Figure 3.

8.3 Likelihood Check Time Taken

As in Section 8.2, all the likelihood check graphs below are shown as graphs with time taken for the check against sentence length.

8.3.1 UES

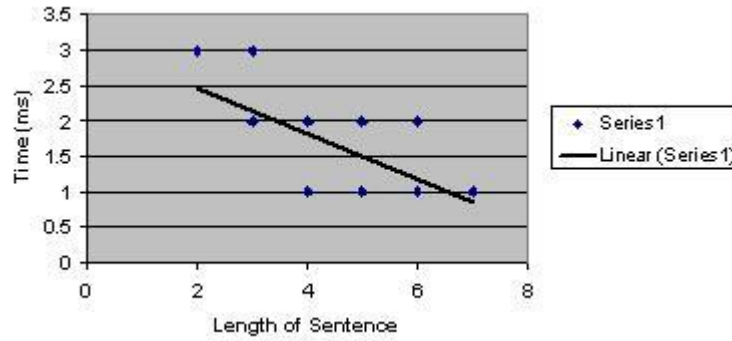


Figure 4: Time Taken for UES Likelihood Check

8.3.2 FES

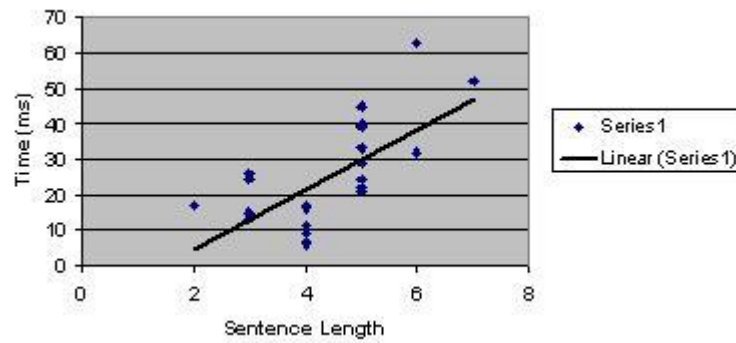


Figure 5: Time Taken for FES Likelihood Check

8.3.3 MLE

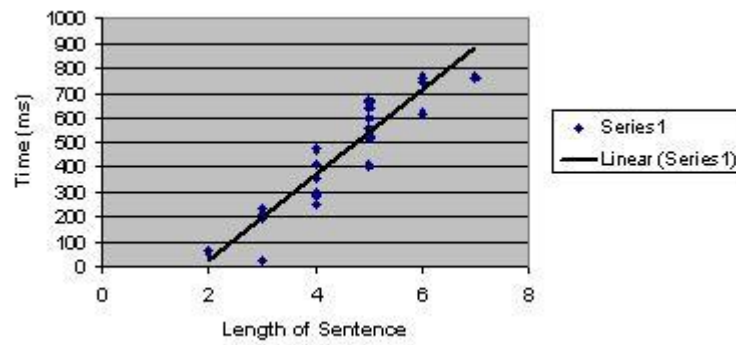


Figure 6: Time Taken for MLE Likelihood Check

8.3.4 Plus-One

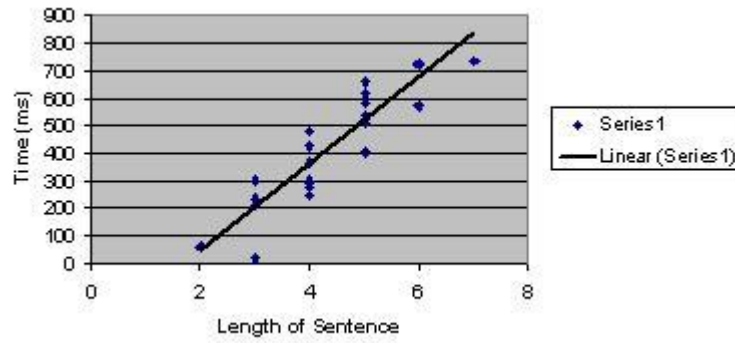


Figure 7: Time Taken for Plus-One Likelihood Check

8.3.5 Witten-Bell

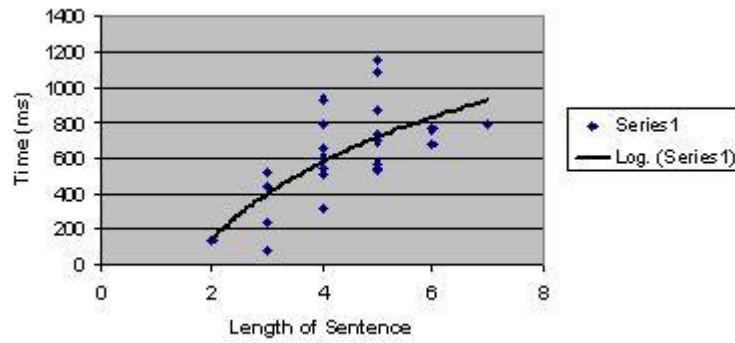


Figure 8: Time Taken for Witten-Bell Likelihood Check

8.3.6 Glenn-Smoothing

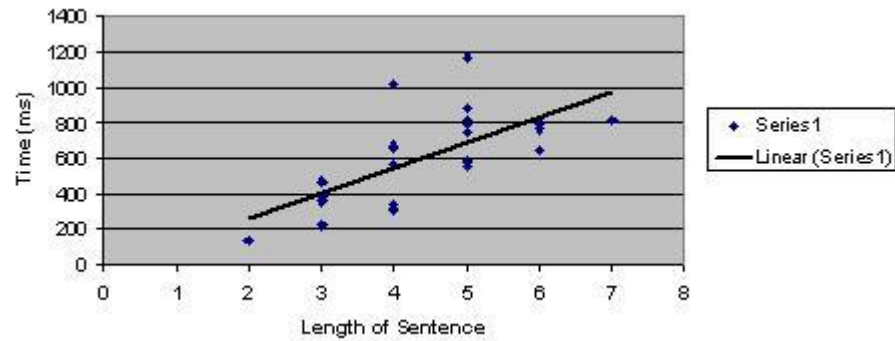


Figure 9: Time Taken for Glenn Smoothing Likelihood Check

8.3.7 Comparison

If we merge the graphs in Figures 4-9, we achieve Figure 10 below:

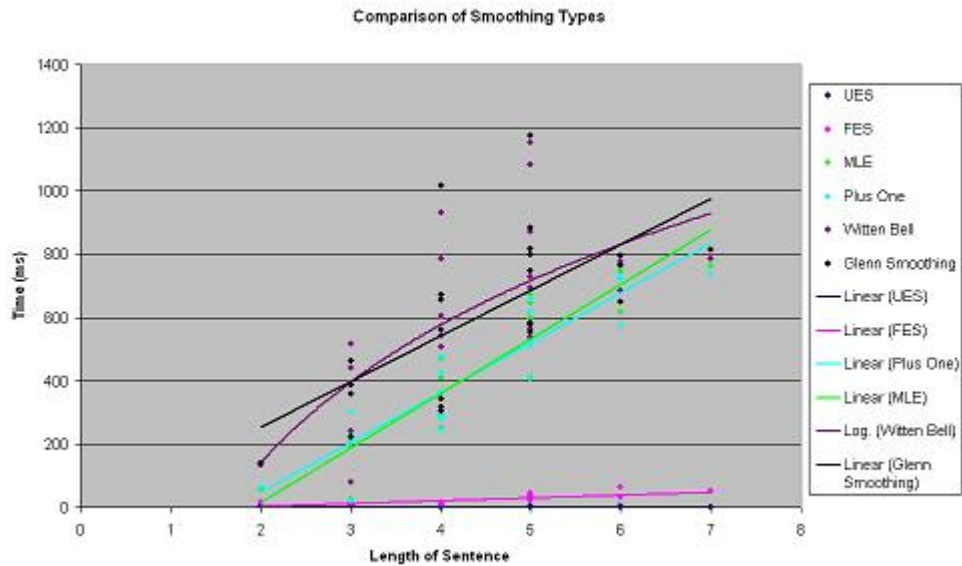


Figure 10: Comparison of the Time Taken by Smoothing Type

From this figure, the following conclusions can be made about the system:

- UES is always the quickest type of smoothing
- FES is always the second quickest type of smoothing
- Witten-Bell and Glenn-Smoothing are always the two slowest types of smoothing
- There appears to be a 'cross-over' point where after a four-word string Plus-One becomes more efficient than MLE

This is pretty much as expected when choosing the types of smoothing for the system which was stated in Section 3.2 - that the more accurate the type of smoothing, the less time efficient it would be.

However, it was a surprise to see that in 54% of the sentences evaluated, Plus-One smoothing was more efficient than MLE. At first these results were questioned, as the nature of the Plus-One smoothing in the system is to perform MLE and then edit the result, and so it was not believed that it could possibly be more efficient, but further tests revealed that on longer strings, this was indeed true. It is not clear why this is the case.

What was clear though, was that when Plus-One was the most efficient of the two, MLE was normally only a very small amount slower, whereas when this was the other way around the gap between the smoothing types was much larger. Therefore it is a safe conclusion that for shorter strings MLE is certainly the quickest whereas as the strings grow in length both smoothing methods have very similar time complexities, but with Plus-One being slightly favourable.

What also appears to happen between Witten-Bell and Glenn-Smoothing is that Glenn-Smoothing actually becomes more efficient for medium length strings between 3 and 6 words long, but Witten-Bell is more efficient for the longer and shorter strings. This mirrors what one would expect, as with longer strings the amount of extra data manipulation that Glenn-Smoothing has to perform will make it less efficient, but hopefully more accurate, than Witten-Bell smoothing.

8.4 Requirements Revisited

In Section 3.1.1 a list of requirements were produced before the system was created to ensure that the system could perform all of the tasks needed. The following list links back to Section 3.1.1 and states whether or not each requirement was met - and to what extent.

1. Basic Program Functions
 - a. Requirement fully met.
 - b. Requirement fully met.
 - c. Requirement fully met.
 - d. Requirement met, but is a menu option rather than a button.
 - e. Requirement fully met.

- f. Requirement fully met.
 - g. Requirement fully met.
 - h. Requirement fully met.
 - i. Requirement fully met.
 - j. Requirement fully met.
 - k. Requirement fully met.
 - l. Requirement met, but only text output is printed and not the graphics associated.
 - m. Requirement fully met.
 - n. Requirement fully met.
 - o. Requirement fully met.
2. Spell Check Functionality
- a. Requirement fully met.
 - b. Requirement fully met.
 - c. Requirement fully met.
3. Likelihood Check Functionality
- a. Requirement fully met.
 - b. Requirement fully met.
 - c. Requirement met, but the option is flawed in Witten-Bell and Glenn smoothing. Therefore this requirement can only be said to be 50% met.
 - d. Requirement met, although the drop down list is inside another menu.
4. User Requirements
- a. Requirement fully met.
 - b. Requirement fully met.
 - c. Requirement partially met. To try and fully meet this requirement for advanced users and those wanting to check large numbers of sentences in a short period of time, the 'lite' system was created.
 - d. Requirement fully met, delays are mostly less than a second for normal likelihood checks.
 - e. Requirement fully met, the system is less than a megabyte in size.
 - f. Requirement partially met. To save space the full system is programmed in very few classes and commented to aid readability for those wishing to extend the code. To permit large amounts of flexibility for those wishing to extend the system the 'lite' system was created and is incredibly simple to add new smoothing types. The full system is not intended to be extended by non-expert users, whereas the lite system is designed intentionally to be as flexible as possible.
 - g. Requirement fully met.

8.5 System Limitations

The limitations of this system are two-fold. The first limitation of the system is caused by the corpora used. The accuracy of the corpora directly affects the accuracy of the output. If the corpora are not detailed enough, the sentences will all be given low percentages and it will be hard to differentiate between more and less likely sentences. Conversely, if the corpora are too detailed then the probability will take too long to pass requirements 4c and 4d, and may actually increase the probability of unlikely sentences making them more difficult to analyse correctly. In this instance, it is felt that the corpora used are not quite detailed enough, and it would be preferable if they were 10 – 25% more detailed as this would not affect running time to a great extent but would increase the system accuracy to an extent to make this change worthwhile. However, as these corpora come from outside sources, in this case the Brown corpus study, it is only possible to edit the releases that they make publicly available. From experience, finding these corpora is very tricky and it is also a challenge to find a unigram and a bigram corpus that can interact together to give acceptable results. If the system were to be publicly available as a spelling aid, it would be possible to purchase the whole corpora from Brown, and then the size could be perfected for the system. However, this was outside of the scope of this project and so as such was not done, with the system relying on edited corpora from publicly available sources.

The final limitation is that the typo finder menu option in the system is currently incomplete for Witten-Bell and Glenn-Smoothing. During the optimisation process some code was changed for efficiency purposes and it was not noticed that the results being obtained with the typo checker were incorrect until very late in the final stages of the build. When the original working code was restored from the backup versions it was determined that the changes made on the 7th of March were responsible. However, the changes made to the system between this date and the discovery of the error were too great, and it was decided that the changes made to the system were more important than the additional functionality given by the typo finder, and so the typo finder was not restored. As the project was being built using the BCS Code of Conduct[25], expanded on in Appendix D, it was important that this issue was not dealt with by cutting back in other areas within the project. As such, it was impossible at such a late stage to finish the project without cutting back in other areas, and so therefore this was not fixed. It is possible however that with extra programming time that this issue could be solved and the typo finder option restored correctly. This does not affect the accuracy of the typo checker for FES or Plus-One smoothing, and UES and MLE were not part of this check in the first place.

8.6 System Strengths

The system has met all of the initial requirements that were placed upon it. It is very strong at pointing out differences between strings of the same length, and in most cases even small typographical errors will be displayed as a mistake.

The system has proved the concept that using these statistical techniques is a very good way of detecting errors within written documents and that the expansion of this system and its corpora could result in a more accurate result than that achieved by Andrew Golding and Dan Roth's Winnow-based spelling correction algorithm. If the system were expanded or inserted as an add-in to a more extensive Word Processing package, it is possible that this could provide a real time sentence analysis, where as soon as a full stop and a space is typed the sentence could be flagged as unlikely if it contains a mistake.

This project is, as far as research has found, the first interactive program capable of providing MLE, Plus-One and Witten-Bell smoothing results to the end user which is available free of use, or indeed publicly available. Previously, such techniques had to be calculated by hand. As such, this solution is fundamentally improved on the previous solution when it came to sentence likelihood analysis calculations.

Another strength of the system is the ease at which corpora and dictionaries can be added or edited. This gives complete flexibility to the end user to use the set which best suits their personal needs. It also means that if a person speaking another language from English wished to translate this system into their native tongue that actually all they would have to do would be to change the corpora and dictionary, and make a few minor alterations to change the text on the User Interfaces. This flexibility is a very strong feature of the system.

Finally, the creation of the lite version of the system has given the created product further flexibility, to a degree even more so than required in the initial system requirements in Section 3.1.1. This additional system has meant that there are two options for the end user, depending on what they require the system for. Advanced users who need to check multiple sentences likelihoods as quickly as possible will be able to use the lite system to reduce the time it will take to check the sentences, whereas those users less au fait with computers are more likely to use the full version with increased usability and a greater range of options.

9 Conclusions

It is clear that despite much research and investigation into the field of Natural Language Processing and statistical processing in recent years, that there is still no method to analyse sentences that can currently be 100% effective. However, through the use of various smoothing methods and with a thorough corpus it is certainly possible to achieve highly accurate results even if they cannot be relied on with absolute faith.

The accuracy of the corpora used within the system are a major limiting factor into the current accuracy ratings, but with an improved corpus it is very possible that the accuracy of this system would be greater than that of any other currently publicly available within this field.

Through use of a Buffered Reader, it was possible to create a linear running time of the program for many types of smoothing. However, it may be easier if the system were to be expanded further to store all of the text files in a single database. This may improve the running time for larger string searches as it would reduce the number of different files searched to produce the result.

To see how successful the system has been, it is important to look back at the original main problems the system was designed to solve, listed in Section 1.2 and repeated below:

1. To stop spelling errors in a sentence passing as correct
2. To prevent typographical errors in a sentence passing as correct
3. To prevent a sentence where incorrect words have been substituted for correct words passing as correct

It is clear revisiting that list that actually the system has been very successful in meeting it's original objectives. It is almost faultless in the meeting of the first objective. The second and third objectives are more qualitative in evaluating their completeness with this system. However, after the evaluation performed of the system accuracy in Section 8, it is clear that to a large degree these objectives have been met. Although the system cannot be said to fully solve the objectives, it can be said to fulfil them to a large enough degree to make the system a worthwhile tool.

In conclusion, this project has proved the concept that producing a context-sensitive spell checker using statistical phrase evaluation and smoothing techniques is indeed accurate and quick enough to provide a viable alternative to the current leading market solutions. As mentioned in Section 2.1, this system has only been designed to check single sentences and this would clearly have to be modified to enable this system to be a full alternative. However, the concept explored has been proven as accurate and so therefore makes this modification viable.

9.1 Future Work

There are two different ways this project could, in my opinion, be furthered in such a way to make it a marketable product:

- Improve the current system to expand it into a word-processing package with the added functionality of the likelihood checking engine
- License the current system in such a way that other software manufacturers could incorporate the likelihood checking engine in their own word-processing packages

To enable the first option to take place, the following would have to be done to the current system:

- Formatting options would have to be added to allow the user to change text size, font, kerning, layout, paragraphing etc
- More obvious alerts to spelling and likelihood would have to be made, and this would have to be done automatically as the user was typing instead of after a button press
- The ability to insert graphics, graphs and shapes should be added to the system
- Find/replace and go to options would be required
- WYSIWYG layout would require more obvious pages that is currently the case, so the user knows where the page will break on the printed document
- The spell check function should produce a list of viable alternatives
- Other miscellaneous word-processing default options would be required to aid usability
- System would have to be able to check more than just a single sentence

To enable the second option to take place, the following would have to be done to the current system:

- The user interface would have to be removed
- The system would have to be translated into the language used by the software company
- A single type of smoothing would have to be chosen
- The system would have to be integrated into the existing product in such a way to alert the user to mistakes without distracting from the word-processor nature of the package
- System would have to be able to check more than just a single sentence

Although the first option would potentially be more financially lucrative if the word-processor created were to become a market leading product, the major barrier to market entry with this sort of product would be that there are already numerous word-processing packages available which computer users are already more used to. It would also require a great deal of further development to give the system the same functionalities as available in current packages. Although the added functionality of the likelihood checker would be a major advance on current packages, no user would use a package that is inferior in other ways to their current choice. The companies producing these products have a greatly superior development budget to myself, as well as a much larger software development team, and it is unlikely that any system created would be able to take over the market in such a way to make the time required for this development to the system to be worthwhile.

The second option however is undoubtedly the preferred route for this system. If one of the larger software developers were to incorporate the expanded system explained above into their current word processing packages, which could be done very easily, the royalties for every package that they sold is potentially quite rewarding. For example, the market leading word-processing package (which has it's own 'typo checker' which is less accurate than this system) sold 70 million copies of its 2007 version in the eight months from its release[21]. If this could be achieved, not only would it reduce the number of errors in typed documents, it could save every business and potentially every person in the world money on printing and re-printing documents when it is found out they have typographical mistakes, as well as stopping companies paying for the time taken for staff to correct these mistakes. The reduced printing also has a knock on affect of potential environmental benefits. In a 2009 article[24] Sir Tim Berners-Lee, creator of the World Wide Web, apologised for the addition of the two slashes in web addresses wasting time, printing and paper. This system could potentially save even more time, printing and paper than the removal of the two slashes in web addresses.

In conclusion, the current likelihood check part of the system could easily be incorporated with minimal changes into any word processing package, which could lead to savings of money, time and lead to environmental benefits as well on top of the system reducing the potentially confusing and misleading documents that have been produced in the past.

A Specification Diagrams

A.1 Use Case Diagram

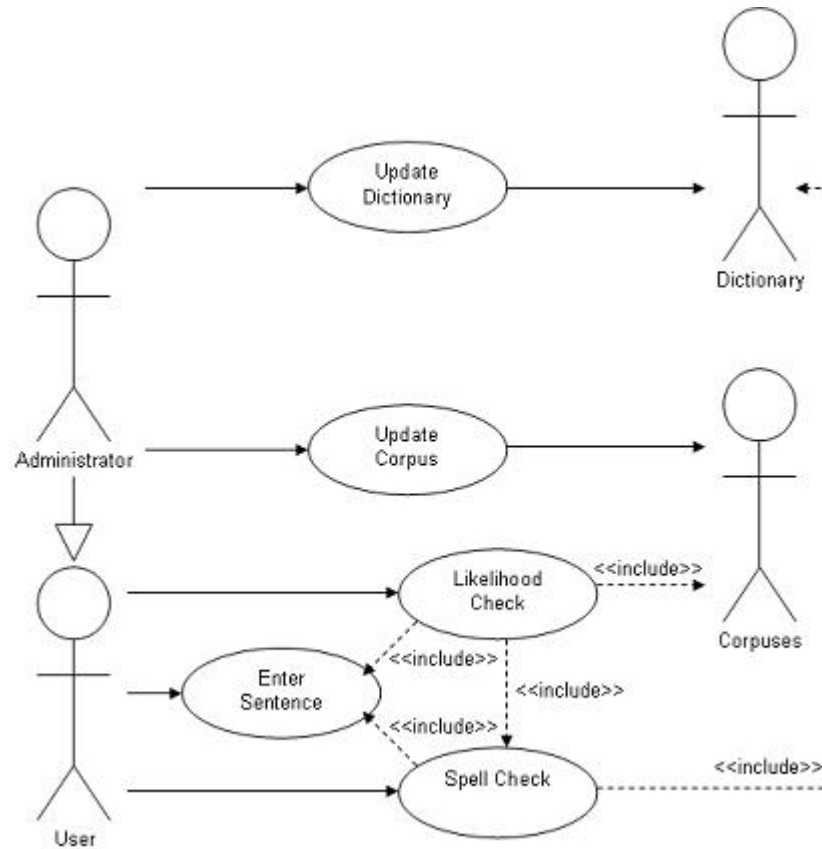


Figure 11: Initial System Use Case Design

A.2 State Chart

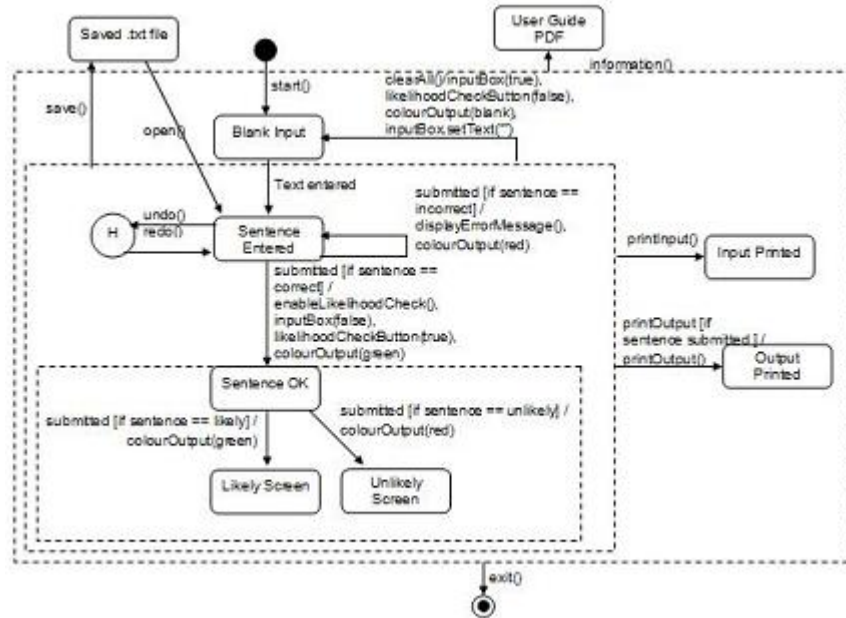


Figure 12: Initial System State Chart Design

A.3 Class Diagram

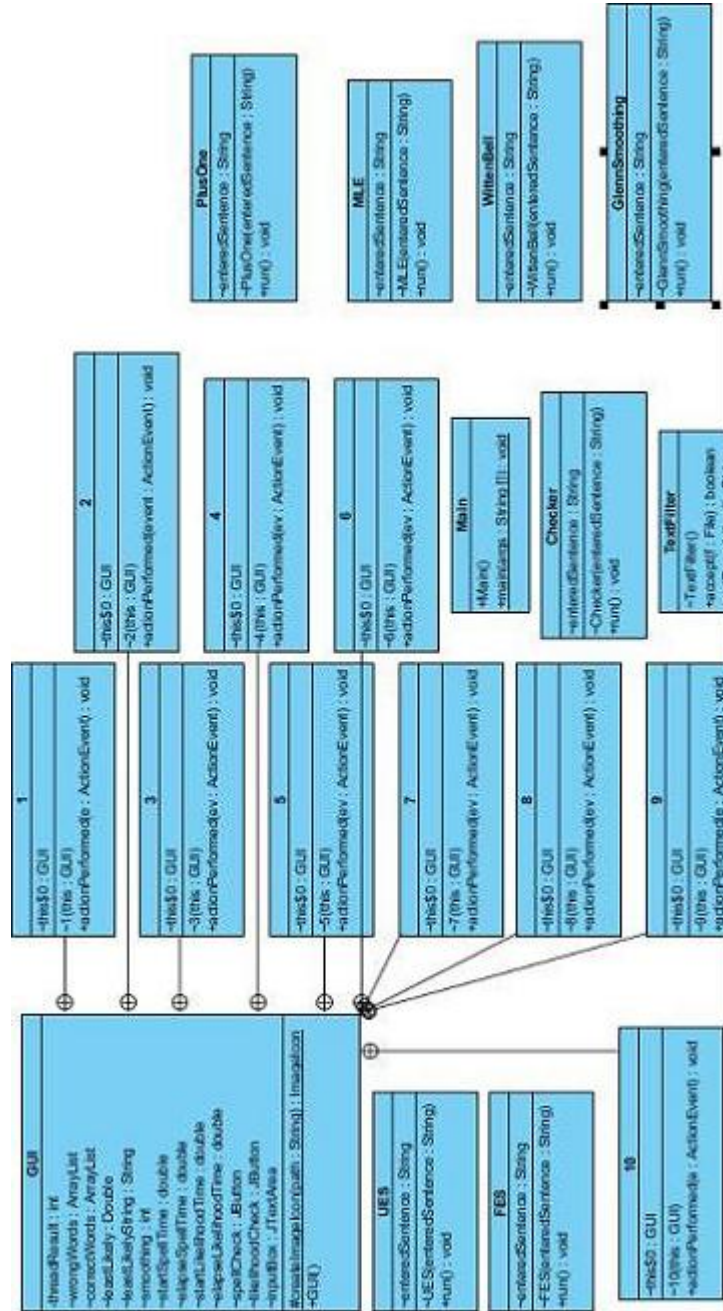


Figure 13: Initial System Class Diagram Design

B User Interface Designs

B.1 Main Screen

The design for the main user interface is shown in Figure 14 below:

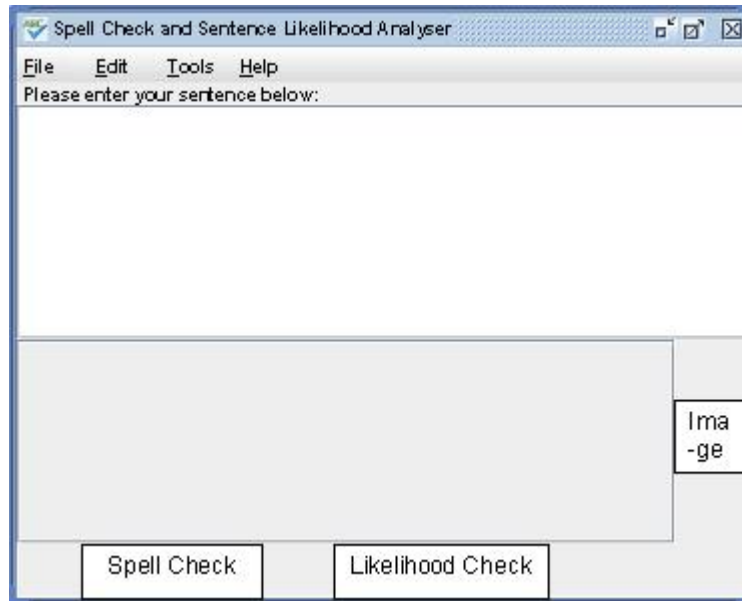


Figure 14: User Interface Design

B.2 Menus

The design for the system menus are shown in Figure 15 below:

<u>F</u> ile	<u>E</u> dit	<u>T</u> ools
 <u>O</u> pen...	 <u>U</u> ndo	 <u>C</u> lear All
 <u>S</u> ave...	 <u>R</u> edo	 View All Spelling Mistakes
 Print Input...	 <u>C</u> ut	 View Possible Typos
 Print Output...	 <u>C</u> opy	<u>T</u> ime Taken ▶
<u>E</u> xit	 <u>P</u> aste	Change Smoothing ▶
	<u>S</u> elect All	
<u>H</u> elp		
 <u>I</u> ntroduction		
<u>A</u> bout		

Figure 15: Menu Design

C Test Plan

Table 10 shows the tests that are to be carried out once the system has been built. Further tests are mentioned in Section 3.8.

Number	Test	How to Carry Out	Expected Result
1.	Check user can type a sentence for checking	Select the input box and try to enter the sentence 'Hello checker' using the keyboard	The program allows 'Hello checker' to be entered within the box
2.	Check user can submit sentence for spell checking	Follow test 1, and once the sentence has been entered, attempt to click on the Spell Check button	The program allows the user to click the button
3.	Check user can submit sentence for likelihood checking	Follow test 2 as above, and ensure the sentence is spelt correctly. Once the sentence is accepted, try to select the now available Likelihood Check button	The program allows the user to click the button
4.	Check a second sentence can be entered	Follow test 2, and once the sentence has been accepted or rejected, select the Tools menu, and then the Clear All option. Repeat test 1.	The screen should clear, and then allow 'Hello checker' to be entered within the box
5.	Check clear all button works	Follow test 1, and then select Clear All from the Tools menu.	The screen should clear of all input and output
6.	Check user is unable to likelihood check before the sentence is correctly spelt	Follow test 1 and then try to click on the likelihood check button	The program does not allow the user to click the likelihood check button

Continued on Next Page...

Table 10 – Continued

7.	Check system prevents user entering another sentence if the current one has been accepted, unless clear all is selected	Follow test 2 and then try to enter another sentence as in test 1	System should prevent the user typing in new input
8.	Check the exit function in the File menu	Click on the File menu and then select the Exit option	System should exit
9.	Check the copy and paste function works correctly	Follow test 1, and then using the mouse, select the word 'Hello'. Select the Copy option from the Edit menu. Then select Paste from the Edit menu.	The word 'Hello' should be inserted at the cursor location
10.	Check that the open function works	Go to the Open option in the File menu. From the dialogue box that appears select a .txt file	The contents of the file should appear in the input box
11.	Check that the save function works	Type input as in test 1. Go to the Save option in the file menu, and name the file 'Hello.txt' and save on the desktop. Go to the desktop and double-click on file 'Hello.txt'	The string 'Hello checker' should display in notepad
12.	Check the user can print input	Type input as in test 1. Select the Print Input option in the file menu and select the local printer	'Hello checker' should print on the local printer
13.	Check the user can print output	Follow test 2. Select the Print Output option in the file menu and select the local printer	The generated system output should print on the local printer
14.	Check the system's help section works	Select Introduction from the Help menu	The user guide should open in a new window
15.	Check undo button works	Follow test 9. After performing test 9, select the undo button	The pasted in text should be removed

Continued on Next Page...

Table 10 – Continued

16.	Check redo button works	Follow test 15. After performing test 15, select the redo button	The pasted in text should reappear
17.	Check timers are zero before input	First select Tools, Time Taken and then View Spelling. Select OK and repeat for the View Likelihood option	Both message boxes should state that no input has been received
18.	Check timers are not zero after input and checking has taken place	Follow test 3. Then select Tools, Time Taken and then View Spelling. Select OK and repeat for the View Likelihood option	Both message boxes should state a time for checking
19.	Check system produces a visual display to show sentence is spelt entirely correctly	Follow test 2	System should accept the sentence, explain that it is all spelt correctly and a green tick should be shown
20.	Check system produces a visual display to show sentence is not spelt entirely correctly	Follow test 2, replacing a correct phrase with one such as 'Hellllooooo world'	System should reject the sentence, list the number of errors and display a red cross
21.	Check system shows all incorrectly spelt words	Follow test 20, then go to View All Mistakes in the Tools menu	All spelling errors should be shown in the pop-up window
22.	Check system shows the number of incorrectly spelt words	Enter 'Hellllllloooooo worllld' in the input box and select the spell check button	In the output string, it should state there are two spelling mistakes
23.	Check system states whether a string is likely or not	Follow test 3	The system should display a green tick or a red cross explaining if the sentence is likely or not

Continued on Next Page...

Table 10 – Continued

24.	Check system produces a numerical probability of sentence likelihood	Follow test 3	The system should display a probability for the string
25.	Check system presents the least likely substring to the user	Follow test 3, but enter 'The big doge instead. Then go to Tools and View Possible Typos	The substring 'big doge' should display in the pop-up window
26.	Check the system is easy to back up	Once the program is closed, right click on the file and select "Send To" from the list of options that appears. From the new list, select an available USB drive	The system should be copied onto the memory stick within a sensible period of time
27.	Check that the system is easy to use	Open the program, show it to someone who has never seen it before and get them to try and analyse a sentence's likelihood	The user should be able to spell check and then analyse a sentence's likelihood
28.	Check the text box accepts text quickly	Try to enter the sentence 'Hello Checker' into the text box	The sentence should appear almost instantaneously to when it is typed
29.	Check the Spell Check button starts the spell check procedure quickly	Enter the sentence 'Hello checker' into the text box and press the Spell Check button	The program should begin to run almost instantaneously to when it is pressed
30.	Check the system does not have excessive delays between input and output	Enter the sentence 'Hello checker' into the text box and press the Spell Check button	The program should run, and produce output almost instantaneously

Continued on Next Page...

Table 10 – Continued

31.	Check the system is programmed in a way to easily allow extensions to the code	Show the source code to a programmer of equal or greater ability and ask them to add a new type of smoothing to the system	The programmer should be able to easily add a new type of smoothing to the system
32.	Check the exit shortcut key works	While in the main menu, select ALT-F4	System should exit
33.	Check the open shortcut key works	While in the main menu, select CTRL-O	The open window should open
34.	Check the save shortcut key works	While in the main menu, select CTRL-S	The save window should open
35.	Check the print shortcut key works	While in the main menu, select CTRL-P	The print window should open
36.	Check the undo shortcut key works	While in the main menu, select CTRL-Z	The last change should be undone
37.	Check the redo shortcut key works	While in the main menu, select CTRL-Y	The last undo should be undone
38.	Check the cut shortcut key works	While in the main menu, select CTRL-X	The selected text should be removed and added into temporary memory
39.	Check the copy shortcut key works	While in the main menu, select CTRL-C	The selected text should be added to temporary memory
40.	Check the paste shortcut key works	While in the main menu, select CTRL-V	The text stored in temporary memory should be inserted into the selected part of the document

Continued on Next Page...

Table 10 – Continued

41.	Check the select all shortcut key works	While in the main menu, select CTRL-A	All the text in the input box should now be highlighted
42.	Check the view all mistakes shortcut key works	While in the main menu, select CTRL-K	All mistakes from the last spell check operation should be visible
43.	Check the view possible typos shortcut key works	While in the main menu, select CTRL-T	The most likely typo/grammatical mistake from the last likelihood check should be visible
44.	Check the view user guide introduction shortcut key works	While in the main menu, select CTRL-H	The user guide introduction should display

Table 10: Test Plan of the System

D Ethics and Standards Used

Ethics and Standards from the BCS Code of Conduct[25] and Code of Good Practise[26] were followed throughout this project, and reviewed systematically throughout. However, at the end of the project, the following analysis was undertaken to check that all codes were followed thoroughly.

D.1 Ethics

The only ethics really involved in the project could be the loss of any jobs suffered by proof-readers caused by the system taking their jobs. However, this is seen as a small risk as the project cannot be 100% accurate and so therefore there is still a need for proof-readers, much in the same way as there still was after word-processors with spell-checkers were introduced.

Furthermore, to protect the environment, throughout this project it was ensured that no hard-copies were printed unnecessarily in an effort to conserve paper and printer ink.

D.2 Standards

This project has been built conforming to the Code of Conduct standard for members of the BCS. Particular points of the code of conduct relevant to this project are discussed below:

The Public Interest:

- Point 1: This project has been performed with utmost regard for the environment
- Point 2: Regard to legitimate rights of third parties
 - As providing this software as a free download, holding no copyright or patents on what I have achieved, this should not provide an issue
- Point 3: Knowledge and understanding of relevant legislation, regulations and standards
 - Disability Discrimination Act
 - * Provided tooltips for blind users who use screenreaders and for inexperienced users
 - * Simple interface for visually impaired/short of sight
 - * System has been checked to be colourblind friendly
 - * No sound outputs so as not to exclude the deaf
 - Data Protection Act
 - * No data from user inputs will be stored and so this does not provide a problem
 - Computer Misuse Act

- Point 4: Activities were conducted without discrimination against colleagues
- Point 5: Not making any offers of bribery or inducement

Duty to Relevant Authority:

- Point 6: Work has been carried out with due care and diligence and has been attempted to design everything for ease of use for system users
- Point 7: No situations where conflict was likely to occur was encountered in this project
- Point 8: No confidential information was disclosed within the project, and so this point is not relevant
- Point 9: No attempts to misrepresent or withhold information on the product's performance have been made. All tests have been carried out, transcribed and recorded with the utmost care, and any errors contained herein would be through genuine human error as opposed to an attempt to misrepresent the system

Duty to the Profession:

- Point 10: Not relevant to the project
- Point 11: All relationships with people involved in this project have been conducted with absolute integrity
- Point 12: No opinions or official statements have been made during this project and so as such this point is irrelevant
- Point 13: Not relevant to the project

Professional Competence and Integrity:

- Point 14: Throughout this project I have been upgrading my professional knowledge and skill - both directly through the project and indirectly through other avenues
- Point 15: I have at no point misrepresented my competence or incompetence in the field. I have undertaken a project which although very challenging, has been within the acceptable boundaries of my programming and research knowledge, or within the scope of what I could learn in the six month period in which the project was being constructed.
- Point 16: Clauses considered relevant from the BCS code of good practise are listed in the section below
- Point 17: I take full professional responsibility for my work

The BCS Code of Good Practise was also conformed to in the construction of this project. The following points were regarded as relevant to this project, all other points were disregarded as irrelevant:

Practises Common to All Disciplines (Section 2)

- Maintenance of technical competence
 - IT skills improved
 - Up to date with technological advances
- Adhere to Regulations
 - Followed standards relevant to KCL course
 - Used standards in an intelligent and effective manner to achieve well-engineered results
 - System applies where relevant to the Disability Discrimination Act (2004)
 - Comply with non-discriminatory legislation
- Act professionally as a specialist
 - The boundaries of my specialist knowledge are well understood by my supervisor and myself. If at any point I felt required to cross this boundary I sought advice and at no point attempted to make misleading claims about my expertise
- Use appropriate methods and tools
 - I have kept up to date with new methods and the tools to support these methods through the duration of this project
- Manage Your Workload Efficiently
 - Any overruns were reported in the progress reports or meeting with my supervisor as they became apparent
 - I at no point undertook or committed to more assignments than I felt reasonable in a given time frame
- Participate maturely
 - Any criticism of my work was appreciated in the knowledge that the product would benefit from the best possible solutions
 - Good working relationships were maintained throughout, and no strong disagreements took place

Key IT Practises (Section 3)

- When planning

- Scope, deliverables, timescales and responsibilities were agreed in advance of the project
- Realistic estimates of timescales and resource requirements were made based on previous experience
- I was self assured that the resources required were available to complete the work
- When managing project risks
 - The temptation to identify only the manageable risks was resisted
 - A modular system was built to manage the risk of the likelihood engine not working. This would at least mean that part of the system would be completed
- When Tracking Progress
 - Metrics of all project activities were kept
 - The effort spent on each task was accurately recorded
 - Early warning of any potential deadline overruns were given either in writing or via email to my supervisor throughout the project
 - Overruns were not dealt with by cutting back in other areas within the project
- When Closing a Project
 - Honest summaries of the mistakes made, good fortune encountered and lessons learnt were made within the evaluation and conclusion section
 - Future changes to benefit later projects were recommended

Practises Specific to Education and Research Functions (Section 4)

- When Performing Research
 - The research area of the project was potentially of benefit to the university, but not to the detriment of society or the public
 - Investigation into the analysis and research of others and organisations into related topics was performed and acknowledged in the project report

Practises Specific to Business Functions (Section 5)

- None relevant, as this project was not related to a business function in any way

E Testing Screenshots

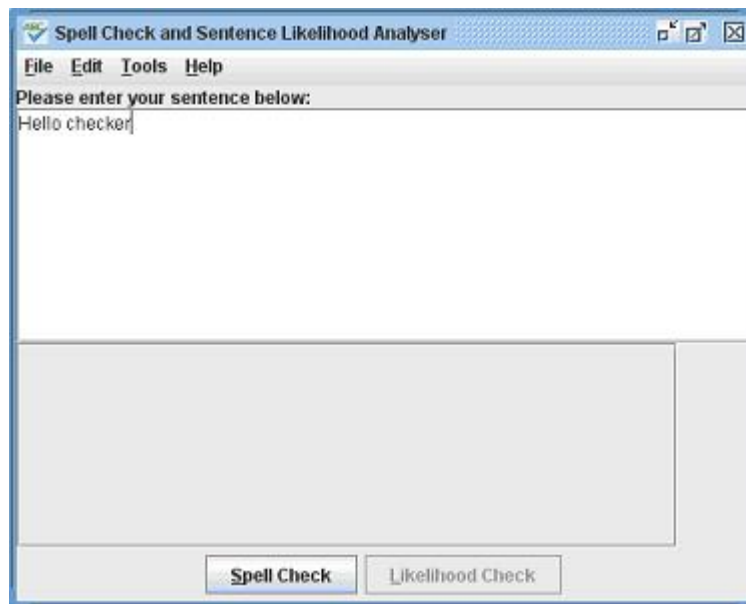


Figure 16: Test 1 Screenshot

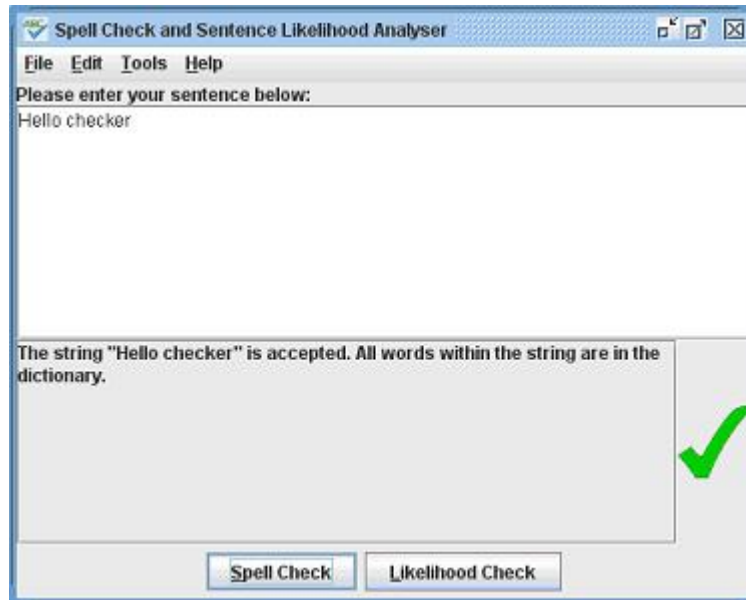


Figure 17: Test 2 Screenshot

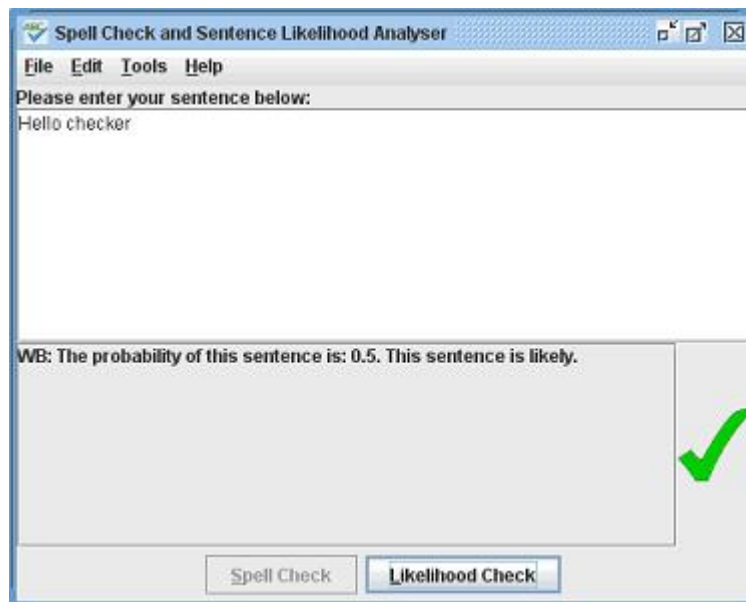
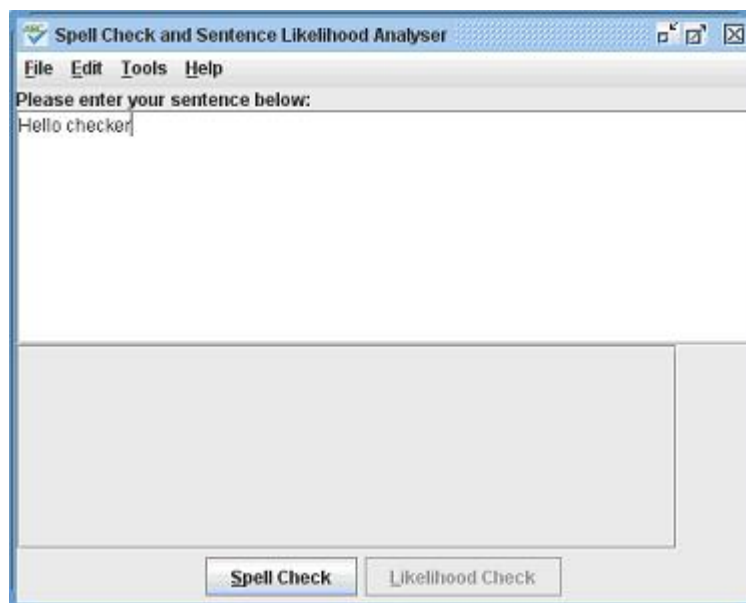


Figure 18: Test 3 Screenshot

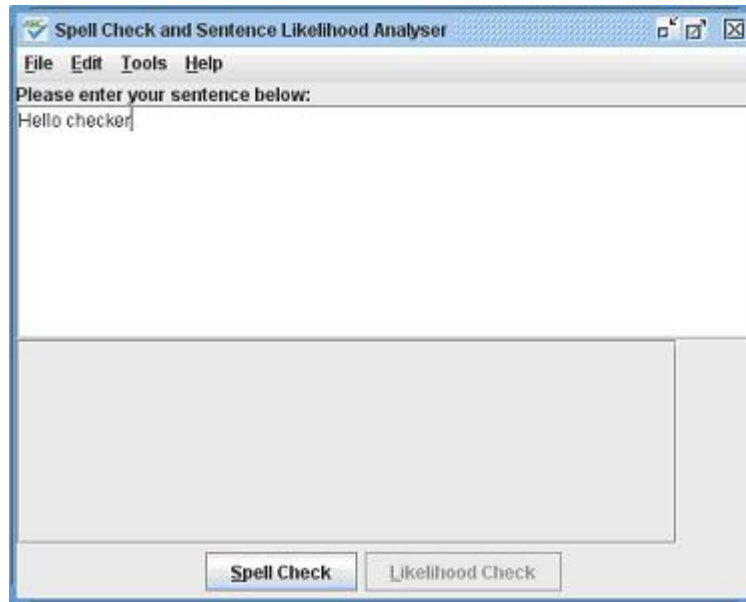


(a)



(b)

Figure 19: Test 4 Screenshots

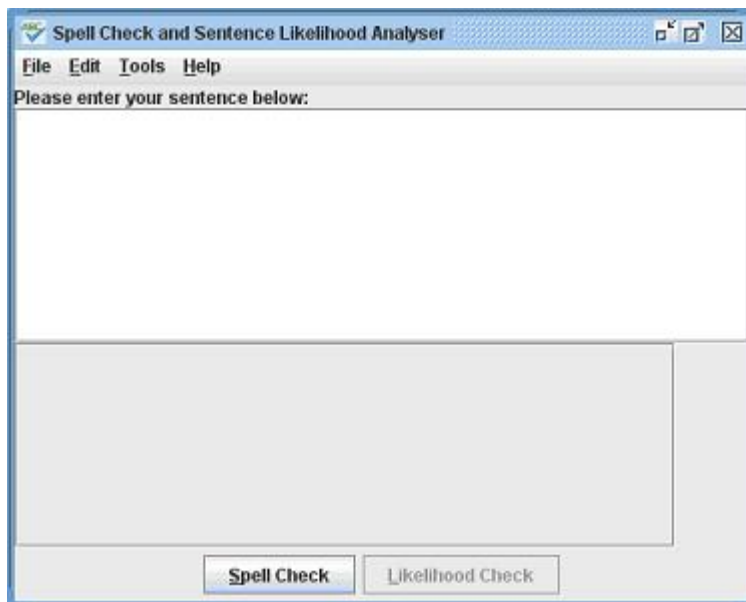


(a)



(b)

Figure 20: Test 5 Screenshots



(c)

Figure 20: Test 5 Screenshots (Continued)

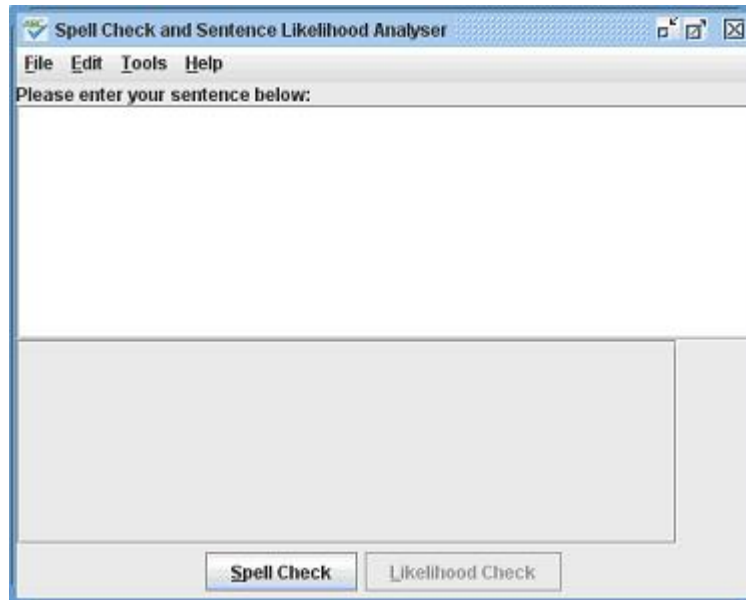


Figure 21: Test 6 Screenshot

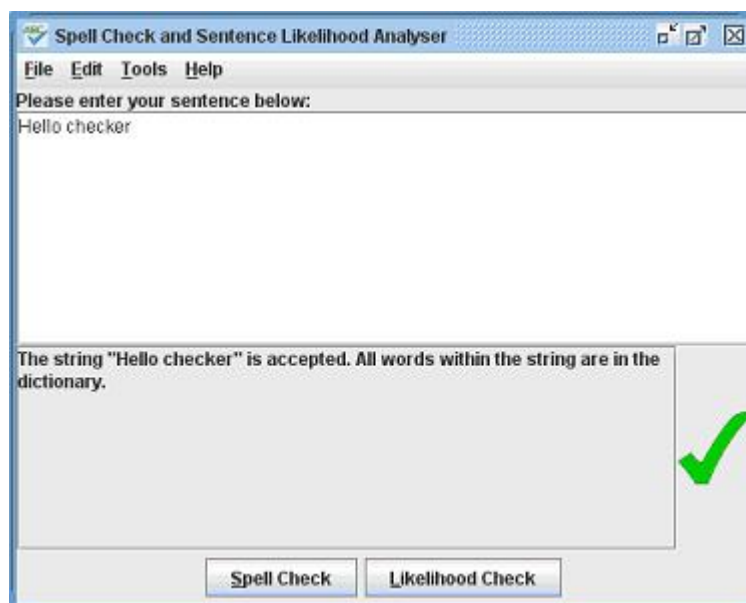


Figure 22: Test 7 Screenshot

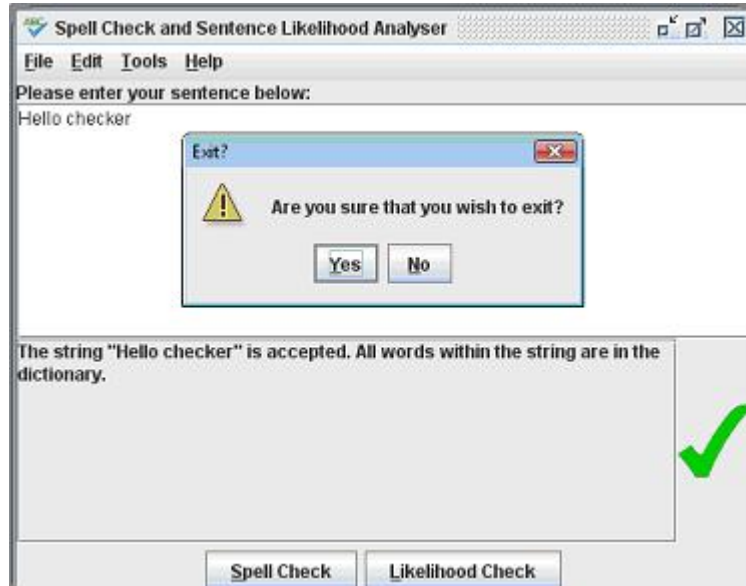
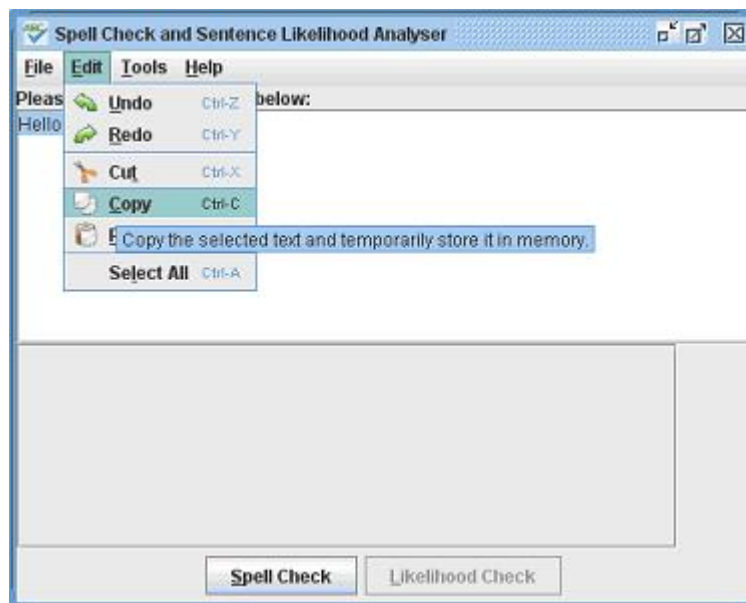
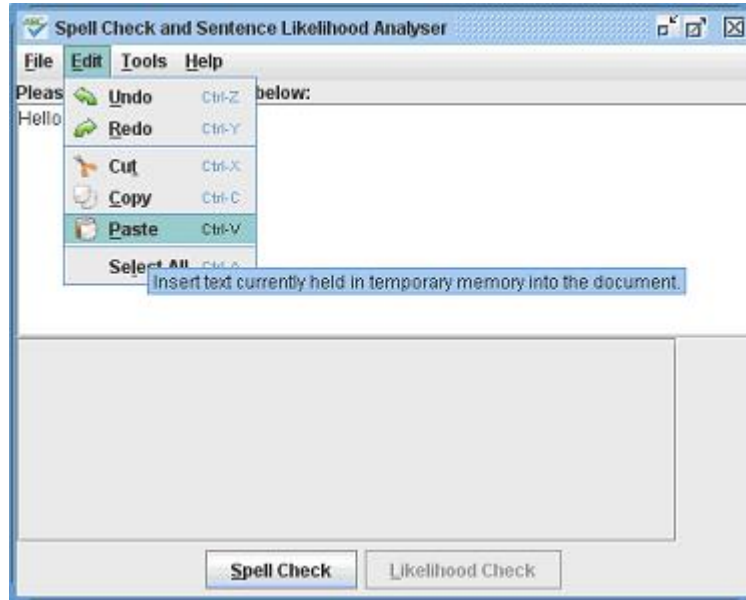


Figure 23: Test 8 Screenshot

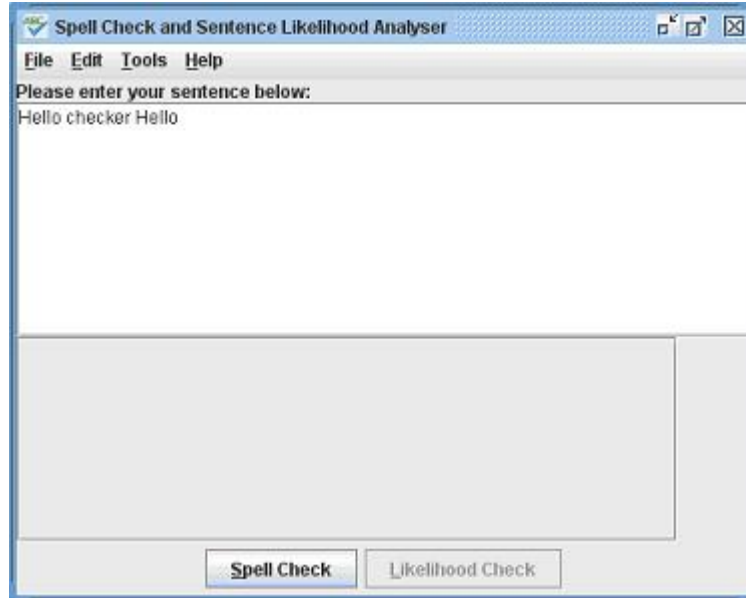


(a)

Figure 24: Test 9 Screenshots

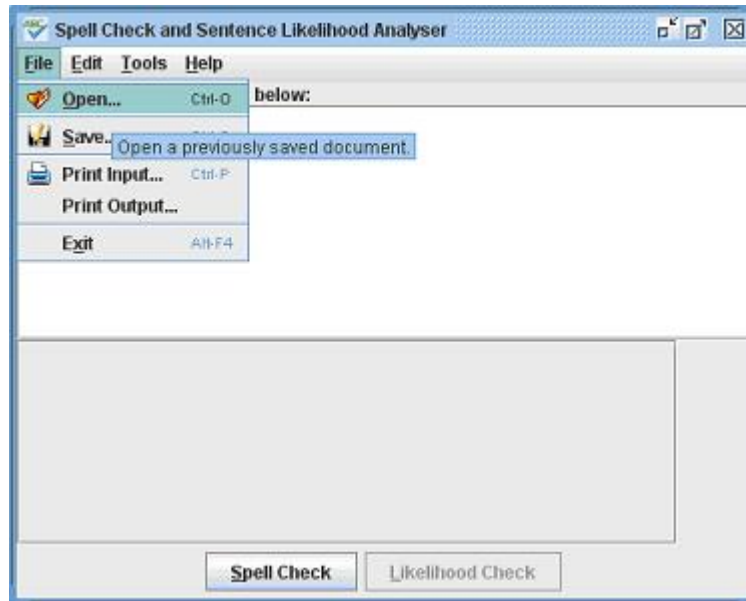


(b)

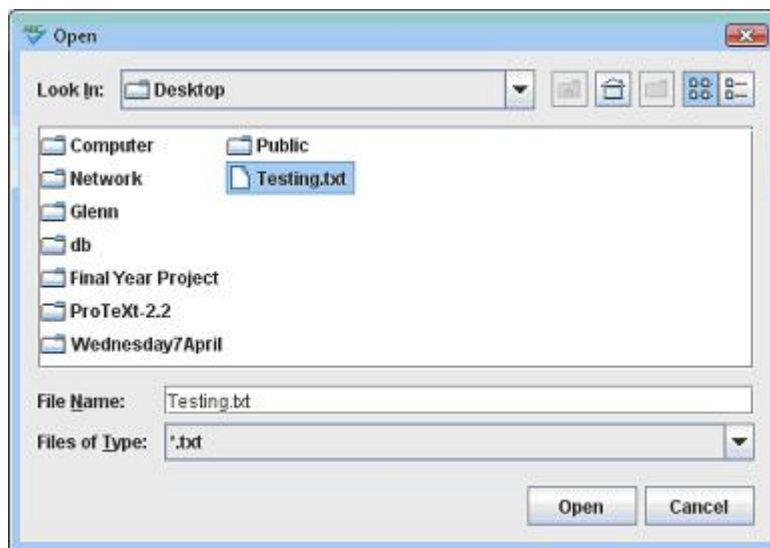


(c)

Figure 24: Test 9 Screenshots (Continued)

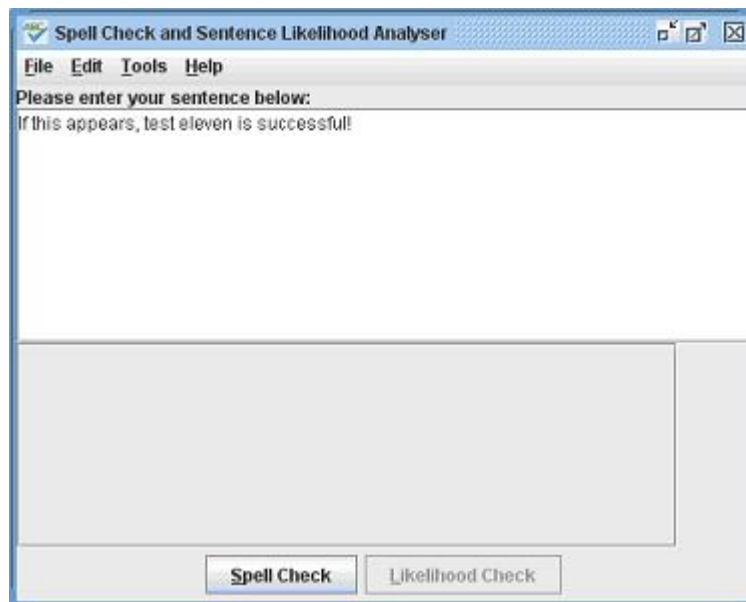


(a)



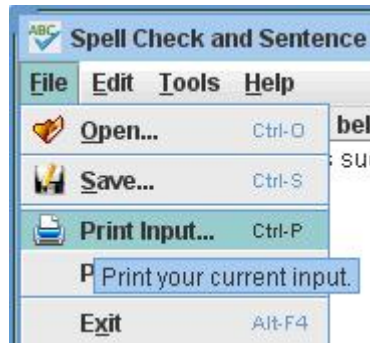
(b)

Figure 25: Test 10 Screenshots



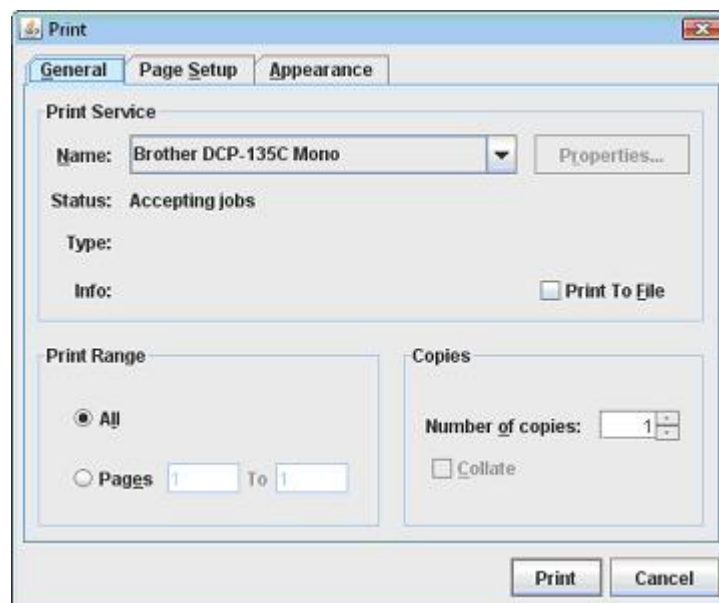
(c)

Figure 25: Test 10 Screenshots (Continued)



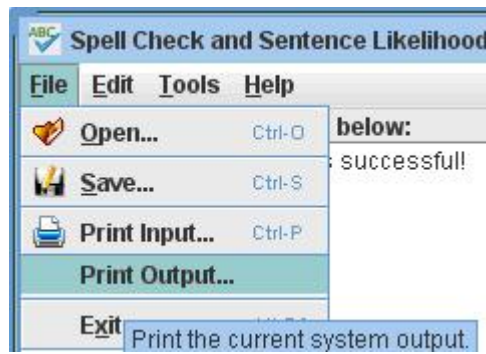
(a)

Figure 26: Test 12 Screenshots

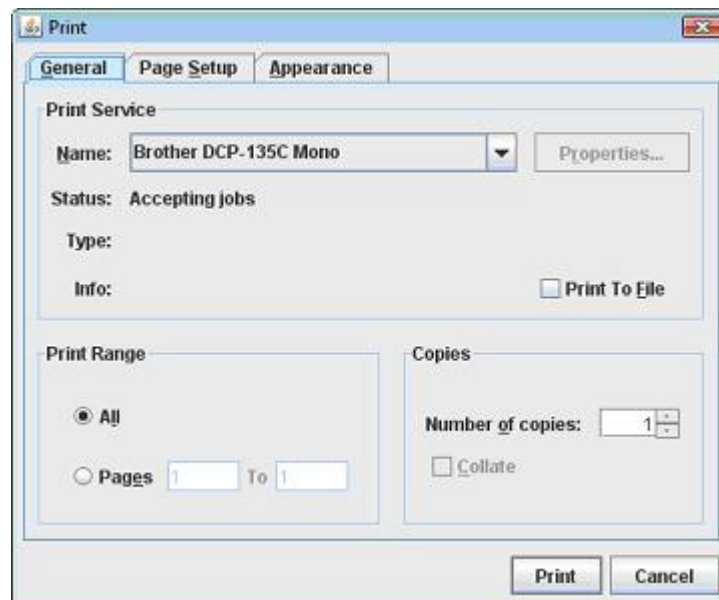


(b)

Figure 26: Test 12 Screenshots (Continued)

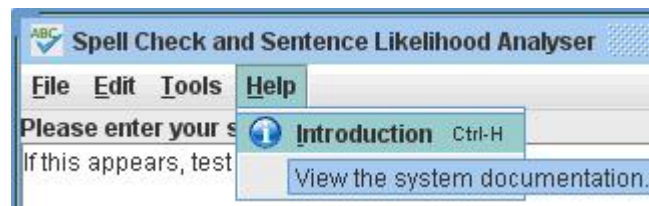


(a)

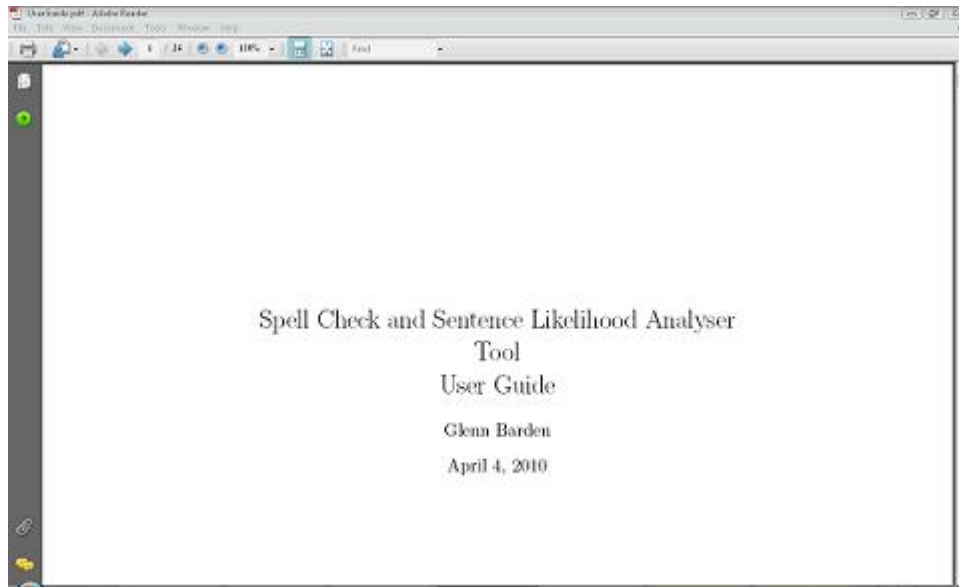


(b)

Figure 27: Test 13 Screenshots

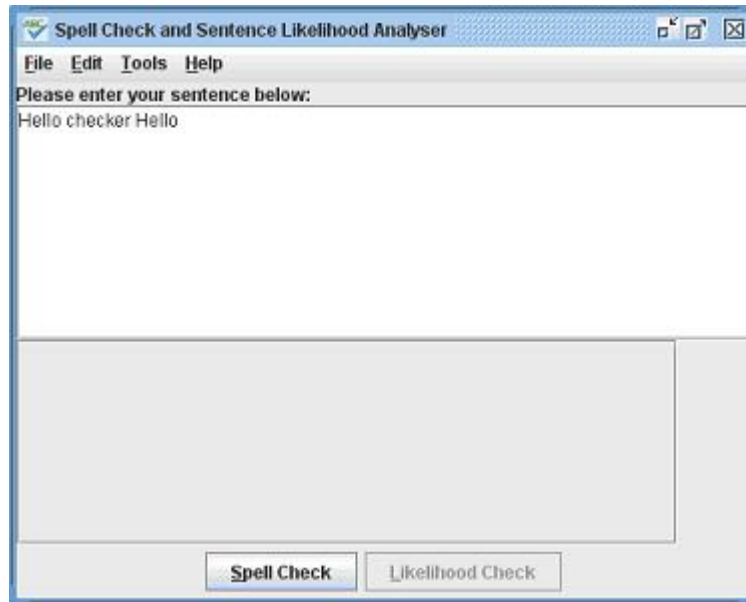


(a)



(b)

Figure 28: Test 14 Screenshots

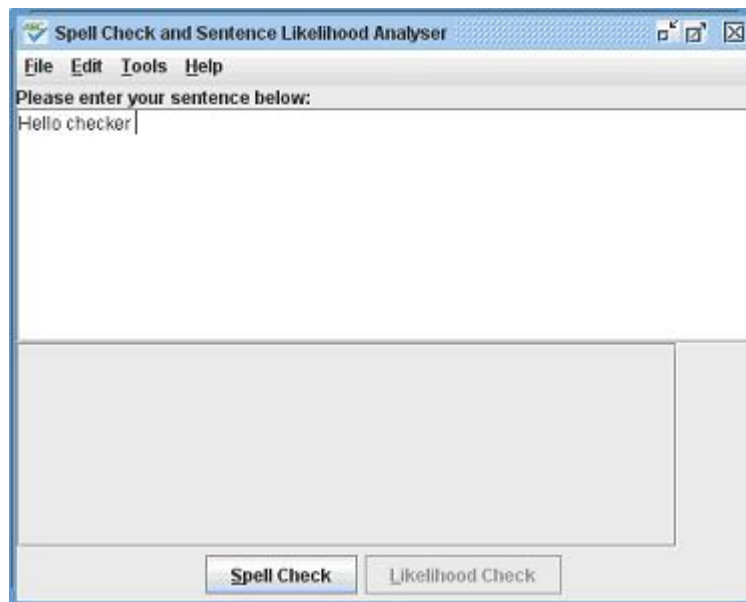


(a)



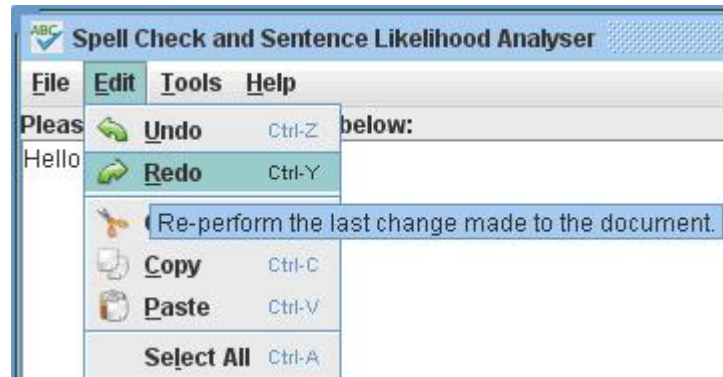
(b)

Figure 29: Test 15 Screenshots



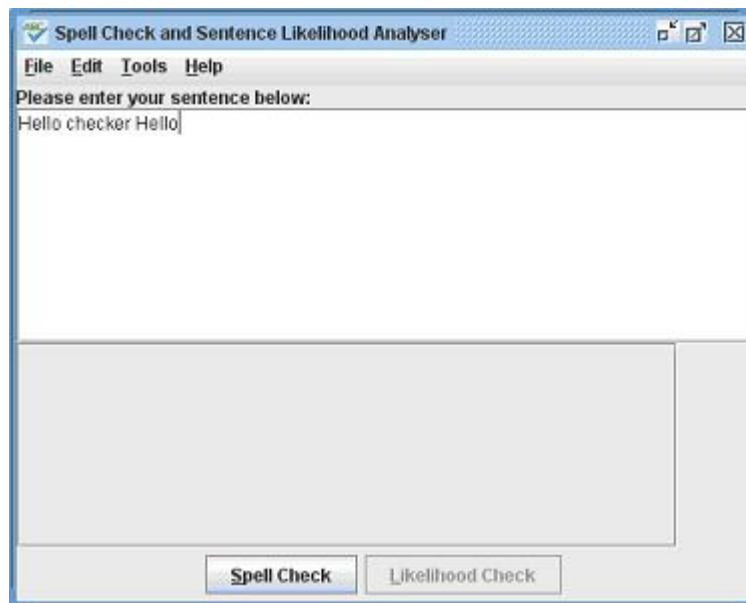
(c)

Figure 29: Test 15 Screenshots (Continued)



(a)

Figure 30: Test 16 Screenshots



(b)

Figure 30: Test 16 Screenshots (Continued)



(a)



(b)

Figure 31: Test 17 Screenshots



(a)



(b)

Figure 32: Test 18 Screenshots

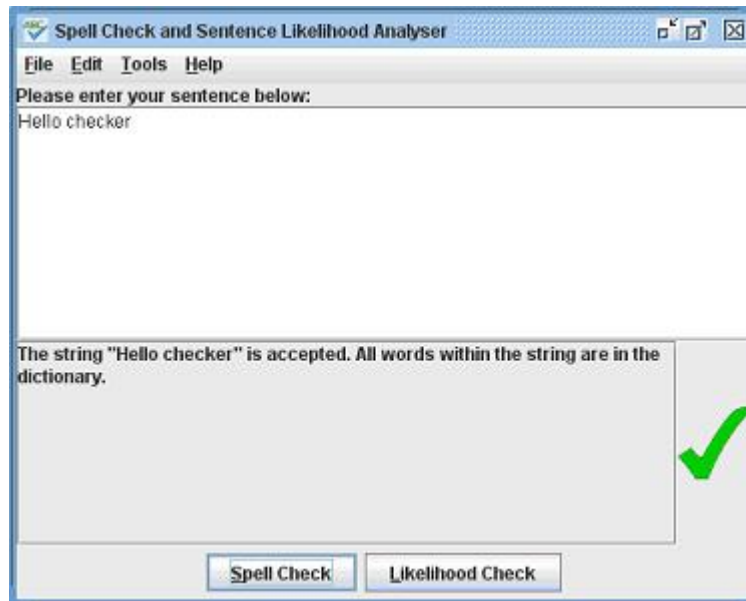


Figure 33: Test 19 Screenshot

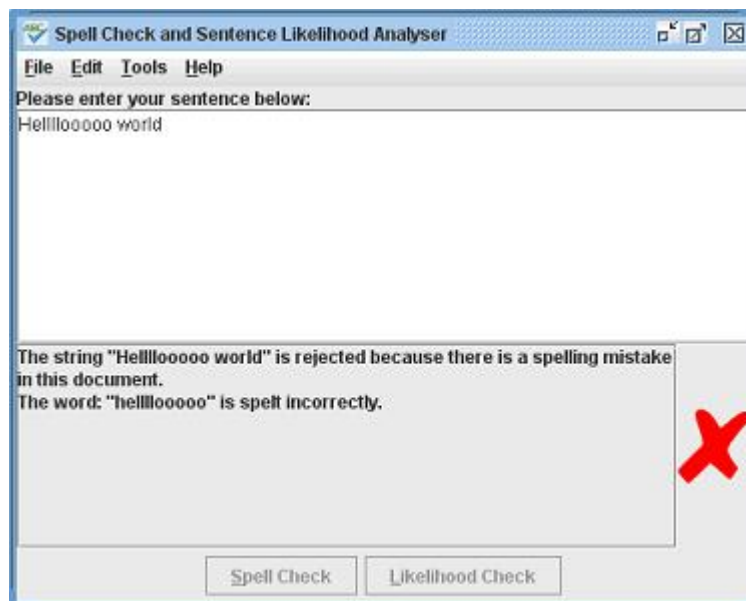


Figure 34: Test 20 Screenshot

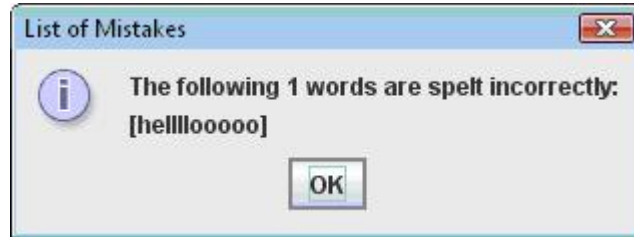


Figure 35: Test 21 Screenshot

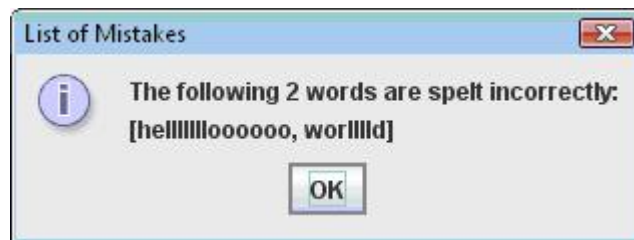


Figure 36: Test 22 Screenshot

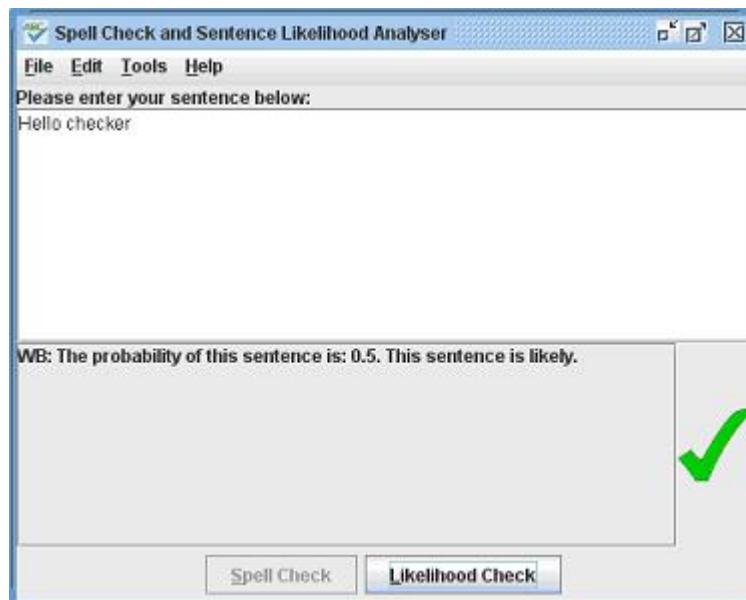
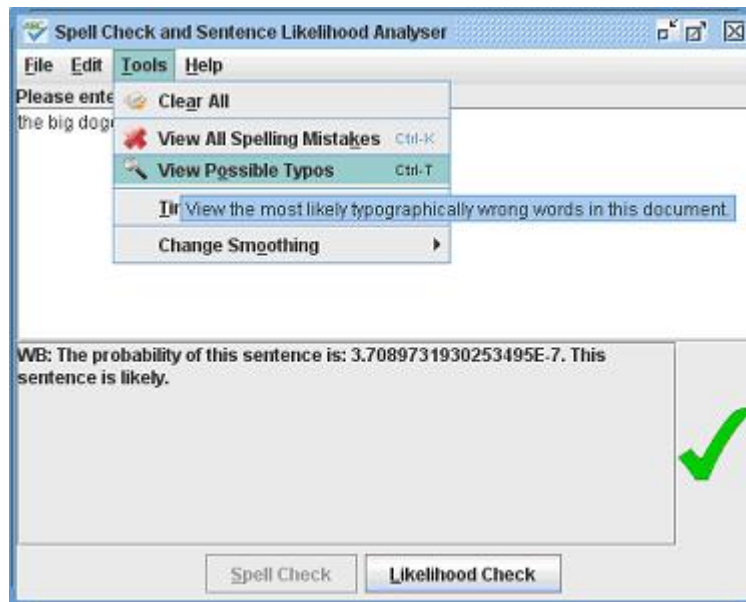
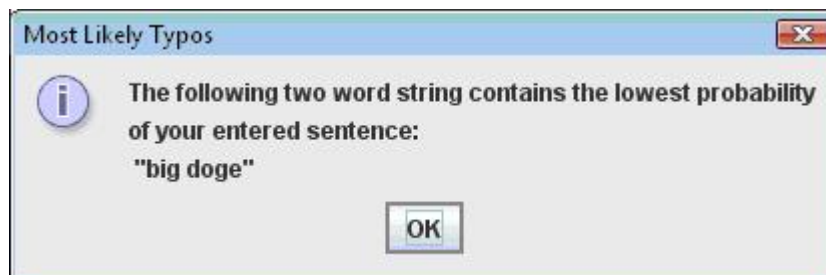


Figure 37: Test 23 Screenshot



(a)



(b)

Figure 38: Test 25 Screenshots

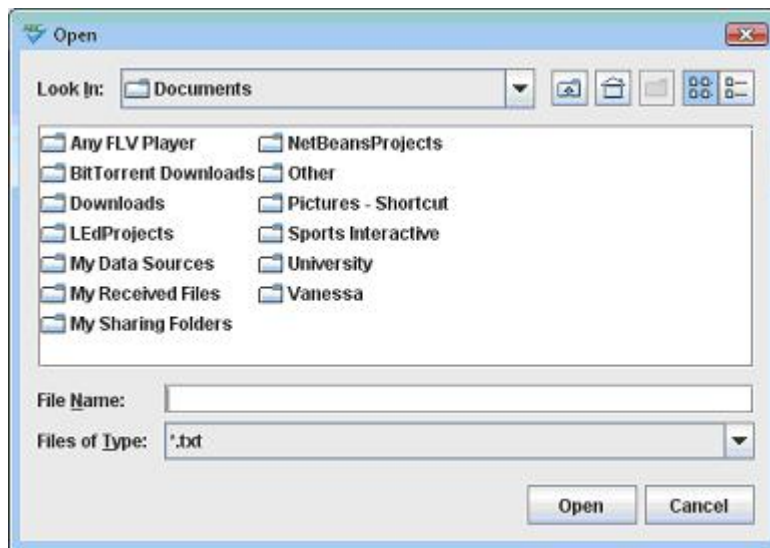


Figure 39: Test 33 Screenshot

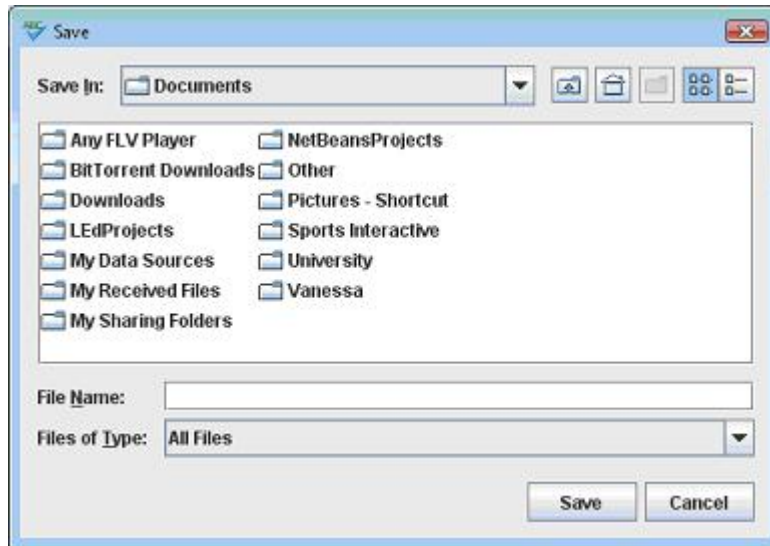


Figure 40: Test 34 Screenshot

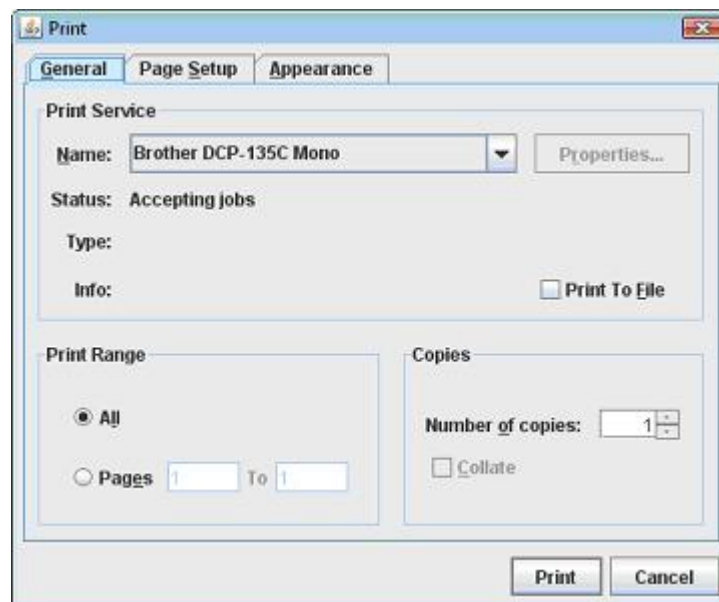
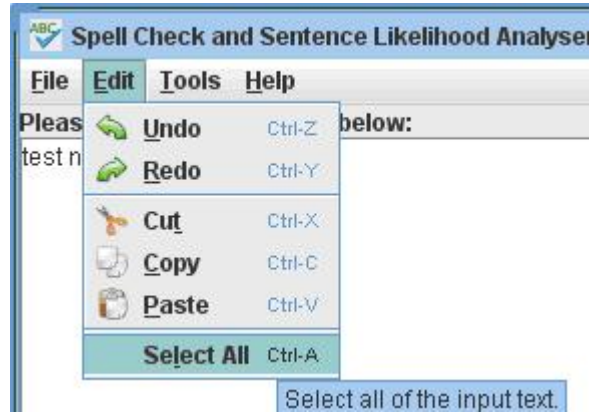
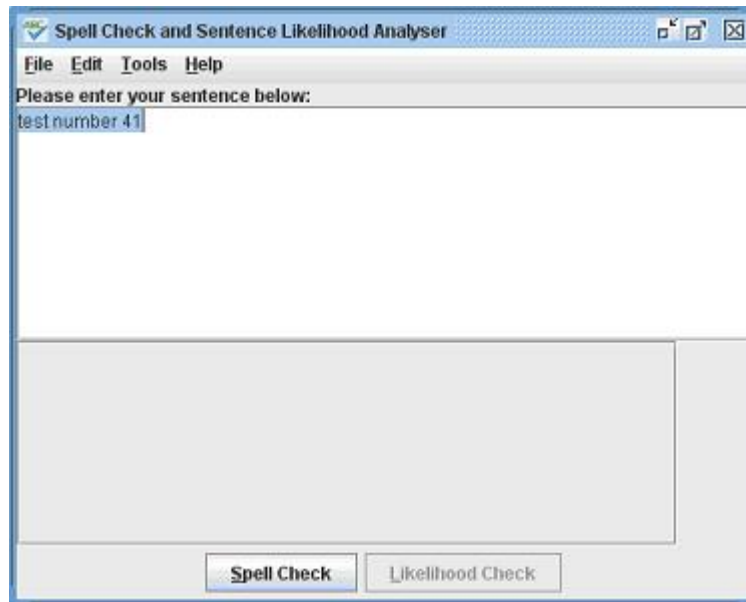


Figure 41: Test 35 Screenshot



(a)



(b)

Figure 42: Test 41 Screenshots

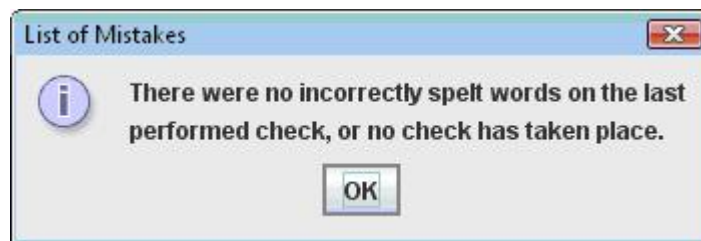


Figure 43: Test 42 Screenshot

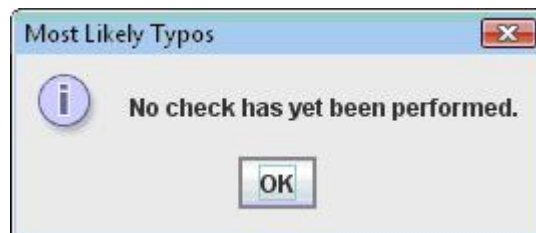


Figure 44: Test 43 Screenshot

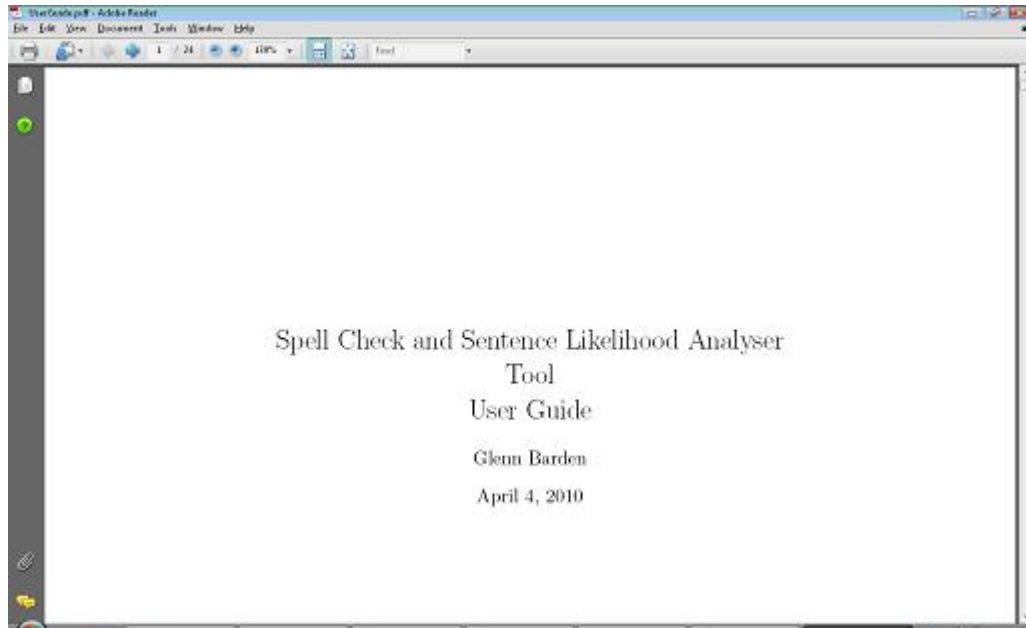


Figure 45: Test 44 Screenshot

F Program Listings

All code in the following section has been written by myself, although the following parts have been adapted from the source listed:

- TextFilter class
 - <http://www.java2s.com/Code/JavaAPI/javaw.swing/JFileChooseraddChoosableFileFilterFileFilterfilter.htm>
- JFileFilter
 - <http://forums.sun.com/thread.jspa?threadID=496612>
- Opening PDF file
 - <http://www.rgagnon.com/javadetails/java-0014.html>
- Importing .txt files
 - <http://www.java-tips.org/java-se-tips/java.io/how-to-read-file-in-java.html>
- ImageIcons
 - <http://java.sun.com/docs/books/tutorial/uiswing/components/icon.html>

F.1 Full System

F.1.1 Application Class

```
/**
 * @(#)Application.java
 * @author Glenn Barden
 * @version 1.00 2010/3/31
 */

//Import
import java.awt.* ;
import java.awt.event.* ;
import java.awt.print.*;
import java.io.*;
import java.util.*;
import javax.swing.* ;
import javax.swing.filechooser.*;
import javax.swing.undo.UndoManager;

public class Application extends JFrame implements ActionListener{
//Application components
JButton spellCheck;
JButton likelihoodCheck;
JLabel colourOutput = new JLabel();
JLabel nameLabel;
JTextArea inputBox;
JTextArea outputBox;
//Images
ImageIcon green = createImageIcon("green.png");
```

```
ImageIcon red = createImageIcon("red.png");
ImageIcon blank = createImageIcon("blank.png");
//Undo manager
UndoManager um;
//Import dictionary
File dictionary = new File("Dictionary.txt");
StringBuffer contents = new StringBuffer();
BufferedReader dictionaryReader = null;
StringBuffer contents2 = new StringBuffer();
BufferedReader reader2 = null;
//Initialise ArrayLists for wrong and correct words
ArrayList<String> wrongWords = new ArrayList<String>();
ArrayList<String> correctWords = new ArrayList<String>();
//Initialise string and double for least likely pair of words
Double leastLikely = 1.0;
String leastLikelyString = "";
//Set default smoothing as Witten Bell
int smoothing = 4;
//TIMERS
//Spelling timers
double startSpellTime;
double elapseSpellTime;
//Likelihood timers
double startLikelihoodTime;
double elapseLikelihoodTime;

//Import images
protected static ImageIcon createImageIcon(String path){
    java.net.URL imgURL = Application.class.getResource(path);
    if (imgURL != null){
        return new ImageIcon(imgURL);
    }
    else{
        System.err.println("Image not found");
        return null;
    }
}

//BEGIN SPELL CHECK
/**
 * @Pre-Condition
 * A string must be input
 * @Post-Condition
 * A result must be displayed corresponding to the input
 */
public void actionPerformed(ActionEvent event){
    if (event.getSource () == spellCheck){
        wrongWords.clear();
        correctWords.clear();
        startSpellTime = System.currentTimeMillis(); //Start timer
        String enteredSentence = inputBox.getText();
        //Read from file Dictionary.txt
        try{
            dictionaryReader = new BufferedReader(new FileReader(dictionary));
            String text = null;
            while ((text = dictionaryReader.readLine()) != null){
                contents.append(text).append(System.getProperty("line.separator"));
            }
        }
        catch (FileNotFoundException e){
            e.printStackTrace();
        }
        catch (IOException e){
            e.printStackTrace();
        }
        finally{
            try{
                if (dictionaryReader != null){
```

```

dictionaryReader.close();
}
}
catch (IOException e){
e.printStackTrace();
}
}
//Transform dictionary into a string
String dictionary = contents.toString();
//Remove punctuation from input string
String enteredSentenceNoStops = enteredSentence.replace(".", "");
String enteredSentenceNoCommas = enteredSentenceNoStops.replace(",", "");
String enteredSentenceNoExclamation = enteredSentenceNoCommas.replace("!", "");
String enteredSentenceNoQuestion = enteredSentenceNoExclamation.replace("?", "");
String enteredSentenceNoSpeech = enteredSentenceNoQuestion.replace("\'", "");
String wholeSentence = enteredSentenceNoSpeech.replace("-", " ");
String[] words = wholeSentence.toLowerCase().split(" ");

    for (int i=0; i < words.length; i++){ //Loop through the input string
    if (words[i].isEmpty()){
        wrongWords.add("EMPTY STRING"); //If no input, set the unique empty string value
    }
    else if (dictionary.contains(words[i])){
        correctWords.add(words[i]);
    }
    else{
        wrongWords.add(words[i]);
    }
    }

    if (wrongWords.isEmpty()){ //Accept
        outputBox.setText("The string \"" + enteredSentence + "\" is accepted. All words
within the string are in the dictionary.");
        likelihoodCheck.setEnabled(true);
        colourOutput.setIcon(green);
        inputBox.setEditable(false);
    }
    else if (wrongWords.contains("EMPTY STRING")){ //Reject unique empty string
        outputBox.setText("No string has been entered.");
        colourOutput.setIcon(red);
        inputBox.setEditable(true);
    }
    else if (wrongWords.size()==1){ //Reject single spelling mistake
        outputBox.setText("The string \"" + enteredSentence + "\" is rejected because
there is a spelling mistake in this document.\n" + "The word: \""+wrongWords.get(0)
+"\" is spelt incorrectly.");
        likelihoodCheck.setEnabled(false);
        colourOutput.setIcon(red);
    }
    else{ //Reject multiple spelling mistake
        outputBox.setText("The string is rejected because there are "+wrongWords.size()
+" spelling mistakes in this document.\n" + "The word: \""+wrongWords.get(0)
+ "\" is the first mistake.");
        likelihoodCheck.setEnabled(false);
        colourOutput.setIcon(red);
    }
    }

    elapsedTime = System.currentTimeMillis()-startSpellTime; //End timer
    and subtract from start time
    }

    //BEGIN LIKELIHOOD CHECK
    /**
    * @Pre-Condition
    * A string must be input
    * @Post-Condition
    * A result must be displayed corresponding to the input. This result must consist of:
    * Numerical Answer between 0 and 1
    * Colour output icon

```



```

        for(int i = 0; i < words.length; i++){
            length++;
        }

//Begin for loop to split input into single and two-word strings
for (int i=length; i>1 ; i--){
    String w0 = words[i-2];
    String w1 = words[i-1];
    String w0w1 = new String(w0+ " " +w1);

/**
 * @UES Smoothing
 * Takes the length of the input and performs the calculation  $1/V^{\{length\}}$ 
 * Results checked and verified 10/4
 * @author Glenn Barden
 * @version Final
 */

    if (smoothing == 0){ //Start uniform event space (UES)
convert = Integer.toString(length);
power = Double.parseDouble(convert);
UES = (1.0/Math.pow(V,power));
leastLikelyString = "UES";
if (UES>=thresholdValueUES){ //likely
    outputBox.setText("UES: The probability of this sentence is: "+UES
+". This sentence is likely.");
    likelihoodCheck.setEnabled(true);
    spellCheck.setEnabled(false);
    colourOutput.setIcon(green);
    inputBox.setEditable(false);
}
    else{ //unlikely
outputBox.setText("UES: The probability of this sentence is: "+UES
+". This sentence is unlikely.");
    likelihoodCheck.setEnabled(true);
    spellCheck.setEnabled(false);
    colourOutput.setIcon(red);
    inputBox.setEditable(false);
}
    }
elapsedLikelihoodTime = System.currentTimeMillis()-startLikelihoodTime ; //Stop timer
break;
} //End uniform event space (UES)

/**
 * @FES Smoothing
 * Takes each word individually and compares with the result for that word in
'SingleCorpus.txt'
 * The number retrieved is then divided by the sumOfSingleCorpus
 * 'Zero' results checked and verified 10/4
 * 'Non-zero' results checked and verified 10/4
 * @author Glenn Barden
 * @version Final
 */

else if (smoothing == 1){ //Start frequentist event space (FES)
String[] wordsFES = wholeSentence.toLowerCase().split(" ");
for (int j=length-1; j>-1 ; j--){
try{
    FileInputStream fstream3 = new FileInputStream("SingleCorpus.txt");
    DataInputStream FESCalculation = new DataInputStream(fstream3);
    BufferedReader brSingleDictionary = new BufferedReader
(new InputStreamReader(FESCalculation));
    while ((line2 = brSingleDictionary.readLine()) != null){
        linecount2++;
        numeratorFES=(line2.split(delimiter));
        if(numeratorFES[0].equals(wordsFES[j])){
            numeratorIntFES = Double.parseDouble(numeratorFES[1]);
            FES = FES*(numeratorIntFES/sumOfSingleCorpus);

```

```

        if ((numeratorIntFES/sumOfSingleCorpus)<leastLikely){
            leastLikely = (numeratorIntFES/sumOfSingleCorpus);
            leastLikelyString = w0w1;
        }
        else {
            //no body to else loop
        }
        counterFES++;
        break;
    }
    else if (!numeratorFES[0].equals(wordsFES[j])) {
        leastLikelyString = w0w1;
    }
}
}
}
catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}
}
if (FES>=thresholdValueFES && counterFES == length){ //likely
    outputBox.setText("FES: The probability of this sentence is: "+FES
    +". This sentence is likely.");
    likelihoodCheck.setEnabled(true);
    spellCheck.setEnabled(false);
    colourOutput.setIcon(green);
    inputBox.setEditable(false);
}
else if (counterFES != length){ //unlikely
    outputBox.setText("FES: The probability of this sentence is: 0.
    This sentence is unlikely.");
    likelihoodCheck.setEnabled(true);
    spellCheck.setEnabled(false);
    colourOutput.setIcon(red);
    inputBox.setEditable(false);
}
else{ //unlikely
    outputBox.setText("FES: The probability of this sentence is: "+FES
    +". This sentence is unlikely.");
    likelihoodCheck.setEnabled(true);
    spellCheck.setEnabled(false);
    colourOutput.setIcon(red);
    inputBox.setEditable(false);
}
}
elapsedLikelihoodTime = System.currentTimeMillis()-startLikelihoodTime ; //Stop timer
break;
} //End frequentist event space (FES)

/**
 * @MLE Smoothing
 * Takes each two word substring and compares with the result for those two words together
 * in 'DoubleCorpus.txt'
 * Then takes the first word of each two word substring and compares with the result for
 * that word in 'SingleCorpus.txt'
 * If the word is not found in the singleCorpus, there is no chance of any result apart
 * from 0 and so the counter placed here
 * ensures that if not every word is found, the result cannot be anything apart from 0
 * 'Zero' results checked and verified 11/4
 * 'Non-zero' results checked and verified 11/4
 * @author Glenn Barden
 * @version Final
 */

        else if (smoothing == 2){ //Start maximum likelihood estimation (MLE)
            try{
                FileInputStream fstream = new FileInputStream("DoubleCorpus.txt");
                DataInputStream in = new DataInputStream(fstream);
                BufferedReader brDictionary = new BufferedReader(new InputStreamReader(in));
                while ((line = brDictionary.readLine()) != null){

```

```

        linecount++;
        numerator=(line.split(delimiter));
        if(numerator[0].equals(w0w1)){
            numeratorInt = Double.parseDouble(numerator[1]);
            break;
        }
        else{
            numeratorInt = 0.0;
        }
    }
}
catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}

    try{
        FileInputStream fstream2 = new FileInputStream("SingleCorpus.txt");
        DataInputStream mate = new DataInputStream(fstream2);
        BufferedReader brSingleDictionary = new BufferedReader(new InputStreamReader(mate));
        while ((line2 = brSingleDictionary.readLine()) != null){
            linecount2++;
            denominator=(line2.split(delimiter));
            if(denominator[0].equals(w0)){
                denominatorInt = Double.parseDouble(denominator[1]);
                MLE = MLE*(numeratorInt/denominatorInt);
            }
            if ((numeratorInt/denominatorInt)<leastLikely){
                leastLikely = (numeratorInt/denominatorInt);
                leastLikelyString = w0w1;
            }
            else {
                //no body to else loop
            }
            counter++;
            break;
        }
        else{
            denominatorInt = 0.0;
        }
    }
}
catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}

    if (MLE>=thresholdValueMLE && counter == length){ //likely
        outputBox.setText("MLE: The probability of this sentence is: "+MLE
        +". This sentence is likely.");
        likelihoodCheck.setEnabled(true);
        spellCheck.setEnabled(false);
        colourOutput.setIcon(green);
        inputBox.setEditable(false);
    }
    else if (counter != length || MLE == 0.0){ //unlikely with zero probability
        outputBox.setText("MLE: The probability of this sentence is: 0.
        This sentence is unlikely.");
        likelihoodCheck.setEnabled(true);
        spellCheck.setEnabled(false);
        colourOutput.setIcon(red);
        inputBox.setEditable(false);
        leastLikelyString = "MLE";
    }
    else{ //unlikely with non-zero probability
        outputBox.setText("MLE: The probability of this sentence is: "+MLE
        +". This sentence is unlikely.");
        likelihoodCheck.setEnabled(true);
        spellCheck.setEnabled(false);
        colourOutput.setIcon(red);
        inputBox.setEditable(false);
    }
}

```

```

        elapseLikelihoodTime = System.currentTimeMillis()-startLikelihoodTime ; //Stop timer
    } //End maximum likelihood estimation (MLE)

/**
 * @PlusOne Smoothing
 * Takes each two word substring and compares with the result for those two words
 * together in 'DoubleCorpus.txt'
 * Then takes the first word of each two word substring and compares with the result for
 * that word in 'SingleCorpus.txt'
 * The result here, unlike MLE can never equal zero
 * Once the checks with the corpora are performed, the value from the Double Corpus
 * (numerator) has one added to it
 * The value from the Single Corpus (denominator) then has the number of V added to it
 * Numerator is divided by denominator to obtain the result
 * 'Zero' results checked and verified as impossible 11/4
 * 'Non-zero' results checked and verified 11/4
 * @author Glenn Barden
 * @version Final
 */

else if (smoothing == 3){ //Start plus one smoothing
try{
    FileInputStream fstream = new FileInputStream("DoubleCorpus.txt");
    DataInputStream in = new DataInputStream(fstream);
    BufferedReader brDictionary = new BufferedReader(new InputStreamReader(in));
    while ((line = brDictionary.readLine()) != null){
        linecount++;
        numerator=(line.split(delimiter));
        if(numerator[0].equals(w0w1)){
            numeratorInt = Double.parseDouble(numerator[1]);
            break;
        }
        else{
            numeratorInt = 0.0;
        }
    }
}
catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}

    try{
        FileInputStream fstream2 = new FileInputStream("SingleCorpus.txt");
        DataInputStream mate = new DataInputStream(fstream2);
        BufferedReader brSingleDictionary = new BufferedReader(new InputStreamReader(mate));
        while ((line2 = brSingleDictionary.readLine()) != null){
            linecount2++;
            denominator=(line2.split(delimiter));
            if(denominator[0].equals(w0)){
                counter2 = 0;
                denominatorInt = Double.parseDouble(denominator[1]);
                plusOne = plusOne*((numeratorInt+1)/(denominatorInt+V));
                if (((numeratorInt+1)/(denominatorInt+V))<leastLikely){
                    leastLikely = ((numeratorInt+1)/(denominatorInt+V));
                    leastLikelyString = w0w1;
                }
            }
            else {
                //no body to else loop
            }
            break;
        }
        else if(!denominator[0].equals(w1)){
            counter2++;
            if (counter2 == length){
                plusOne = plusOne*((numeratorInt+1)/(V));
                if (((numeratorInt+1)/(V))<leastLikely){
                    leastLikely = ((numeratorInt+1)/(V));
                    leastLikelyString = w0w1;
                }
            }
        }
    }
}

```

```

    }
    else {
        //no body to else loop
    }
    counter2 = 0;
    break;
}
else{
    //No body to else loop
}
}
}

    linecount2 = 0;
    counter2= 0;
}
catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}

    if (plusOne>=thresholdValuePlusOne){ //likely
        outputBox.setText("A0: The probability of this sentence is: "+plusOne
        +". This sentence is likely.");
        likelihoodCheck.setEnabled(true);
        spellCheck.setEnabled(false);
        colourOutput.setIcon(green);
        inputBox.setEditable(false);
    }
    else{ //unlikely
        outputBox.setText("A0: The probability of this sentence is: "+plusOne
        +". This sentence is unlikely.");
        likelihoodCheck.setEnabled(true);
        spellCheck.setEnabled(false);
        colourOutput.setIcon(red);
        inputBox.setEditable(false);
    }
    elapseLikelihoodTime = System.currentTimeMillis()-startLikelihoodTime ; //Stop timer
} //End plus one smoothing

/**
 * @Witten Bell Smoothing
 * First of all this computes the MLE result
 * If MLE is zero, the zero branch is computed
 * If MLE is non-zero, the non-zero branch is computed
 * 'Zero' results in MLE calculation checked and verified as possible 16/4
 * 'Non-zero' results in MLE calculation checked and verified 16/4
 * @author Glenn Barden
 * @version Final
 */

else if (smoothing == 4){ //Start Witten Bell smoothing
try{
    FileInputStream fstream = new FileInputStream("DoubleCorpus.txt");
    DataInputStream in = new DataInputStream(fstream);
    BufferedReader brDictionary = new BufferedReader(new InputStreamReader(in));
    while ((line = brDictionary.readLine()) != null){
        linecount++;
        numerator=(line.split(delimiter));
        if(numerator[0].equals(w0w1)){
            numeratorInt = Double.parseDouble(numerator[1]);
            break;
        }
        else{
            numeratorInt = 0.0;
        }
    }
}
catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}
}

```

```

    }

    try{
        FileInputStream fstream2 = new FileInputStream("SingleCorpus.txt");
        DataInputStream mate = new DataInputStream(fstream2);
        BufferedReader brSingleDictionary = new BufferedReader(new InputStreamReader(mate));
        while ((line2 = brSingleDictionary.readLine()) != null){
            linecount2++;
            denominator=(line2.split(delimiter));
            if(denominator[0].equals(w0)){
                denominatorInt = Double.parseDouble(denominator[1]);
                MLE = MLE*(numeratorInt/denominatorInt);
                counter++;
                break;
            }
            else{
                denominatorInt = 0.0;
            }
        }
    }
    catch (Exception e){
        System.err.println("Error: " + e.getMessage());
    }
}

/**
 * @NON-ZERO Witten-Bell Count
 * The two-word substring is compared against the DoubleCorpus
 * The first word in each substring's T value is taken from the TCorpus
 * The first word in each substring's value is taken from the SingleCorpus
 * Answer is calculated by dividing countValue by NValue plus TValueInt
 * Results checked and verified 16/4 (checked as accurate with reduced corpora)
 * @author Glenn Barden
 * @version Final
 */
if (MLE != 0){
    String[] wordsWB = wholeSentence.toLowerCase().split(" ");

    for (int k=length-offset; k>0 ; k--){
        String WBO = wordsWB[k-1];
        String WB1 = wordsWB[k];
        String WBOWB1 = new String(WBO+ " " +WB1);

        try{
            FileInputStream fstreamWBDouble = new FileInputStream("DoubleCorpus.txt");
            DataInputStream inWBDouble = new DataInputStream(fstreamWBDouble);
            BufferedReader brDictionaryWBDouble = new BufferedReader
(new InputStreamReader(inWBDouble));
            while ((lineWB = brDictionaryWBDouble.readLine()) != null){
                linecount++;
                String[] countWB =(lineWB.split(delimiter));
                if(countWB[0].equals(WBOWB1)){
                    countValue = Double.parseDouble(countWB[1]);
                    break;
                }
                else {
                    //No body to else loop
                }
            }
        }
        catch (Exception e){
            System.err.println("Error: " + e.getMessage());
        }
    }

    //N AND T VALUES
    try{
        FileInputStream fstreamNew = new FileInputStream("T.txt");
        DataInputStream inNew = new DataInputStream(fstreamNew);
        BufferedReader brDictionaryNew = new BufferedReader(new InputStreamReader(inNew));

```

```

while ((lineWBNew = brDictionaryNew.readLine()) != null){
    linecount++;
    String[] TValue =(lineWBNew.split(delimiter));
    if(TValue[0].equals(WB0)){
        TValueInt = Double.parseDouble(TValue[1]);
    }
    else{
        //No body to else loop
    }
}
}
catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}

try{
    FileInputStream fstream2 = new FileInputStream("SingleCorpus.txt");
    DataInputStream mate = new DataInputStream(fstream2);
    BufferedReader brSingleDictionary = new BufferedReader(new InputStreamReader(mate));
    while ((line2 = brSingleDictionary.readLine()) != null){
        linecount2++;
        String[] NValue =(line2.split(delimiter));
        if(NValue[0].equals(WB0)){
            NValueInt = Double.parseDouble(NValue[1]);
            break;
        }
        else {
            //No body to else loop
        }
    }
}
catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}

WittenBellCalculator = WittenBellCalculator*(countValue/(NValueInt+TValueInt));
if ((countValue/(NValueInt+TValueInt))<leastLikely){
    leastLikely = (countValue/(NValueInt+TValueInt));
    leastLikelyString = WBOWB1;
}
else{
    //No body to else loop
}
offset++;
break;
}
}

/**
 * @ZERO Witten-Bell Count
 * The first word in each substring's T value is taken from the TCorpus
 * The first word's Z Value is calculated by subtracting TValue from V
 * The first word in each substring's value is taken from the SingleCorpus
 * Answer is calculated by dividing TValue by ((ZValueInt*NValueInt)+TValueInt
 * Results checked and verified 16/4 (checked as accurate with reduced corpora)
 * @author Glenn Barden
 * @version Final
 */
else{
    System.out.println("ZERO LOOP");
    String[] wordsWB = wholeSentence.toLowerCase().split(" ");

    for (int k=length-1; k>0 ; k--){
        String WB0 = wordsWB[k-1];
        String WB1 = wordsWB[k];
        String WBOWB1 = new String(WB0+ " " +WB1);

        try{
            FileInputStream fstream = new FileInputStream("T.txt");

```

```

DataInputStream in = new DataInputStream(fstream);
BufferedReader brDictionary = new BufferedReader(new InputStreamReader(in));
while((lineWB = brDictionary.readLine()) != null){
    linecount++;
    String[] TValue =(lineWB.split(delimiter));
    if(TValue[0].equals(WB0)){
        TValueInt = Double.parseDouble(TValue[1]);
        ZValueInt = V - TValueInt;
        break;
    }
    else{
        TValueInt = 0.0;
        ZValueInt = V - TValueInt;
    }
}
}
catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}

try{
    FileInputStream fstream2 = new FileInputStream("SingleCorpus.txt");
    DataInputStream mate = new DataInputStream(fstream2);
    BufferedReader brSingleDictionary = new BufferedReader(new InputStreamReader(mate));
    while((line2 = brSingleDictionary.readLine()) != null){
        linecount2++;
        String[] NValue =(line2.split(delimiter));
        if(NValue[0].equals(WB0)){
            NValueInt = Double.parseDouble(NValue[1]);
            break;
        }
        else {
            //No body to else loop
        }
    }
}
catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}

WittenBellCalculator =
WittenBellCalculator*(TValueInt / ((ZValueInt*NValueInt)+TValueInt));
    if ((TValueInt / ((ZValueInt*NValueInt)+TValueInt))<leastLikely){
        leastLikely = (TValueInt / ((ZValueInt*NValueInt)+TValueInt));
        leastLikelyString = WBOWB1;
    }
    else{
        //No body to else loop
    }
break;
}
}

if (WittenBellCalculator>=thresholdValueWB){ //likely
    outputBox.setText("WB: The probability of this sentence is: "+WittenBellCalculator+
". This sentence is likely.");
    likelihoodCheck.setEnabled(true);
    spellCheck.setEnabled(false);
    colourOutput.setIcon(green);
    inputBox.setEditable(false);
}
else{ //unlikely
    outputBox.setText("WB: The probability of this sentence is: "+WittenBellCalculator+
". This sentence is unlikely.");
    likelihoodCheck.setEnabled(true);
    spellCheck.setEnabled(false);
    colourOutput.setIcon(red);
    inputBox.setEditable(false);
}

```

```

    }
    elapseLikelihoodTime = System.currentTimeMillis()-startLikelihoodTime ; //Stop Timer
} //End Witten Bell smoothing

/**
 * @Glenn Smoothing
 * First of all this computes the MLE result
 * This then computes Witten-Bell smoothing as above
 * 'Zero' results in MLE calculation checked and verified as possible 16/4
 * 'Non-zero' results in MLE calculation checked and verified 16/4
 * Result from Witten-Bell section verified 17/4
 * @author Glenn Barden
 * @version Final
 */
else if (smoothing == 5){ //Start Glenn smoothing
try{
    FileInputStream fstream = new FileInputStream("DoubleCorpus.txt");
    DataInputStream in = new DataInputStream(fstream);
    BufferedReader brDictionary = new BufferedReader(new InputStreamReader(in));
    while ((line = brDictionary.readLine()) != null){
        linecount++;
        numerator=(line.split(delimiter));
        if(numerator[0].equals(w0w1)){
            numeratorInt = Double.parseDouble(numerator[1]);
            break;
        }
        else{
            numeratorInt = 0.0;
        }
    }
}
catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}

    try{
        FileInputStream fstream2 = new FileInputStream("SingleCorpus.txt");
        DataInputStream mate = new DataInputStream(fstream2);
        BufferedReader brSingleDictionary = new BufferedReader(new InputStreamReader(mate));
        while ((line2 = brSingleDictionary.readLine()) != null){
            linecount2++;
            denominator=(line2.split(delimiter));
            if(denominator[0].equals(w0)){
                denominatorInt = Double.parseDouble(denominator[1]);
                MLE = MLE*(numeratorInt/denominatorInt);
                counter++;
                break;
            }
            else{
                denominatorInt = 0.0;
            }
        }
    }
    catch (Exception e){
        System.err.println("Error: " + e.getMessage());
    }

    //NON-ZERO COUNT
    if (MLE != 0){
        String[] wordsGS = wholeSentence.toLowerCase().split(" ");

        for (int l=length-offset; l>0 ; l--){
            String GS0 = wordsGS[l-1];
            String GS1 = wordsGS[l];
            String GS0GS1 = new String(GS0+ " " +GS1);

            try{
                FileInputStream fstreamGSDouble = new FileInputStream("DoubleCorpus.txt");

```

```

        DataInputStream inGSDouble = new DataInputStream(fstreamGSDouble);
        BufferedReader brDictionaryGSDouble = new BufferedReader
(new InputStreamReader(inGSDouble));
        while ((lineGS = brDictionaryGSDouble.readLine()) != null){
            linecount++;
            String[] countGS =(lineGS.split(delimiter));
            if(countGS[0].equals(GS0GS1)){
                countValue = Double.parseDouble(countGS[1]);
                break;
            }
            else {
//No body to else loop
            }
        }
        catch (Exception e){
            System.err.println("Error: " + e.getMessage());
        }

        //N AND T VALUES
        try{
            FileInputStream fstreamNew = new FileInputStream("T.txt");
            DataInputStream inNew = new DataInputStream(fstreamNew);
            BufferedReader brDictionaryNew = new BufferedReader(new InputStreamReader(inNew));
            while ((lineGSNew = brDictionaryNew.readLine()) != null){
                linecount++;
                String[] TValue =(lineGSNew.split(delimiter));
                if(TValue[0].equals(GS0)){
                    TValueInt = Double.parseDouble(TValue[1]);
                }
                else{
//No body to else loop
                }
            }
        }
        catch (Exception e){
            System.err.println("Error: " + e.getMessage());
        }

        try{
            FileInputStream fstream2 = new FileInputStream("SingleCorpus.txt");
            DataInputStream mate = new DataInputStream(fstream2);
            BufferedReader brSingleDictionary = new BufferedReader(new InputStreamReader(mate));
            while ((line2 = brSingleDictionary.readLine()) != null){
                linecount2++;
                String[] NValue =(line2.split(delimiter));
                if(NValue[0].equals(GS0)){
                    NValueInt = Double.parseDouble(NValue[1]);
                    break;
                }
                else {
//No body to else loop
                }
            }
        }
        catch (Exception e){
            System.err.println("Error: " + e.getMessage());
        }

        GlennCalculator = GlennCalculator*(countValue/(NValueInt+TValueInt));
        if ((countValue/(NValueInt+TValueInt))<leastLikely){
            leastLikely = (countValue/(NValueInt+TValueInt));
            leastLikelyString = GS0GS1;
        }
        else{
//No body to else loop
        }
        offset++;

```

```

break;
    }
    }
    //ZERO COUNT
else{
System.out.println("ZERO LOOP");
String[] wordsGS = wholeSentence.toLowerCase().split(" ");

for (int l=length-1; l>0 ; l--){
String GS0 = wordsGS[l-1];
String GS1 = wordsGS[l];
String GS0GS1 = new String(GS0+ " " +GS1);

try{
    FileInputStream fstream = new FileInputStream("T.txt");
    DataInputStream in = new DataInputStream(fstream);
    BufferedReader brDictionary = new BufferedReader(new InputStreamReader(in));
    while((lineGS = brDictionary.readLine()) != null){
        linecount++;
        String[] TValue =(lineGS.split(delimiter));
        if(TValue[0].equals(GS0)){
            TValueInt = Double.parseDouble(TValue[1]);
            ZValueInt = V - TValueInt;
            break;
        }
        else{
            TValueInt = 0.0;
            ZValueInt = V - TValueInt;
        }
    }
}
catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}

    try{
        FileInputStream fstream2 = new FileInputStream("SingleCorpus.txt");
        DataInputStream mate = new DataInputStream(fstream2);
        BufferedReader brSingleDictionary = new BufferedReader(new InputStreamReader(mate));
        while((line2 = brSingleDictionary.readLine()) != null){
            linecount2++;
            String[] NValue =(line2.split(delimiter));
            if(NValue[0].equals(GS0)){
                NValueInt = Double.parseDouble(NValue[1]);
                break;
            }
            else {
                //No body to else loop
            }
        }
    }
}
catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}

    GlennCalculator = GlennCalculator*(TValueInt / ((ZValueInt*NValueInt)+TValueInt));
    if ((TValueInt / ((ZValueInt*NValueInt)+TValueInt))<leastLikely){
        leastLikely = (TValueInt / ((ZValueInt*NValueInt)+TValueInt));
        leastLikelyString = GS0GS1;
    }
    else{
        //No body to else loop
    }
}
break;
}
    GlennCalculator = ((GlennCalculator)/(sumOfDoubleCorpus)); //To reduce value of
Zero branch answers
}
    GlennCalculator = GlennCalculator*(length - Math.pow(1,length-1)); //To reduce

```

```

value of short strings and increase those of long strings

        if (GlennCalculator>=thresholdValueGS){ //likely
            outputBox.setText("GS: The probability of this sentence is: "+GlennCalculator+
". This sentence is likely.");
            likelihoodCheck.setEnabled(true);
                spellCheck.setEnabled(false);
                colourOutput.setIcon(green);
                inputBox.setEditable(false);
        }
        else{ //unlikely
            outputBox.setText("GS: The probability of this sentence is: "+GlennCalculator+
". This sentence is unlikely.");
            likelihoodCheck.setEnabled(true);
                spellCheck.setEnabled(false);
                colourOutput.setIcon(red);
                inputBox.setEditable(false);
        }
        elapseLikelihoodTime = System.currentTimeMillis()-startLikelihoodTime ; //Stop Timer
    }
}
}
}

//Start Application
public Application(){
    super("Spell Check and Sentence Likelihood Analyser"); //Window title

    JMenu fileMenu = new JMenu ("File"); //File menu
    fileMenu.setMnemonic ('F');
    JMenu editMenu = new JMenu ("Edit"); //Edit menu
    editMenu.setMnemonic ('E');
    JMenu toolsMenu = new JMenu ("Tools"); //Tools menu
    toolsMenu.setMnemonic ('T');
    JMenu helpMenu = new JMenu ("Help"); //Help menu
    helpMenu.setMnemonic ('H');

    //ADD ITEMS TO FILE MENU
    //OPEN
    JMenuItem openDocument = new JMenuItem ("Open...", new ImageIcon("open.png"));
    openDocument.setMnemonic ('O');
    openDocument.setAccelerator(KeyStroke.getKeyStroke("ctrl O"));
    openDocument.setToolTipText("Open a previously saved document.");
    fileMenu.add(openDocument);
    //LINE
    fileMenu.addSeparator();
    //SAVE
    JMenuItem saveDocument = new JMenuItem ("Save...", new ImageIcon("save.png"));
    saveDocument.setMnemonic ('S');
    saveDocument.setAccelerator(KeyStroke.getKeyStroke("ctrl S"));
    saveDocument.setToolTipText("Save your current document.");
    fileMenu.add(saveDocument);
    //LINE
    fileMenu.addSeparator();
    //PRINT INPUT
    JMenuItem printInput = new JMenuItem ("Print Input...", new ImageIcon("print.gif"));
    printInput.setAccelerator(KeyStroke.getKeyStroke("ctrl P"));
    printInput.setToolTipText("Print your current input.");
    fileMenu.add(printInput);
    //PRINT OUTPUT
    JMenuItem printOutput = new JMenuItem ("Print Output...");
    printOutput.setToolTipText("Print the current system output.");
    fileMenu.add(printOutput);
    //LINE
    fileMenu.addSeparator();
    //EXIT
    JMenuItem exitItem = new JMenuItem ("Exit");
    exitItem.setMnemonic ('x');

```

```
exitItem.setAccelerator(KeyStroke.getKeyStroke("alt F4"));
exitItem.setToolTipText("Close the current document, discarding any unsaved changes.");
fileMenu.add(exitItem);

//ADD ITEMS TO EDIT MENU
//UNDO
JMenuItem undoItem = new JMenuItem ("Undo", new ImageIcon("undo.png"));
undoItem.setMnemonic ('U');
undoItem.setAccelerator(KeyStroke.getKeyStroke("ctrl Z"));
undoItem.setToolTipText("Revert the last change made to the document.");
editMenu.add(undoItem);
//REDO
JMenuItem redoItem = new JMenuItem ("Redo", new ImageIcon("redo.png"));
redoItem.setMnemonic ('R');
redoItem.setAccelerator(KeyStroke.getKeyStroke("ctrl Y"));
redoItem.setToolTipText("Re-perform the last change made to the document.");
editMenu.add(redoItem);
//LINE
editMenu.addSeparator();
//CUT
JMenuItem cutItem = new JMenuItem ("Cut", new ImageIcon("cut.png"));
cutItem.setMnemonic ('t');
cutItem.setAccelerator(KeyStroke.getKeyStroke("ctrl X"));
cutItem.setToolTipText("Remove the selected text and temporarily store it in memory.");
editMenu.add(cutItem);
//COPY
JMenuItem copyItem = new JMenuItem ("Copy", new ImageIcon("copy.png"));
copyItem.setMnemonic ('C');
copyItem.setAccelerator(KeyStroke.getKeyStroke("ctrl C"));
copyItem.setToolTipText("Copy the selected text and temporarily store it in memory.");
editMenu.add(copyItem);
//PASTE
JMenuItem pasteItem = new JMenuItem ("Paste", new ImageIcon("paste.png"));
pasteItem.setMnemonic ('P');
pasteItem.setAccelerator(KeyStroke.getKeyStroke("ctrl V"));
pasteItem.setToolTipText("Insert text currently held in temporary memory into the
document.");
editMenu.add(pasteItem);
//LINE
editMenu.addSeparator();
//SELECT ALL
JMenuItem selectAllItem = new JMenuItem ("Select All");
selectAllItem.setMnemonic ('L');
selectAllItem.setAccelerator(KeyStroke.getKeyStroke("ctrl A"));
selectAllItem.setToolTipText("Select all of the input text.");
editMenu.add(selectAllItem);

//ADD ITEMS TO TOOLS MENU
//CLEAR ALL
JMenuItem clearItem = new JMenuItem ("Clear All", new ImageIcon("clear.png"));
clearItem.setMnemonic ('A');
clearItem.setToolTipText("Remove input text and output results. ");
toolsMenu.add(clearItem);
//LINE
toolsMenu.addSeparator();
//VIEW ALL MISTAKES
JMenuItem viewMistakes = new JMenuItem ("View All Spelling Mistakes",
new ImageIcon("mistakes.png"));
viewMistakes.setMnemonic ('k');
viewMistakes.setAccelerator(KeyStroke.getKeyStroke("ctrl K"));
viewMistakes.setToolTipText("View the spelling errors in the last run check of this
document.");
toolsMenu.add(viewMistakes);
//VIEW POSSIBLE TYPOS
JMenuItem viewTypos = new JMenuItem ("View Possible Typos", new ImageIcon("typo.png"));
viewTypos.setMnemonic ('o');
viewTypos.setAccelerator(KeyStroke.getKeyStroke("ctrl T"));
viewTypos.setToolTipText("View the most likely typographically wrong words in this
```

```

document.");
toolsMenu.add(viewTypos);
//LINE
toolsMenu.addSeparator();
//TIME TAKEN
JMenu timeTaken = new JMenu ("Time Taken");
timeTaken.setMnemonic ('T');
toolsMenu.add(timeTaken);
//SUBMENU IN TIME TAKEN
ButtonGroup groupTimeTaken = new ButtonGroup();
//SPELLING TIME
JMenuItem spellingTime = new JMenuItem ("View Spelling");
spellingTime.setMnemonic ('S');
spellingTime.setToolTipText("View the time taken to perform the last spell check.");
groupTimeTaken.add(spellingTime);
timeTaken.add(spellingTime);
//LIKELIHOOD TIME
JMenuItem likelihoodTime = new JMenuItem ("View Likelihood");
likelihoodTime.setMnemonic ('L');
likelihoodTime.setToolTipText("View the time taken to perform the last likelihood check.");
groupTimeTaken.add(likelihoodTime);
timeTaken.add(likelihoodTime);
//LINE
toolsMenu.addSeparator();
//CHANGE SMOOTHING
JMenu changeSmoothing = new JMenu ("Change Smoothing");
changeSmoothing.setMnemonic ('O');
toolsMenu.add(changeSmoothing);
//SUBMENU IN CHANGE SMOOTHING
ButtonGroup group = new ButtonGroup();
//Uniform Event Space
JRadioButtonMenuItem uniformEventSpace = new
JRadioButtonMenuItem("Uniform Event Space");
group.add(uniformEventSpace);
changeSmoothing.add(uniformEventSpace);
//Frequentist Event Space
JRadioButtonMenuItem frequentistEventSpace = new
JRadioButtonMenuItem("Frequentist Event Space");
group.add(frequentistEventSpace);
changeSmoothing.add(frequentistEventSpace);
//MLE
JRadioButtonMenuItem maximumLikelihoodEstimation = new
JRadioButtonMenuItem("Maximum Likelihood Estimation");
group.add(maximumLikelihoodEstimation);
changeSmoothing.add(maximumLikelihoodEstimation);
//PLUS-ONE
JRadioButtonMenuItem plusOne = new JRadioButtonMenuItem("Plus One");
group.add(plusOne);
changeSmoothing.add(plusOne);
//WITTEN-BELL
JRadioButtonMenuItem wittenBell = new JRadioButtonMenuItem("Witten Bell");
wittenBell.setSelected(true);
group.add(wittenBell);
changeSmoothing.add(wittenBell);
//GLENN-SMOOTHING
JRadioButtonMenuItem glennSmoothing = new JRadioButtonMenuItem("Glenn Smoothing");
group.add(glennSmoothing);
changeSmoothing.add(glennSmoothing);

//ADD ITEMS TO HELP MENU
//INTRODUCTION
JMenuItem introductionItem = new JMenuItem ("Introduction", new ImageIcon("help.png"));
introductionItem.setMnemonic ('I');
introductionItem.setAccelerator(KeyStroke.getKeyStroke("ctrl H"));
introductionItem.setToolTipText("View the system documentation. ");
helpMenu.add(introductionItem);
//ABOUT

```

```
JMenuItem aboutItem = new JMenuItem ("About");
aboutItem.setMnemonic ('B');
aboutItem.setToolTipText("View information about the system.");
helpMenu.add(aboutItem);

//ACTION LISTENERS
//FILE MENU
//OPEN
openDocument.addActionListener(new ActionListener(){
public void actionPerformed(ActionEvent e){
JFileChooser fc = new JFileChooser();
fc.addChoosableFileFilter(new TextFilter());
int returnVal = fc.showOpenDialog(inputBox);

try{
    String strLine;
File selectedFile = fc.getSelectedFile();
    FileInputStream in = new FileInputStream(selectedFile);
BufferedReader br = new BufferedReader(new InputStreamReader(in));
while ((strLine = br.readLine()) != null){
inputBox.setText(strLine);
}

    }
    catch(Exception ae){
        //No body to catch loop
    }
    while (returnVal == JFileChooser.APPROVE_OPTION
&& !fc.getSelectedFile().getName().endsWith(".txt")) {
        JOptionPane.showMessageDialog(inputBox, "The file " + fc.getSelectedFile()
+ " is not a supported file format.", "Error", JOptionPane.ERROR_MESSAGE);
        inputBox.setText("");
        break;
    }
}
});
//SAVE
saveDocument.addActionListener(new ActionListener(){
public void actionPerformed(ActionEvent e){
JFileChooser fc = new JFileChooser();
int returnVal = fc.showSaveDialog(inputBox);
if (returnVal == JFileChooser.APPROVE_OPTION){
    String filename = fc.getSelectedFile().getAbsolutePath();
    File file = new File(filename);
if (file.exists()){
int r = JOptionPane.showConfirmDialog(inputBox,
"Are you sure that you wish to overwrite file: \n"+filename+"?",
"Overwrite?", JOptionPane.YES_NO_OPTION);
if (r == JOptionPane.YES_OPTION){
    try{
        BufferedWriter bw = new BufferedWriter(new FileWriter(filename));
        bw.write(inputBox.getText());
        bw.flush();
        bw.close();
    }
    catch (IOException ioe){
        //No body to catch loop
    }
    }
    else{
        fc.showSaveDialog(inputBox);
    }
    }
    else{
try{
        BufferedWriter bw = new BufferedWriter(new FileWriter(filename));
        bw.write(inputBox.getText());
        bw.flush();

```

```
        bw.close();
    }
    catch (IOException ioe){
        //No body to catch loop
    }
}
}
}
});
//PRINT INPUT
printInput.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent e){
        if (inputBox.getText().equals("")){
            JOptionPane.showMessageDialog(inputBox,
                "Cannot print empty input string.", "Error", JOptionPane.ERROR_MESSAGE);
        }
        else{
            try{
                inputBox.print();
            }
            catch(Exception ae){
                //No body to catch loop
            }
        }
    }
});
//PRINT OUTPUT
printOutput.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent e){
        if (outputBox.getText().equals("")){
            JOptionPane.showMessageDialog(inputBox,
                "Cannot print empty output string.", "Error", JOptionPane.ERROR_MESSAGE);
        }
        else{
            try{
                outputBox.print();
            }
            catch(Exception ae){
                //No body to catch loop
            }
        }
    }
});
//EXIT
exitItem.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent e){
        int exit = JOptionPane.showConfirmDialog(inputBox,
            "Are you sure that you wish to exit?", "Exit?", JOptionPane.YES_NO_OPTION,
            JOptionPane.WARNING_MESSAGE);
        if (exit == JOptionPane.YES_OPTION){
            System.exit(0);
        }
        else {
            //No body to else loop
        }
    }
});
//EDIT MENU
//UNDO
undoItem.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent e){
        if (um.canUndo()){
            um.undo();
        }
        else {
            //No body to else loop
        }
    }
});
```

```
    }
    });
//REDO
redoItem.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent e){
        if (um.canRedo()){
            um.redo();
        }
        else{
            //No body to else loop
        }
    }
});
//CUT
cutItem.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent ev){
        inputBox.cut();
    }
});
//COPY
copyItem.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent ev){
        inputBox.copy();
    }
});
//PASTE
pasteItem.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent ev){
        inputBox.paste();
    }
});
//SELECT ALL
selectAllItem.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent ev){
        inputBox.selectAll();
    }
});

//TOOLS MENU
//CLEAR ALL
clearItem.addActionListener(new ActionListener(){
    public void actionPerformed (ActionEvent event){
        inputBox.setText("");
        inputBox.setEditable(true);
        spellCheck.setEnabled(true);
        likelihoodCheck.setEnabled(false);
        outputBox.setText("");
        colourOutput.setIcon(blank);
        wrongWords.clear();
        correctWords.clear();
        elapseSpellTime = 0.0;
        elapseLikelihoodTime = 0.0;
        leastLikely = 1.0;
        leastLikelyString = "";
    }
});
//VIEW ALL MISTAKES
viewMistakes.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent ev){
        if (wrongWords.isEmpty()){
            JOptionPane.showMessageDialog(inputBox,
                "There were no incorrectly spelt words on the last\nperformed check,
                or no check has taken place.", "List of Mistakes", JOptionPane.INFORMATION_MESSAGE);
        }
        else {
            JOptionPane.showMessageDialog(inputBox, "The following "+
                wrongWords.size()+" words are spelt incorrectly:\n"+wrongWords,
```

```

"List of Mistakes", JOptionPane.INFORMATION_MESSAGE);
    }
    }
    });
    //VIEW POSSIBLE TYPOS
    viewTypos.addActionListener(new ActionListener(){
        public void actionPerformed(ActionEvent ev){
            if (leastLikelyString.equals("")){ //No check performed
                JOptionPane.showMessageDialog(inputBox,
                    "No check has yet been performed.",
                    "Most Likely Typos", JOptionPane.INFORMATION_MESSAGE);
            }
            else if (leastLikelyString.equals("UES")){
                JOptionPane.showMessageDialog(inputBox,
                    "In UES all strings have the same probability.\n
                    Change smoothing to obtain the most likely typos.",
                    "Most Likely Typos", JOptionPane.INFORMATION_MESSAGE);
            }
            else if (leastLikelyString.equals("MLE")){
                JOptionPane.showMessageDialog(inputBox,
                    "Unable to obtain least likely probability in\nMLE when probability is equal to 0.",
                    "Most Likely Typos", JOptionPane.INFORMATION_MESSAGE);
            }
            else {
                JOptionPane.showMessageDialog(inputBox,
                    "The following two word string contains the lowest probability\n
                    of your entered sentence:\n \""+leastLikelyString+ "\"", "Most Likely Typos",
                    JOptionPane.INFORMATION_MESSAGE);
            }
        }
    });
    //VIEW SPELLING TIME TAKEN
    spellingTime.addActionListener(new ActionListener(){
        public void actionPerformed(ActionEvent ev){
            if (elapsedSpellTime == 0.0){
                JOptionPane.showMessageDialog(inputBox,
                    "No spell check has taken place.",
                    "Time Taken for Spelling Result",
                    JOptionPane.INFORMATION_MESSAGE);
            }
            else {
                JOptionPane.showMessageDialog(inputBox,
                    "The last spell check took "+(int)elapsedSpellTime
                    +"milliseconds (ms).", "Time Taken for Spelling Result",
                    JOptionPane.INFORMATION_MESSAGE);
            }
        }
    });
    //VIEW LIKELIHOOD TIME TAKEN
    likelihoodTime.addActionListener(new ActionListener(){
        public void actionPerformed(ActionEvent ev){
            if (elapsedLikelihoodTime == 0.0){
                JOptionPane.showMessageDialog(inputBox, "No likelihood check
                has taken place.", "Time Taken for Likelihood Result",
                JOptionPane.INFORMATION_MESSAGE);
            }
            else {
                JOptionPane.showMessageDialog
                (inputBox, "The last likelihood check took "+(int)elapsedLikelihoodTime
                +"milliseconds (ms).", "Time Taken for Likelihood Result",
                JOptionPane.INFORMATION_MESSAGE);
            }
        }
    });
    //CHANGE SMOOTHING
    //RADIO BUTTONS
    //Uniform Event Space
    uniformEventSpace.addActionListener(new ActionListener(){

```

```

        public void actionPerformed(ActionEvent ev){
            smoothing = 0;
            leastLikelyString = "";
            leastLikely = 1.0;
        }
    });
    //Frequentist Event Space
    frequentistEventSpace.addActionListener(new ActionListener(){
        public void actionPerformed(ActionEvent ev){
            smoothing = 1;
            leastLikelyString = "";
            leastLikely = 1.0;
        }
    });
    //MLE
    maximumLikelihoodEstimation.addActionListener(new ActionListener(){
        public void actionPerformed(ActionEvent ev) {
            smoothing = 2;
            leastLikelyString = "";
            leastLikely = 1.0;
        }
    });
    //PLUS-ONE
    plusOne.addActionListener(new ActionListener(){
        public void actionPerformed(ActionEvent ev){
            smoothing = 3;
            leastLikelyString = "";
            leastLikely = 1.0;
        }
    });
    //WITTEN-BELL
    wittenBell.addActionListener(new ActionListener(){
        public void actionPerformed(ActionEvent ev){
            smoothing = 4;
            leastLikelyString = "";
            leastLikely = 1.0;
        }
    });
    //WITTEN-BELL
    glennSmoothing.addActionListener(new ActionListener(){
        public void actionPerformed(ActionEvent ev){
            smoothing = 5;
            leastLikelyString = "";
            leastLikely = 1.0;
        }
    });

    //HELP MENU
    //INTRODUCTION ITEM
    introductionItem.addActionListener(new ActionListener(){
        public void actionPerformed (ActionEvent event){
            try{
                Runtime.getRuntime().exec
                ("rundll32 url.dll,FileProtocolHandler " + "UserApplicationde.pdf");
            }
            catch (Exception e){
                //No body to catch loop
            }
        }
    });
    //ABOUT
    aboutItem.addActionListener(new ActionListener(){
        public void actionPerformed(ActionEvent ev){
            JOptionPane aboutOptionPane = new JOptionPane ();
            aboutOptionPane.showMessageDialog(inputBox, "Created by
            Glenn Barden 2010", "Spell Checker and Likelihood Analyser",
            JOptionPane.PLAIN_MESSAGE, new ImageIcon("aboutIcon.gif"));
        }
    });

```

```
});

/**
 * @Create JMenuBar
 * -----
 * File Edit Tools Help
 * -----
 * @author Glenn Barden
 * @version Final
 */
JMenuBar bar = new JMenuBar ();
setJMenuBar (bar);
//Add file, edit, tools and help to JMenuBar
bar.add (fileMenu);
bar.add (editMenu);
bar.add (toolsMenu);
bar.add (helpMenu);

nameLabel = new JLabel ("Please enter your sentence below:");
inputBox = new JTextArea(2,25);
inputBox.setLineWrap(true);
//Add Undo Manager to inputBox
um = new UndoManager();
    inputBox.getDocument().addUndoableEditListener(um);
JScrollPane inputScrollPane = new JScrollPane(inputBox);
outputBox = new JTextArea ();
outputBox.setEditable(false);
outputBox.setLineWrap(true);
outputBox.setWrapStyleWord(true);
outputBox.setOpaque (false);
Font font = new Font("Arial", Font.BOLD, 12);
outputBox.setFont(font);
JScrollPane outputScrollPane = new JScrollPane(outputBox);
colourOutput.setIcon(blank);

spellCheck = new JButton ("Spell Check");
spellCheck.setMnemonic(KeyEvent.VK_S);
spellCheck.addActionListener(this);
likelihoodCheck = new JButton ("Likelihood Check");
likelihoodCheck.setMnemonic(KeyEvent.VK_L);
likelihoodCheck.addActionListener(this);
likelihoodCheck.setEnabled(false);

this.setSize(500,400);
this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

JPanel inputFlow = new JPanel();
inputFlow.setLayout (new BorderLayout());
inputFlow.add(inputScrollPane, BorderLayout.CENTER);

JPanel outputFlow = new JPanel();
outputFlow.setLayout (new BorderLayout());
outputFlow.add(outputScrollPane, BorderLayout.CENTER);
outputFlow.add(colourOutput, BorderLayout.EAST);

JPanel buttonPane = new JPanel();
buttonPane.setLayout (new FlowLayout());
buttonPane.add(spellCheck);
buttonPane.add(likelihoodCheck);

JPanel topPane = new JPanel();
topPane.setLayout(new BorderLayout());
topPane.add(nameLabel, BorderLayout.NORTH);
topPane.add(inputFlow, BorderLayout.CENTER);

JPanel bottomPane = new JPanel();
bottomPane.setLayout(new BorderLayout());
bottomPane.add(outputFlow, BorderLayout.CENTER);
```

```
bottomPane.add(buttonPane, BorderLayout.SOUTH);

JPanel whole = new JPanel();
whole.setLayout(new GridLayout(2,1));
whole.add (topPane);
whole.add (bottomPane);
this.setContentPane(whole);
}

public static void main (String [] args){
JFrame frame = new Application();
frame.getRootPane().setWindowDecorationStyle(JRootPane.FRAME);
frame.setUndecorated(true);
frame.setVisible(true);
frame.setIconImage(Toolkit.getDefaultToolkit().getImage("frameicon.png"));
}
}
//}END
```

F.1.2 TextFilter Class

```
/**
 * @(#)TextFilter.java
 *
 *
 * @author
 * @version 1.00 2010/3/24
 */

import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.io.File;
import javax.swing.JButton;
import javax.swing.JFileChooser;
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.filechooser.FileFilter;

class TextFilter extends FileFilter {

    public boolean accept(File f) {
        if (f.isDirectory())
            return true;
        String s = f.getName();
        int i = s.lastIndexOf('.');

        if (i > 0 && i < s.length() - 1)
            if (s.substring(i + 1).toLowerCase().equals("txt"))
                return true;

        return false;
    }

    public String getDescription() {
        return "*.txt";
    }
}
```

F.2 Lite System

F.2.1 Checker Class

```
/**
```

```
* @(#)Checker.java
* @author Glenn Barden
* @version Final 17/4/2010
*/

import java.awt.* ;
import java.awt.event.* ;
import java.awt.print.*;
import java.io.*;
import java.util.*;
import javax.swing.* ;
import javax.swing.filechooser.*;
import javax.swing.undo.UndoManager;

public class Checker extends GUI implements Runnable{

String enteredSentence;

Checker(String enteredSentence){
this.enteredSentence = enteredSentence;
}

    public void run() {
        //Import dictionary
        File dictionaryFile = new File("Dictionary.txt");
        StringBuffer contents = new StringBuffer();
        BufferedReader dictionaryReader = null;
        StringBuffer contents2 = new StringBuffer();
        BufferedReader reader2 = null;

        //BEGIN SPELL CHECK
        /**
         * @Pre-Condition
         * A string must be input
         * @Post-Condition
         * A result must be displayed corresponding to the input
         */

        //Read from file Dictionary.txt
        try{
            dictionaryReader = new BufferedReader(new FileReader(dictionaryFile));
            String text = null;
            while ((text = dictionaryReader.readLine()) != null){
                contents.append(text).append(System.getProperty("line.separator"));
            }
        }
        catch (FileNotFoundException e){
            e.printStackTrace();
        }
        catch (IOException e){
            e.printStackTrace();
        }
        finally{
            try{
                if (dictionaryReader != null){
                    dictionaryReader.close();
                }
            }
            catch (IOException e){
                e.printStackTrace();
            }
        }
        String dictionary = contents.toString();
        //Set input string as string without punctuation
        String enteredSentenceNoStops = enteredSentence.replace(".", "");
        String enteredSentenceNoCommas = enteredSentenceNoStops.replace(",", "");
        String enteredSentenceNoExclamation = enteredSentenceNoCommas.replace("!", "");
    }
}
```

```

String enteredSentenceNoQuestion = enteredSentenceNoExclamation.replace ("?", "");
String enteredSentenceNoSpeech = enteredSentenceNoQuestion.replace ("\"", "");
String wholeSentence = enteredSentenceNoSpeech.replace ("-", " ");
String[] words = wholeSentence.toLowerCase().split (" ");

for (int i=0; i < words.length; i++){
if (words[i].isEmpty()){
    wrongWords.add("EMPTY STRING");
}
else if (dictionary.contains(words[i])){
    correctWords.add(words[i]);
}
else{
    wrongWords.add(words[i]);
}
}

if (wrongWords.isEmpty()){ //Accept
    System.out.println("The string \"" + enteredSentence + "\" is accepted.
All words within the string are in the dictionary.");
}

else if (wrongWords.contains("EMPTY STRING")){ //Reject empty string
    System.out.println("No string has been entered.");
}

else if (wrongWords.size()==1){ //Reject single spelling mistake
    System.out.println("The string \"" + enteredSentence + "\" is rejected because there
is a spelling mistake in this document.\n" + "The word: \""+wrongWords.get(0)
+ "\" is spelt incorrectly.");
}

else{ //Reject multiple spelling mistake
    System.out.println("The string is rejected because there are "+wrongWords.size()
+" spelling mistakes in this document.\n" + "The word: \""+wrongWords.get(0)
+ "\" is the first mistake.");
}
}
}
//}END

```

F.2.2 FES Class

```

/**
 * @FES Smoothing
 * Takes each word individually and compares with the result for that word in
'SingleCorpus.txt'
 * The number retrieved is then divided by the sumOfSingleCorpus
 * 'Zero' results checked and verified 17/4
 * 'Non-zero' results checked and verified 17/4
 * @author Glenn Barden
 * @version Final
 */

import java.io.*;

public class FES extends GUI implements Runnable{

String enteredSentence;

FES(String enteredSentence){
this.enteredSentence = enteredSentence;
}

    public void run() {
        //BEGIN LIKELIHOOD CHECK
        Double FES = 1.0;
        Double sumOfSingleCorpus = 769460.0; //Sum of all values in SingleCorpus.txt
        Double numeratorIntFES = 1.0;
        Double thresholdValueFES = 0.0000000000036834356959525;
        int linecount= 0;

```

```

int length = 0;
int counterFES = 0;
String line;
    String delimiter = "%";
    String acceptedEnteredSentence = enteredSentence;
    String[] numeratorFES;
String enteredSentenceNoStops = acceptedEnteredSentence.replace(".", "");
String enteredSentenceNoCommas = enteredSentenceNoStops.replace(",", "");
String enteredSentenceNoExclamation = enteredSentenceNoCommas.replace("!", "");
String enteredSentenceNoQuestion = enteredSentenceNoExclamation.replace("?", "");
String enteredSentenceNoSpeech = enteredSentenceNoQuestion.replace("\\", "");
String wholeSentence = enteredSentenceNoSpeech.replace("-", " ");
    String[] words = wholeSentence.toLowerCase().split(" ");

    for(int i = 0; i < words.length; i++){
        length++;
    }

for (int i=length; i>1 ; i--){
    String w0 = words[i-2];
    String w1 = words[i-1];
    String w0w1 = new String(w0+ " " +w1);
    String[] wordsFES = wholeSentence.toLowerCase().split(" ");
for (int j=length-1; j>-1 ; j--){
    try{
        FileInputStream fstream3 = new FileInputStream("SingleCorpus.txt");
        DataInputStream FESCalculation = new DataInputStream(fstream3);
        BufferedReader brSingleDictionary = new BufferedReader
(new InputStreamReader(FESCalculation));
        while ((line = brSingleDictionary.readLine()) != null){
            linecount++;
            numeratorFES=(line.split(delimiter));
            if(numeratorFES[0].equals(wordsFES[j])){
                numeratorIntFES = Double.parseDouble(numeratorFES[1]);
                FES = FES*(numeratorIntFES/sumOfSingleCorpus);
                counterFES++;
                break;
            }
            else if (!numeratorFES[0].equals(wordsFES[j])) {
                leastLikelyString = w0w1;
            }
        }
    }
    catch (Exception e){
        System.err.println("Error: " + e.getMessage());
    }
}
if (FES>=thresholdValueFES && counterFES == length){ //likely
    System.out.println("FES: The probability of this sentence is: "+FES
+" . This sentence is likely.");
}
else if (counterFES != length){ //unlikely
    System.out.println("FES: The probability of this sentence is: 0.
This sentence is unlikely.");
}
else{ //unlikely
    System.out.println("FES: The probability of this sentence is: "+FES
+" . This sentence is unlikely.");
}
elapsedLikelihoodTime = System.currentTimeMillis()-startLikelihoodTime ; //Stop timer
break;
} //End frequentist event space (FES)
}
}
//}END

```

```

/**
 * @Glenn Smoothing
 * First of all this computes the MLE result
 * This then computes Witten-Bell smoothing as above
 * 'Zero' results in MLE calculation checked and verified as possible 16/4
 * 'Non-zero' results in MLE calculation checked and verified 16/4
 * Result from Witten-Bell section verified 17/4
 * @author Glenn Barden
 * @version Final
 */

import java.io.*;

public class GlennSmoothing extends GUI implements Runnable{

String enteredSentence;

GlennSmoothing(String enteredSentence){
this.enteredSentence = enteredSentence;
}

    public void run() {
        //BEGIN LIKELIHOOD CHECK
        Double numeratorInt =1.0;
        Double denominatorInt =1.0;
        Double MLE = 1.0;
        Double ZValueInt = 1.0;
Double NValueInt = 1.0;
Double TValueInt = 1.0;
Double V = 1000000.0; //Number of words in Brown Corpus
Double sumOfDoubleCorpus = 406942.0; //Sum of all values in DoubleCorpus.txt
Double WittenBellCalculator = 1.0;
Double GlennCalculator = 1.0;
Double thresholdValueGS = 0.0000000000000000000000000000000579;
Double countValue = 1.0;
int offset = 1;
int counter = 1;
int length = 0;
int linecount = 0;
int linecount2= 0;
String line;
String line2;
String lineGS;
String lineGSNew;
String[] numerator;
String[] denominator;
String delimiter = "%";
String acceptedEnteredSentence = enteredSentence;
        String enteredSentenceNoStops = acceptedEnteredSentence.replace(".", "");
String enteredSentenceNoCommas = enteredSentenceNoStops.replace(",", "");
String enteredSentenceNoExclamation = enteredSentenceNoCommas.replace("!", "");
        String enteredSentenceNoQuestion = enteredSentenceNoExclamation.replace("?", "");
String enteredSentenceNoSpeech = enteredSentenceNoQuestion.replace("\\""", "");
String wholeSentence = enteredSentenceNoSpeech.replace("-", " ");
        String[] words = wholeSentence.toLowerCase().split(" ");

        for(int i = 0; i <words.length;i++){
            length++;
        }

for (int i=length; i>1 ; i--){
        String w0 = words[i-2];
        String w1 = words[i-1];
        String w0w1 = new String(w0+ " " +w1);

        try{

```

```
FileInputStream fstream = new FileInputStream("DoubleCorpus.txt");
DataInputStream in = new DataInputStream(fstream);
BufferedReader brDictionary = new BufferedReader(new InputStreamReader(in));
while ((line = brDictionary.readLine()) != null){
    linecount++;
    numerator=(line.split(delimiter));
    if(numerator[0].equals(w0w1)){
        numeratorInt = Double.parseDouble(numerator[1]);
        break;
    }
    else{
        numeratorInt = 0.0;
    }
}
catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}

try{
    FileInputStream fstream2 = new FileInputStream("SingleCorpus.txt");
    DataInputStream mate = new DataInputStream(fstream2);
    BufferedReader brSingleDictionary = new BufferedReader(new InputStreamReader(mate));
    while ((line2 = brSingleDictionary.readLine()) != null){
        linecount2++;
        denominator=(line2.split(delimiter));
        if(denominator[0].equals(w0)){
            denominatorInt = Double.parseDouble(denominator[1]);
            MLE = MLE*(numeratorInt/denominatorInt);
            counter++;
            break;
        }
        else{
            denominatorInt = 0.0;
        }
    }
}
catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}

//NON-ZERO COUNT
if (MLE != 0){
    String[] wordsGS = wholeSentence.toLowerCase().split(" ");
    for (int l=length-offset; l>0 ; l--){
        String GS0 = wordsGS[l-1];
        String GS1 = wordsGS[l];
        String GS0GS1 = new String(GS0+ " " +GS1);

        try{
            FileInputStream fstreamGSDouble = new FileInputStream("DoubleCorpus.txt");
            DataInputStream inGSDouble = new DataInputStream(fstreamGSDouble);
            BufferedReader brDictionaryGSDouble = new BufferedReader
(new InputStreamReader(inGSDouble));
            while ((lineGS = brDictionaryGSDouble.readLine()) != null){
                linecount++;
                String[] countGS =(lineGS.split(delimiter));
                if(countGS[0].equals(GS0GS1)){
                    countValue = Double.parseDouble(countGS[1]);
                    break;
                }
                else {
//No body to else loop
                }
            }
        }
        catch (Exception e){
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

```

    }

    //N AND T VALUES
    try{
        FileInputStream fstreamNew = new FileInputStream("T.txt");
        DataInputStream inNew = new DataInputStream(fstreamNew);
        BufferedReader brDictionaryNew = new BufferedReader(new InputStreamReader(inNew));
        while ((lineGSNew = brDictionaryNew.readLine()) != null){
            linecount++;
            String[] TValue =(lineGSNew.split(delimiter));
            if(TValue[0].equals(GS0)){
                TValueInt = Double.parseDouble(TValue[1]);
            }
            else{
                //No body to else loop
            }
        }
    }
    catch (Exception e){
        System.err.println("Error: " + e.getMessage());
    }

    try{
        FileInputStream fstream2 = new FileInputStream("SingleCorpus.txt");
        DataInputStream mate = new DataInputStream(fstream2);
        BufferedReader brSingleDictionary = new BufferedReader(new InputStreamReader(mate));
        while ((line2 = brSingleDictionary.readLine()) != null){
            linecount2++;
            String[] NValue =(line2.split(delimiter));
            if(NValue[0].equals(GS0)){
                NValueInt = Double.parseDouble(NValue[1]);
                break;
            }
            else {
                //No body to else loop
            }
        }
    }
    catch (Exception e){
        System.err.println("Error: " + e.getMessage());
    }

    GlennCalculator = GlennCalculator*(countValue/(NValueInt+TValueInt));
    offset++;
    break;
}
//ZERO COUNT
else{
    String[] wordsGS = wholeSentence.toLowerCase().split(" ");

    for (int l=length-1; l>0 ; l--){
        String GS0 = wordsGS[l-1];
        String GS1 = wordsGS[l];
        String GS0GS1 = new String(GS0+ " " +GS1);

        try{
            FileInputStream fstream = new FileInputStream("T.txt");
            DataInputStream in = new DataInputStream(fstream);
            BufferedReader brDictionary = new BufferedReader(new InputStreamReader(in));
            while((lineGS = brDictionary.readLine()) != null){
                linecount++;
                String[] TValue =(lineGS.split(delimiter));
                if(TValue[0].equals(GS0)){
                    TValueInt = Double.parseDouble(TValue[1]);
                    ZValueInt = V - TValueInt;
                    break;
                }
            }
        }
    }
}

```

```
        else{
            TValueInt = 0.0;
            ZValueInt = V - TValueInt;
        }
    }
}
catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}

    try{
        FileInputStream fstream2 = new FileInputStream("SingleCorpus.txt");
        DataInputStream mate = new DataInputStream(fstream2);
        BufferedReader brSingleDictionary = new BufferedReader(new InputStreamReader(mate));
        while((line2 = brSingleDictionary.readLine()) != null){
            linecount2++;
            String[] NValue =(line2.split(delimiter));
            if(NValue[0].equals(GS0)){
                NValueInt = Double.parseDouble(NValue[1]);
                break;
            }
            else {
                //No body to else loop
            }
        }
    }
    catch (Exception e){
        System.err.println("Error: " + e.getMessage());
    }

    GlennCalculator = GlennCalculator*(TValueInt / ((ZValueInt*NValueInt)+TValueInt));
    break;
    GlennCalculator = ((GlennCalculator)/(sumOfDoubleCorpus));
    }
    GlennCalculator = GlennCalculator*(length - Math.pow(1,length-1));

    if (GlennCalculator>=thresholdValueGS){ //likely
        System.out.println("GS: The probability of this sentence is: "+GlennCalculator
+ ". This sentence is likely.");
    }
    else{ //unlikely
        System.out.println("GS: The probability of this sentence is: "+GlennCalculator
+ ". This sentence is unlikely.");
    }
    elapseLikelihoodTime = System.currentTimeMillis()-startLikelihoodTime ; //Stop Timer
}
}
}
```

F.2.4 GUI Class

```
/**
 * @(#)GUI.java
 * @author Glenn Barden
 * @version Final 17/4/2010
 */

import java.awt.* ;
import java.awt.event.* ;
import java.io.*;
import java.util.*;
import javax.swing.* ;

public class GUI extends JFrame{
    private int threadResult;
    //Initialise ArrayLists for wrong and correct words
    ArrayList<String> wrongWords = new ArrayList<String>();
```

```
ArrayList<String> correctWords = new ArrayList<String>();
//Initialise string and double for least likely pair of words
Double leastLikely = 1.0;
String leastLikelyString = "";
//Set default smoothing as Witten Bell
int smoothing = 4;
//TIMERS
//Spelling timers
double startSpellTime;
double elapseSpellTime;
//Likelihood timers
double startLikelihoodTime;
double elapseLikelihoodTime;

//Import images
protected static ImageIcon createImageIcon(String path){
    java.net.URL imgURL = GUI.class.getResource(path);
    if (imgURL != null){
        return new ImageIcon(imgURL);
    }
    else{
        System.err.println("Image not found");
        return null;
    }
}

//GUI components
JButton spellCheck;
JButton likelihoodCheck;
JTextArea inputBox;

//Start GUI
public GUI(){
    super("Spell Check and Sentence Likelihood Analyser Lite"); //Window title

    JMenu fileMenu = new JMenu ("File"); //File menu
    fileMenu.setMnemonic ('F');
    JMenu toolsMenu = new JMenu ("Tools"); //Tools menu
    toolsMenu.setMnemonic ('T');

    //ADD ITEMS TO FILE MENU
    //EXIT
    JMenuItem exitItem = new JMenuItem ("Exit");
    exitItem.setMnemonic ('x');
    exitItem.setAccelerator(KeyStroke.getKeyStroke("alt F4"));
    exitItem.setToolTipText("Close the current document, discarding any unsaved changes.");
    fileMenu.add(exitItem);

    //ADD ITEMS TO TOOLS MENU
    //CLEAR ALL
    JMenuItem clearItem = new JMenuItem ("Clear All");
    clearItem.setMnemonic ('A');
    clearItem.setToolTipText("Remove input text and output results. ");
    toolsMenu.add(clearItem);
    //LINE
    toolsMenu.addSeparator();
    //CHANGE SMOOTHING
    JMenu changeSmoothing = new JMenu ("Change Smoothing");
    changeSmoothing.setMnemonic ('O');
    toolsMenu.add(changeSmoothing);
    //SUBMENU IN CHANGE SMOOTHING
    ButtonGroup group = new ButtonGroup();
    //Uniform Event Space
    JRadioButtonMenuItem uniformEventSpace = new JRadioButtonMenuItem("Uniform Event Space");
    group.add(uniformEventSpace);
```

```
changeSmoothing.add(uniformEventSpace);
//Frequentist Event Space
JRadioButtonMenuItem frequentistEventSpace = new JRadioButtonMenuItem("Frequentist
Event Space");
group.add(frequentistEventSpace);
changeSmoothing.add(frequentistEventSpace);
//MLE
JRadioButtonMenuItem maximumLikelihoodEstimation = new JRadioButtonMenuItem("Maximum
Likelihood Estimation");
group.add(maximumLikelihoodEstimation);
changeSmoothing.add(maximumLikelihoodEstimation);
//PLUS-ONE
JRadioButtonMenuItem plusOne = new JRadioButtonMenuItem("Plus One");
group.add(plusOne);
changeSmoothing.add(plusOne);
//WITTEN-BELL
JRadioButtonMenuItem wittenBell = new JRadioButtonMenuItem("Witten Bell");
wittenBell.setSelected(true);
group.add(wittenBell);
changeSmoothing.add(wittenBell);
//GLENN-SMOOTHING
JRadioButtonMenuItem glennSmoothing = new JRadioButtonMenuItem("Glenn Smoothing");
group.add(glennSmoothing);
changeSmoothing.add(glennSmoothing);

//ACTION LISTENERS
//FILE MENU
//EXIT
exitItem.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent e){
        int exit = JOptionPane.showConfirmDialog(inputBox, "Are you sure that you wish to exit?",
        "Exit?", JOptionPane.YES_NO_OPTION, JOptionPane.WARNING_MESSAGE);
        if (exit == JOptionPane.YES_OPTION){
            System.exit(0);
        }
        else {
            //No body to else loop
        }
    }
});

//TOOLS MENU
//CLEAR ALL
clearItem.addActionListener(new ActionListener(){
    public void actionPerformed (ActionEvent event){
        inputBox.setText("");
        inputBox.setEditable(true);
        wrongWords.clear();
        correctWords.clear();
    }
});
//CHANGE SMOOTHING
//RADIO BUTTONS
//Uniform Event Space
uniformEventSpace.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent ev){
        smoothing = 0;
    }
});
//Frequentist Event Space
frequentistEventSpace.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent ev){
        smoothing = 1;
    }
});
//MLE
```

```
maximumLikelihoodEstimation.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent ev) {
        smoothing = 2;
    }
});
//PLUS-ONE
plusOne.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent ev){
        smoothing = 3;
    }
});
//WITTEN-BELL
wittenBell.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent ev){
        smoothing = 4;
    }
});
//GLENN-SMOOTHING
glennSmoothing.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent ev){
        smoothing = 5;
    }
});

//Create JMenuBar
JMenuBar bar = new JMenuBar ();
setJMenuBar (bar);
//Add file, edit, tools and help to JMenuBar
bar.add (fileMenu);
bar.add (toolsMenu);

inputBox = new JTextArea(2,2);
inputBox.setLineWrap(true);
//Add Undo Manager to inputBox
JScrollPane inputScrollPane = new JScrollPane(inputBox);

spellCheck = new JButton ("Spell Check");
spellCheck.setMnemonic(KeyEvent.VK_S);
spellCheck.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        startSpellTime = System.currentTimeMillis(); //Start timer
        String enteredSentence = inputBox.getText();
        Checker spellcheck = new Checker(enteredSentence);
        new Thread (spellcheck).start();
        elapseSpellTime = System.currentTimeMillis()-startSpellTime; //End timer
        and subtract from start time
    }
});

likelihoodCheck = new JButton ("Likelihood Check");
likelihoodCheck.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        startLikelihoodTime = System.currentTimeMillis(); //Start timer
        if (smoothing == 0){
            String enteredSentence = inputBox.getText();
            UES uescheck = new UES(enteredSentence);
            new Thread (uescheck).start();
        }
        else if (smoothing == 1){
            String enteredSentence = inputBox.getText();
            FES fescheck = new FES(enteredSentence);
            new Thread (fescheck).start();
        }
        else if (smoothing == 2){
            String enteredSentence = inputBox.getText();
            MLE mlecheck = new MLE(enteredSentence);
        }
    }
});
```

```
        new Thread (mlecheck).start();
    }
    else if (smoothing == 3){
        String enteredSentence = inputBox.getText();
        PlusOne plusonecheck = new PlusOne(enteredSentence);
        new Thread (plusonecheck).start();
    }
    else if (smoothing == 4){
        String enteredSentence = inputBox.getText();
        WittenBell wittenbellcheck = new WittenBell(enteredSentence);
        new Thread (wittenbellcheck).start();
    }
    else if (smoothing == 5){
        String enteredSentence = inputBox.getText();
        GlennSmoothing glenncheck = new GlennSmoothing(enteredSentence);
        new Thread (glenncheck).start();
    }
    elapseLikelihoodTime = System.currentTimeMillis()-startLikelihoodTime ; //Stop timer
}
} );
likelihoodCheck.setMnemonic(KeyEvent.VK_L);

this.setSize(500,400);
this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

JPanel inputFlow = new JPanel();
inputFlow.setLayout (new BorderLayout());
inputFlow.add(inputScrollPane, BorderLayout.CENTER);

JPanel buttonPane = new JPanel();
buttonPane.setLayout (new FlowLayout());
buttonPane.add(spellCheck);
buttonPane.add(likelihoodCheck);

JPanel topPane = new JPanel();
topPane.setLayout(new BorderLayout());
topPane.add(inputFlow, BorderLayout.CENTER);
topPane.add(buttonPane, BorderLayout.SOUTH);
this.setContentPane(topPane);
}

}
//}END
```

F.2.5 Main Class

```
/**
 * @(#)Main.java
 * @author Glenn Barden
 * @version Final 17/4/2010
 */

import javax.swing.* ;

public class Main extends GUI{

    public static void main (String [] args){
        JFrame frame = new GUI();
        frame.getRootPane().setWindowDecorationStyle(JRootPane.PLAIN_DIALOG);
        frame.setUndecorated(true);
        frame.setVisible(true);
        frame.setSize(500,140);
    }
}
```

F.2.6 MLE Class

```

/**
 * @MLE Smoothing
 * Takes each two word substring and compares with the result for those two words together
 * in 'DoubleCorpus.txt'
 * Then takes the first word of each two word substring and compares with the result for
 * that word in 'SingleCorpus.txt'
 * If the word is not found in the singleCorpus, there is no chance of any result apart
 * from 0 and so the counter placed here
 * ensures that if not every word is found, the result cannot be anything apart from 0
 * 'Zero' results checked and verified 11/4
 * 'Non-zero' results checked and verified 11/4
 * @author Glenn Barden
 * @version Final
 */

import java.io.*;

public class MLE extends GUI implements Runnable{

String enteredSentence;

MLE(String enteredSentence){
this.enteredSentence = enteredSentence;
}

    public void run() {
        //BEGIN LIKELIHOOD CHECK
        Double thresholdValueMLE = 0.00000000270200990978616;
        Double numeratorInt =1.0;
        Double denominatorInt =1.0;
        Double MLE = 1.0;
        int counter = 1;
        int linecount = 0;
        int linecount2= 0;
        int length = 0;
        String[] denominator;
        String line2;
        String[] numerator;
        String line;
        String delimiter = "%";
        String acceptedEnteredSentence = enteredSentence;
        String enteredSentenceNoStops = acceptedEnteredSentence.replace(".", "");
        String enteredSentenceNoCommas = enteredSentenceNoStops.replace(",", "");
        String enteredSentenceNoExclamation = enteredSentenceNoCommas.replace("!", "");
        String enteredSentenceNoQuestion = enteredSentenceNoExclamation.replace("?", "");
        String enteredSentenceNoSpeech = enteredSentenceNoQuestion.replace("\\\"", "");
        String wholeSentence = enteredSentenceNoSpeech.replace("-", " ");
        String[] words = wholeSentence.toLowerCase().split(" ");

        for(int i = 0; i < words.length; i++){
            length++;
        }

        for (int i=length; i>1 ; i--){
            String w0 = words[i-2];
            String w1 = words[i-1];
            String w0w1 = new String(w0+ " " +w1);
            try{
                FileInputStream fstream = new FileInputStream("DoubleCorpus.txt");
                DataInputStream in = new DataInputStream(fstream);
                BufferedReader brDictionary = new BufferedReader(new InputStreamReader(in));
                while ((line = brDictionary.readLine()) != null){
                    linecount++;
                    numerator=(line.split(delimiter));
                    if(numerator[0].equals(w0w1)){
                        numeratorInt = Double.parseDouble(numerator[1]);

```

```

        break;
    }
    else{
        numeratorInt = 0.0;
    }
}
}
catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}

    try{
        FileInputStream fstream2 = new FileInputStream("SingleCorpus.txt");
        DataInputStream mate = new DataInputStream(fstream2);
        BufferedReader brSingleDictionary = new BufferedReader(new InputStreamReader(mate));
        while ((line2 = brSingleDictionary.readLine()) != null){
            linecount2++;
            denominator=(line2.split(delimiter));
            if(denominator[0].equals(w0)){
                denominatorInt = Double.parseDouble(denominator[1]);
                MLE = MLE*(numeratorInt/denominatorInt);
                counter++;
                break;
            }
            else{
                denominatorInt = 0.0;
            }
        }
    }
    catch (Exception e){
        System.err.println("Error: " + e.getMessage());
    }
    if (MLE>=thresholdValueMLE && counter == length){ //likely
        System.out.println("MLE: The probability of this sentence is: "+MLE
        +". This sentence is likely.");
    }
    else if (counter != length || MLE == 0.0){ //unlikely with zero probability
        System.out.println("MLE: The probability of this sentence is: 0.
        This sentence is unlikely.");
    }
    else{ //unlikely with non-zero probability
        System.out.println("MLE: The probability of this sentence is: "+MLE
        +". This sentence is unlikely.");
    }
    elapseLikelihoodTime = System.currentTimeMillis()-startLikelihoodTime ; //Stop timer
} //End maximum likelihood estimation (MLE)
}
}
//}END

```

F.2.7 PlusOne Class

```

/**
 * @PlusOne Smoothing
 * Takes each two word substring and compares with the result for those two words together
 in 'DoubleCorpus.txt'
 * Then takes the first word of each two word substring and compares with the result for
 that word in 'SingleCorpus.txt'
 * The result here, unlike MLE can never equal zero
 * Once the checks with the corpora are perfomed, the value from the Double Corpus
 (numerator) has one added to it
 * The value from the Single Corpus (denominator) then has the number of V added to it
 * Numerator is divided by denominator to obtain the result
 * 'Zero' results checked and verified as impossible 17/4
 * 'Non-zero' results checked and verified 17/4
 * @author Glenn Barden
 * @version Final

```

```
*/

import java.io.*;

public class PlusOne extends GUI implements Runnable{

String enteredSentence;

PlusOne(String enteredSentence){
this.enteredSentence = enteredSentence;
}

    public void run() {
        //BEGIN LIKELIHOOD CHECK
        Double plusOne = 1.0;
        Double V = 1000000.0; //Number of words in Brown Corpus
        Double thresholdValuePlusOne = 0.000000000000002153;
        Double denominatorInt =1.0;
        Double numeratorInt =1.0;
        int counter = 0;
        int length = 0;
        int linecount = 0;
        int linecount2= 0;
        String[] numerator;
        String[] denominator;
        String line2;
        String line;
        String delimiter = "%";
        String acceptedEnteredSentence = enteredSentence;
        String enteredSentenceNoStops = acceptedEnteredSentence.replace(".", "");
        String enteredSentenceNoCommas = enteredSentenceNoStops.replace(",", "");
        String enteredSentenceNoExclamation = enteredSentenceNoCommas.replace("!", "");
        String enteredSentenceNoQuestion = enteredSentenceNoExclamation.replace("?", "");
        String enteredSentenceNoSpeech = enteredSentenceNoQuestion.replace("\\\"", "");
        String wholeSentence = enteredSentenceNoSpeech.replace("-", " ");
        String[] words = wholeSentence.toLowerCase().split(" ");

        for(int i = 0; i <words.length;i++){
            length++;
        }

        for (int i=length; i>1 ; i--){
            String w0 = words[i-2];
            String w1 = words[i-1];
            String w0w1 = new String(w0+ " " +w1);

            try{
                FileInputStream fstream = new FileInputStream("DoubleCorpus.txt");
                DataInputStream in = new DataInputStream(fstream);
                BufferedReader brDictionary = new BufferedReader(new InputStreamReader(in));
                while ((line = brDictionary.readLine()) != null){
                    linecount++;
                    numerator=(line.split(delimiter));
                    if(numerator[0].equals(w0w1)){
                        numeratorInt = Double.parseDouble(numerator[1]);
                    }
                    break;
                }
                else{
                    numeratorInt = 0.0;
                }
            }
        }
        catch (Exception e){
            System.err.println("Error: " + e.getMessage());
        }

        try{
```

```
        FileInputStream fstream2 = new FileInputStream("SingleCorpus.txt");
        DataInputStream mate = new DataInputStream(fstream2);
        BufferedReader brSingleDictionary = new BufferedReader(new InputStreamReader(mate));
        while ((line2 = brSingleDictionary.readLine()) != null){
            linecount2++;
            denominator=(line2.split(delimiter));
            if(denominator[0].equals(w0)){
                counter = 0;
                denominatorInt = Double.parseDouble(denominator[1]);
                plusOne = plusOne*((numeratorInt+1)/(denominatorInt+V));
                break;
            }
            else if(!denominator[0].equals(w1)){
                counter++;
                if (counter == length){
                    plusOne = plusOne*((numeratorInt+1)/(V));
                    counter = 0;
                    break;
                }
                else{
                    //No body to else loop
                }
            }
        }

        linecount2 = 0;
        counter= 0;
    }
    catch (Exception e){
        System.err.println("Error: " + e.getMessage());
    }
}

if (plusOne>=thresholdValuePlusOne){ //likely
    System.out.println("A0: The probability of this sentence is: "+plusOne
        +". This sentence is likely.");
}
else{ //unlikely
    System.out.println("A0: The probability of this sentence is: "+plusOne
        +". This sentence is unlikely.");
}
elapsedLikelihoodTime = System.currentTimeMillis()-startLikelihoodTime ; //Stop timer
} //End plus one smoothing
}
}
//}END
```

F.2.8 TextFilter Class

```
/**
 * @(#)TextFilter.java
 * @author Glenn Barden
 * @version Final 17/4/2010
 */

import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.io.File;

import javax.swing.JButton;
import javax.swing.JFileChooser;
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.filechooser.FileFilter;

class TextFilter extends FileFilter {
```

F.2.9 UES Class

154

```

        System.out.println("UES: The probability of this sentence is: "+UES
+ ". This sentence is likely.");
    }
    else{ //unlikely
        System.out.println("UES: The probability of this sentence is: "+UES
+ ". This sentence is unlikely.");
    }
    elapseLikelihoodTime = System.currentTimeMillis()-startLikelihoodTime ; //Stop timer
    break;
    //End uniform event space (UES)
}
    }
}
//}END

```

F.2.10 WittenBell Class

```

/**
 * @Witten Bell Smoothing
 * First of all this computes the MLE result
 * If MLE is zero, the zero branch is computed
 * If MLE is non-zero, the non-zero branch is computed
 * 'Zero' results in MLE calculation checked and verified as possible 17/4
 * 'Non-zero' results in MLE calculation checked and verified 17/4
 * @author Glenn Barden
 * @version Final
 */

import java.io.*;

public class WittenBell extends GUI implements Runnable{

    String enteredSentence;

    WittenBell(String enteredSentence){
        this.enteredSentence = enteredSentence;
    }

    public void run() {
        //BEGIN LIKELIHOOD CHECK
        Double numeratorInt =1.0;
        Double denominatorInt =1.0;
        Double MLE = 1.0;
        Double V = 1000000.0; //Number of words in Brown Corpus
        Double WittenBellCalculator = 1.0;
        Double ZValueInt = 1.0;
        Double NValueInt = 1.0;
        Double TValueInt = 1.0;
        Double countValue = 1.0;
        Double thresholdValueWB = 0.000000000000000119861639910954;
        int offset = 1;
        int counter = 1;
        int linecount2= 0;
        int linecount = 0;
        int length = 0;
        String line;
        String line2;
        String lineWB;
        String lineWBNew;
        String[] numerator;
        String[] denominator;
        String delimiter = "%";
        String acceptedEnteredSentence = enteredSentence;
        String enteredSentenceNoStops = acceptedEnteredSentence.replace(".", "");
        String enteredSentenceNoCommas = enteredSentenceNoStops.replace(",", "");
        String enteredSentenceNoExclamation = enteredSentenceNoCommas.replace("!", "");
        String enteredSentenceNoQuestion = enteredSentenceNoExclamation.replace("?", "");
    }
}

```

```

String enteredSentenceNoSpeech = enteredSentenceNoQuestion.replace ("\"", "");
String wholeSentence = enteredSentenceNoSpeech.replace ("-", " ");
String[] words = wholeSentence.toLowerCase().split(" ");

    for(int i = 0; i < words.length; i++){
        length++;
    }

for (int i=length; i>1 ; i--){
    String w0 = words[i-2];
    String w1 = words[i-1];
    String w0w1 = new String(w0+ " " +w1);

    try{
        FileInputStream fstream = new FileInputStream("DoubleCorpus.txt");
        DataInputStream in = new DataInputStream(fstream);
        BufferedReader brDictionary = new BufferedReader(new InputStreamReader(in));
        while ((line = brDictionary.readLine()) != null){
            linecount++;
            numerator=(line.split(delimiter));
            if(numerator[0].equals(w0w1)){
                numeratorInt = Double.parseDouble(numerator[1]);
                break;
            }
            else{
                numeratorInt = 0.0;
            }
        }
    }
    catch (Exception e){
        System.err.println("Error: " + e.getMessage());
    }

    try{
        FileInputStream fstream2 = new FileInputStream("SingleCorpus.txt");
        DataInputStream mate = new DataInputStream(fstream2);
        BufferedReader brSingleDictionary = new BufferedReader(new InputStreamReader(mate));
        while ((line2 = brSingleDictionary.readLine()) != null){
            linecount2++;
            denominator=(line2.split(delimiter));
            if(denominator[0].equals(w0)){
                denominatorInt = Double.parseDouble(denominator[1]);
                MLE = MLE*(numeratorInt/denominatorInt);
                counter++;
                break;
            }
            else{
                denominatorInt = 0.0;
            }
        }
    }
    catch (Exception e){
        System.err.println("Error: " + e.getMessage());
    }
}

/**
 * @NON-ZERO Witten-Bell Count
 * The two-word substring is compared against the DoubleCorpus
 * The first word in each substring's T value is taken from the TCorpus
 * The first word in each substring's value is taken from the SingleCorpus
 * Answer is calculated by dividing countValue by NValue plus TValueInt
 * Results checked and verified 17/4 (checked as accurate with reduced corpora)
 * @author Glenn Barden
 * @version Final
 */
if (MLE != 0){
    String[] wordsWB = wholeSentence.toLowerCase().split(" ");

```

```

for (int k=length-offset; k>0 ; k--){
String WB0 = wordsWB[k-1];
String WB1 = wordsWB[k];
String WB0WB1 = new String(WB0+ " " +WB1);

try{
    FileInputStream fstreamWBDouble = new FileInputStream("DoubleCorpus.txt");
    DataInputStream inWBDouble = new DataInputStream(fstreamWBDouble);
    BufferedReader brDictionaryWBDouble = new BufferedReader
(new InputStreamReader(inWBDouble));
    while ((lineWB = brDictionaryWBDouble.readLine()) != null){
        linecount++;
        String[] countWB =(lineWB.split(delimiter));
        if(countWB[0].equals(WB0WB1)){
            countValue = Double.parseDouble(countWB[1]);
            break;
        }
        else {
//No body to else loop
        }
    }
} catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}

//N AND T VALUES
try{
    FileInputStream fstreamNew = new FileInputStream("T.txt");
    DataInputStream inNew = new DataInputStream(fstreamNew);
    BufferedReader brDictionaryNew = new BufferedReader(new InputStreamReader(inNew));
    while ((lineWBNew = brDictionaryNew.readLine()) != null){
        linecount++;
        String[] TValue =(lineWBNew.split(delimiter));
        if(TValue[0].equals(WB0)){
            TValueInt = Double.parseDouble(TValue[1]);
        }
        else{
//No body to else loop
        }
    }
} catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}

try{
    FileInputStream fstream2 = new FileInputStream("SingleCorpus.txt");
    DataInputStream mate = new DataInputStream(fstream2);
    BufferedReader brSingleDictionary = new BufferedReader(new InputStreamReader(mate));
    while ((line2 = brSingleDictionary.readLine()) != null){
        linecount2++;
        String[] NValue =(line2.split(delimiter));
        if(NValue[0].equals(WB0)){
            NValueInt = Double.parseDouble(NValue[1]);
            break;
        }
        else {
//No body to else loop
        }
    }
} catch (Exception e){
    System.err.println("Error: " + e.getMessage());
}

WittenBellCalculator = WittenBellCalculator*(countValue/(NValueInt+TValueInt));
if ((countValue/(NValueInt+TValueInt))<leastLikely){

```

```

        leastLikely = (countValue/(NValueInt+TValueInt));
        leastLikelyString = WBOWB1;
    }
    else{
        //No body to else loop
    }
    offset++;
    break;
    }
}

/**
 * @ZERO Witten-Bell Count
 * The first word in each substring's T value is taken from the TCorpus
 * The first word's Z Value is calculated by subtracting TValue from V
 * The first word in each substring's value is taken from the SingleCorpus
 * Answer is calculated by dividing TValue by ((ZValueInt*NValueInt)+TValueInt
 * Results checked and verified 17/4 (checked as accurate with reduced corpora)
 * @author Glenn Barden
 * @version Final
 */
else{
String[] wordsWB = wholeSentence.toLowerCase().split(" ");

    for (int k=length-1; k>0 ; k--){
        String WB0 = wordsWB[k-1];
        String WB1 = wordsWB[k];
        String WBOWB1 = new String(WB0+ " " +WB1);

    try{
        FileInputStream fstream = new FileInputStream("T.txt");
        DataInputStream in = new DataInputStream(fstream);
        BufferedReader brDictionary = new BufferedReader(new InputStreamReader(in));
        while((lineWB = brDictionary.readLine()) != null){
            linecount++;
            String[] TValue =(lineWB.split(delimiter));
            if(TValue[0].equals(WB0)){
                TValueInt = Double.parseDouble(TValue[1]);
                ZValueInt = V - TValueInt;
                break;
            }
            else{
                TValueInt = 0.0;
                ZValueInt = V - TValueInt;
            }
        }
        catch (Exception e){
            System.err.println("Error: " + e.getMessage());
        }

        try{
            FileInputStream fstream2 = new FileInputStream("SingleCorpus.txt");
            DataInputStream mate = new DataInputStream(fstream2);
            BufferedReader brSingleDictionary = new BufferedReader(new InputStreamReader(mate));
            while((line2 = brSingleDictionary.readLine()) != null){
                linecount2++;
                String[] NValue =(line2.split(delimiter));
                if(NValue[0].equals(WB0)){
                    NValueInt = Double.parseDouble(NValue[1]);
                    break;
                }
                else {
                    //No body to else loop
                }
            }
        }
        catch (Exception e){
            System.err.println("Error: " + e.getMessage());
        }
    }
}

```

```
}

WittenBellCalculator = WittenBellCalculator*(TValueInt /
((ZValueInt*NValueInt)+TValueInt));
break;
    }
}

if (WittenBellCalculator>=thresholdValueWB){ //likely
    System.out.println("WB: The probability of this sentence is: "+WittenBellCalculator
+ ". This sentence is likely.");
}
else{ //unlikely
    System.out.println("WB: The probability of this sentence is: "+WittenBellCalculator
+ ". This sentence is unlikely.");
}
elapsedLikelihoodTime = System.currentTimeMillis()-startLikelihoodTime ; //Stop Timer
} //End Witten Bell smoothing
}
}
```

G Dictionary Used

This dictionary is reproduced with kind permission from <http://www.curlewcommunications.co.uk/wordlist.html>. These files are provided 'as is' without any warranty. They may be used and copied freely for non-commercial purposes provided that this notice is included. Downloading of the files signifies acceptance of these conditions.

The dictionary used within this project is stored as a text file called:

- Dictionary.txt

This file can be found in the home directory after installing either LiteInstaller.exe or FullInstaller.exe

H Corpora Used

The Corpora used within this project are stores as three text files called:

- SingleCorpus.txt
- DoubleCorpus.txt
- T.txt

These files can be found in the home directory after installing either LiteInstaller.exe or FullInstaller.exe

I User Guide

The system User Guides are stored as two PDF files (one for the full system and the other for the lite system) called:

- `UserGuideFull.pdf`
- `UserGuideLite.pdf`

These files can be found in the following directory:
`/FinalYearProject/UserGuides.zip/`

J Bibliography

- [1] Global Oneness, http://www.experiencefestival.com/a/Spell_checker_-_History/id/5479557. A History of Spell Checkers, 2008. [accessed 20/04/2010].
- [2] Michael Petrie, <http://www.wordstar.org/wordstar/history/history.htm>. A Potted History of WordStar, 2006. [accessed 20/04/2010].
- [3] Ed Foster, <http://www.infoworld.com/d/adventures-in-it/how-did-wordperfect-go-wrong-478>. How Did WordPerfect go Wrong?, Info World, 2007. [accessed 20/04/2010].
- [4] Andrew R. Golding and Dan Roth, *A Winnow-Based Approach to Context-Sensitive Spelling Correction*. Springer Netherlands, Volume 34, 1999.
- [5] Jerry Zar, <http://www.bios.niu.edu/zar/poem.pdf>. Candidate for a Pullet Surprise, 1992. [accessed 20/04/2010].
- [6] British Broadcasting Corporation, <http://news.bbc.co.uk/1/hi/8627335.stm>. Cook-book misprint costs Australian publishers dear, 2010. [accessed 20/04/2010].
- [7] British Broadcasting Corporation, <http://news.bbc.co.uk/1/hi/world/americas/5219220.stm>. Spellchecker firm corrects typo, 2006. [accessed 20/04/2010].
- [8] Daily Mail Newspaper, <http://www.dailymail.co.uk/news/article-1262859/Workmens-blunder-mix-letters-road-marking-spell-Keer-Cleap.html>. Workmen's blunder after they mix up letters on road marking to spell 'Keer Cleap', 2010. [accessed 20/04/2010].
- [9] Typo Eradication Advancement League, http://www.greattypohunt.com/index.php?option=com_content&view=article&id=1&Itemid=9. TEAL's History, 2009. [accessed 20/04/2010].
- [10] Google Wave, <http://wave.google.com/about.html>. About Google Wave, 2010. [accessed 20/04/2010].
- [11] Thierry Fontenelle, <http://blogs.msdn.com/correcteurorthographiqueoffice/archive/2006/06/05/contextual-spelling-in-the-2007-microsoft-office-system.aspx>. Contextual spelling in the 2007 Microsoft Office system, 2006. [accessed 20/04/2010].
- [12] Wikipedia, http://en.wikipedia.org/wiki/Wikipedia:Lists_of_common_misspellings/Grammar_and_Misc. Wikipedia:Lists of common misspellings/Grammar and Misc, 2010. [accessed 20/04/2010].

-
- [13] Daniel Jurafsky and James H. Martin, *Speech and Language Processing*. Prentice Hall, 2000.
- [14] Stanley F. Chen and Joshua Goodman, *An Empirical Study of Smoothing Techniques for Language Modeling*. Santa Cruz, California, 1996.
- [15] Bill MacCartney, <http://nlp.stanford.edu/wcmac/papers/20050421-smoothing-tutorial.pdf>. NLP Lunch Tutorial, Stanford University, 2005. [accessed 20/04/2010].
- [16] Henry Kucera and W. Nelson Francis, <http://www.edict.com.hk/lexiconindex/frequencylists/words2000.htm>. Brown Corpus, 1967. [accessed 20/04/2010].
- [17] Henry Kucera and W. Nelson Francis, http://cogsci.uni-osnabrueck.de/severt/SIGIL/sigil_R/data/brown_bigrams.tbl. Brown Corpus, 1967. [accessed 20/04/2010].
- [18] Real's How To..., <http://www.rgagnon.com/javadetails/java-0014.html>. Real Gagnon, Quebec Canada, 2002. [accessed 20/04/2010].
- [19] Elliotte Rusty Harold, <http://www.cafeaulait.org/course/week2/02.html>. Java's Primitive Data Types, 1998. [accessed 20/04/2010].
- [20] Glenn S. Barden, <http://glennbarden.webs.com/downloads.htm>. Statistical Sentence Plausibility Checker, 2010. [accessed 20/04/2010].
- [21] Mary Jo Foley, <http://blogs.zdnet.com/microsoft/?p=716>. Is Microsoft Office 2007 outselling Windows Vista?, 2007. [accessed 20/04/2010].
- [22] A. Dix, J. Finlay, G. Abowd, R. Beale, *Human Computer Interaction*. Prentice Hall, 1993.
- [23] J. Nielsen and R. Molich, *Heuristic Evaluation of User Interfaces Methodology. Proceedings of CHI'90 Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 1990.
- [24] British Broadcasting Corporation, <http://news.bbc.co.uk/1/hi/8306631.stm>. Berners-Lee sorry for slashes, 2009. [accessed 20/04/2010].
- [25] British Computing Society, <http://www.bcs.org/server.php?show=nav.6030>. Code of Conduct, 2006. [accessed 20/04/2010].
- [26] British Computing Society, <http://www.bcs.org/server.php?show=conWebDoc.1589>. Code of Good Practise, 2004. [accessed 20/04/2010].